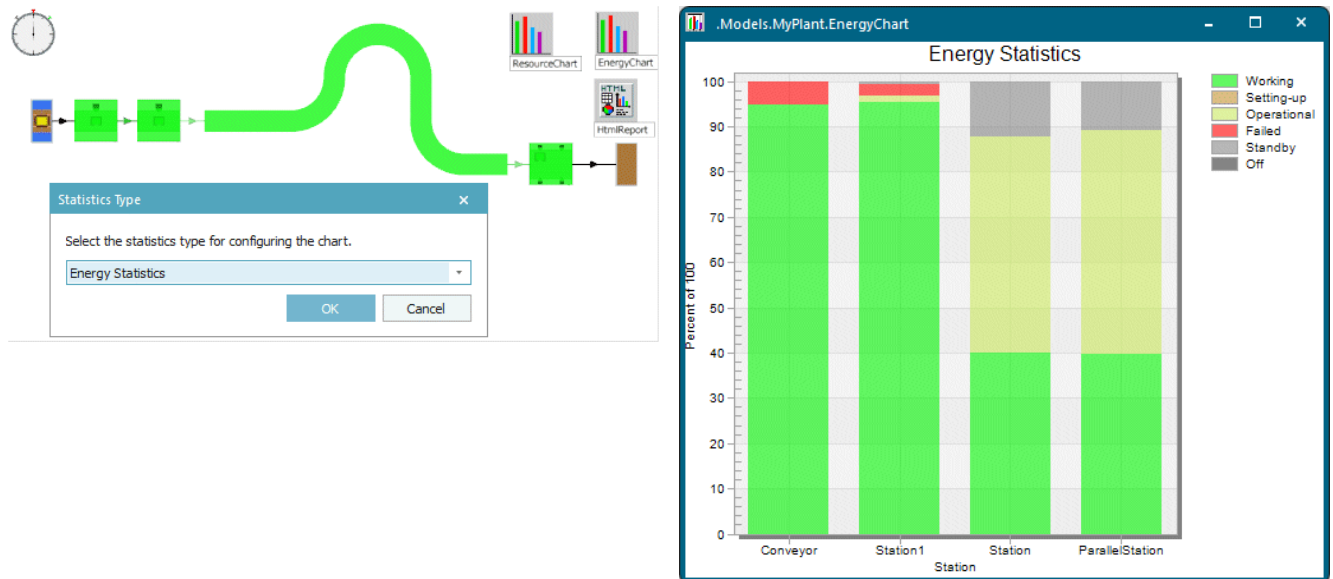




Note

The energy states of the material flow objects differ from the resource states with the same name. The values for the resource states refer to the statistics collection period, while the values for the energy states refer to the total energy consumption.

Back to [Simulate the Power Consumption in Your Facility](#)

Modeling Workers and the Jobs They Do

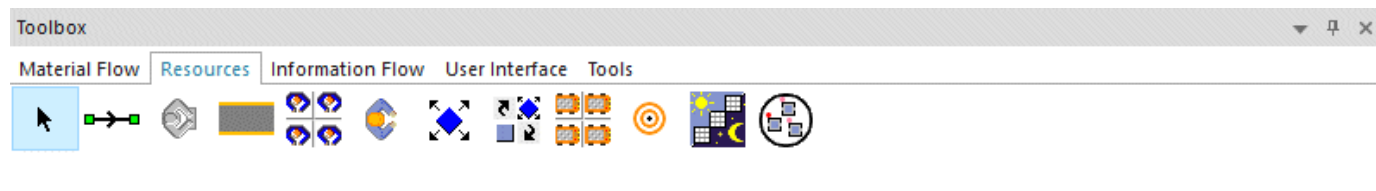
Model Workers and the Jobs They Do

When you simulate your plant, you also have to model *Workers*, who work at the machines or who repair machines.

Define the settings on the [Tab Importer](#) or on the [Sub-tab Failure](#). While the [Worker](#) works at the machine or while he repairs the machine, he stays on a [Workplace](#) at the associated machine. While the *Worker* is [Waiting](#) for a job, he stays in the [WorkerPool](#), which might be the lounge or the staff room of your plant. When the *Worker* has to perform one or several jobs at a machine, the foreman ([Broker](#)) tells him to do so. The *Worker* then walks on a [FootPath](#) from the *WorkerPool* to the *Workplace* of the respective machine and does his job. The *Worker* can also [walk freely](#) within the model without being bound to a network of *FootPaths*.

You can also use the [Broker](#) and the [Exporter](#) to model *Workers* and the tools they need to do their jobs. As a rule, you will use the *Worker*, if the time it takes him to get to a machine is of importance to you. In addition, the *Worker* will be animated on the *FootPath* and the *Workplace* while he walks to the station and works there. You will use *Importer*, *Broker* and *Exporter* if producing a part requires shared resources.

You can insert the required objects into your simulation model from the folder **Resources** in the *Class Library* or from the toolbar **Resources** in the *Toolbox*.



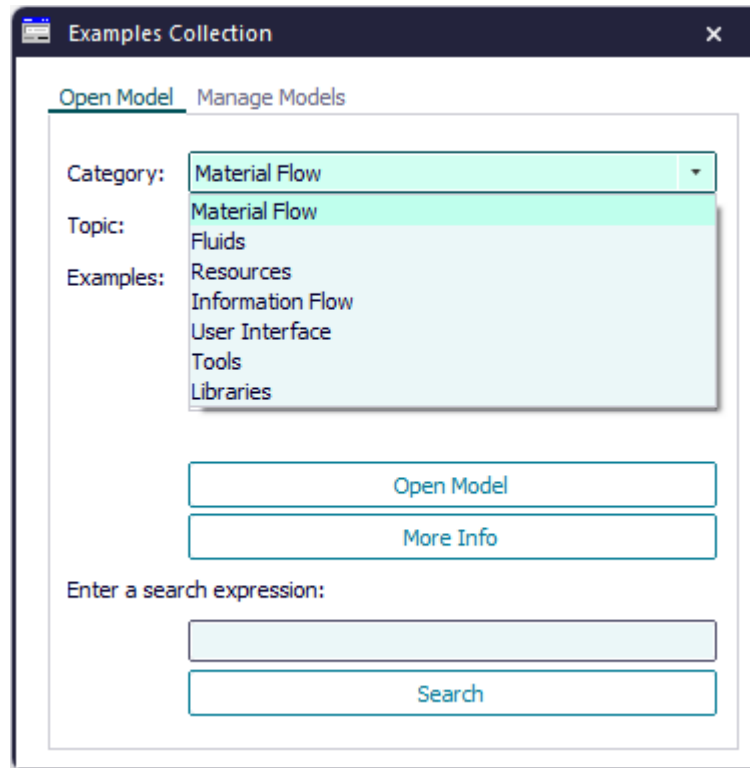
In our examples we will:

- **Model a Worker Who Works at a Machine**
- **Model a Worker Who Repairs a Machine**
- **Model a Worker Who Carries Parts Between Workplaces**
- **Model a Worker Who Walks Freely**
- **Model a Worker Who Waits in Line**
- **Model a Worker Who Returns to the Home Location**
- **Set How Many Workers Are Created During Initialization**
- **Model Workers with Importer, Broker, and Exporter**
- **Set How Workers Deliver Mounting Parts**

Note

If your *Worker* does not walk in your model, make sure that the importer on the tab **Importer** of the station is activated, on whose attached *Workplace* the *Worker* is to work.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Back to [Modeling the Material Flow](#)

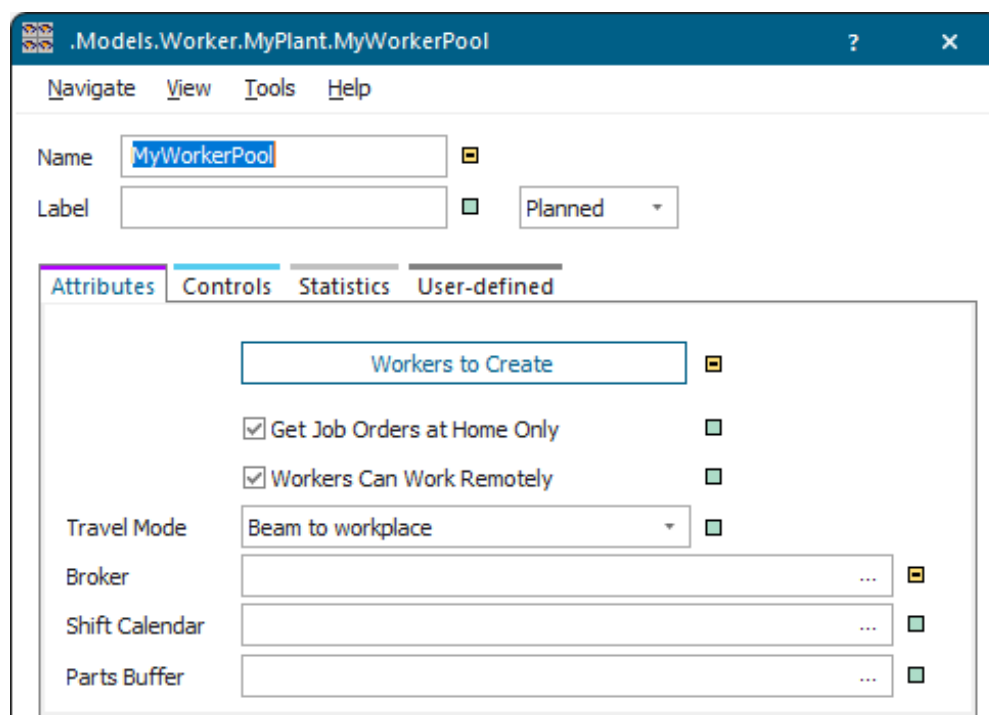
Model a Worker Who Works at a Machine

You can model a *Worker* who works at a machine.

Proceed as follows:

- Configure the *WorkerPool*. This is the place where *Plant Simulation* creates the **Workers** and where they stay, when they do not work and are **Waiting** for a job order.
 - Insert the object *WorkerPool*  into your model.

Deactivate inheritance of the **Workers to Create** table: Click the check box, so that it looks like this . Open the table.

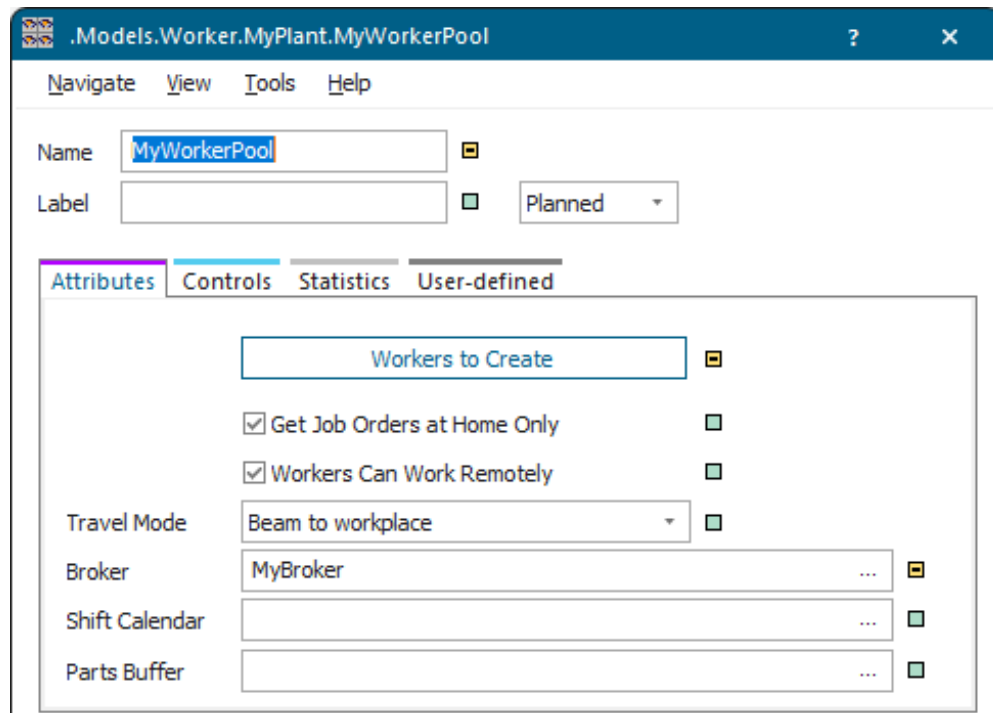


Add the path of the *Worker* class **.Models.Worker.myWorker** to the **Workers to Create** table.

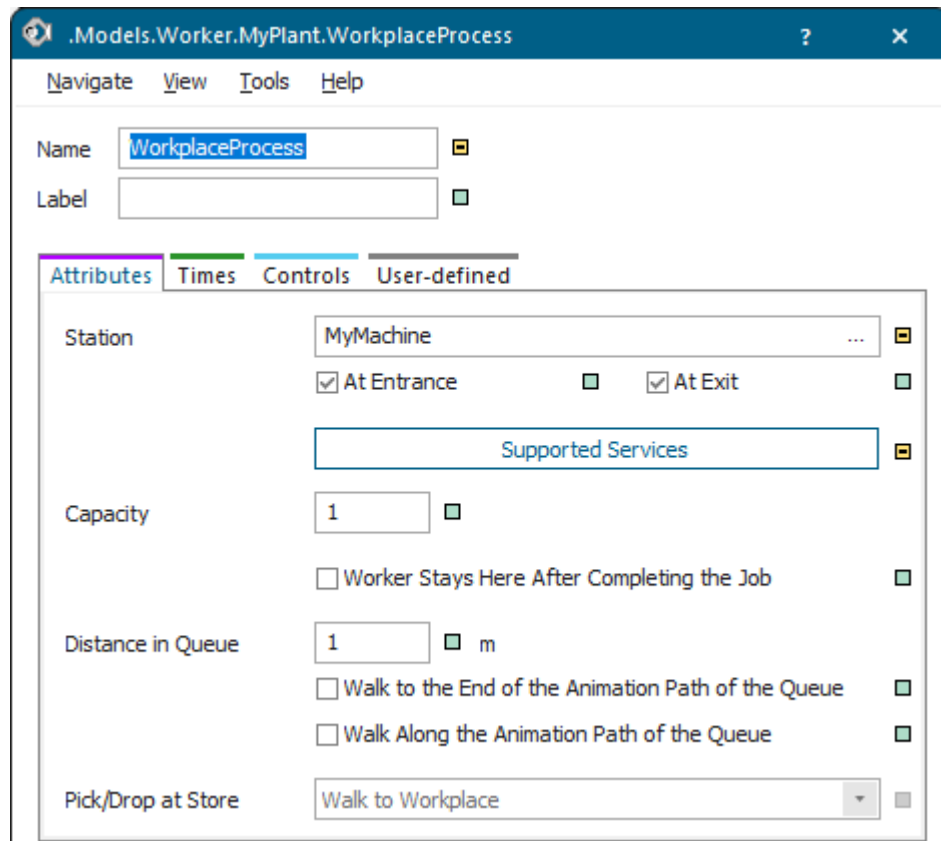
.Models.Worker.myWorker								
	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	.Models.Worker.myWorker	1					-	-

- Insert the object *Broker* into your model. He manages all of the *Workers* in your model.

Click the **Browse** button and add the *Broker* to the *WorkerPool* in the dialog **Select Object**.



- Insert a *Workplace* into our model and attach it to the machine at which the *Worker* performs his job. Click the *WorkPlace*  on the toolbar **Resources** and place it next to the machine.
 - Drag the machine, in our example the *Station MyMachine*, onto the dialog of the *WorkPlace* and drop it. This adds the machine to the text box **Station** in the dialog of the *WorkPlace*.

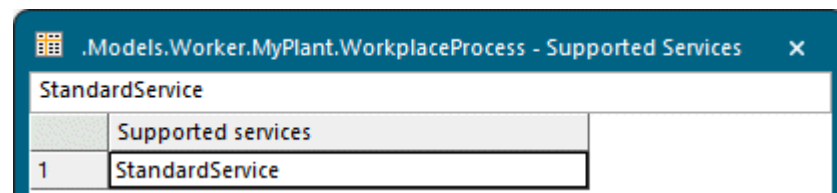


- Deactivate inheritance of the table **Supported Services**: Click the check box so that it looks like this . Open the table. Type in StandardService.

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

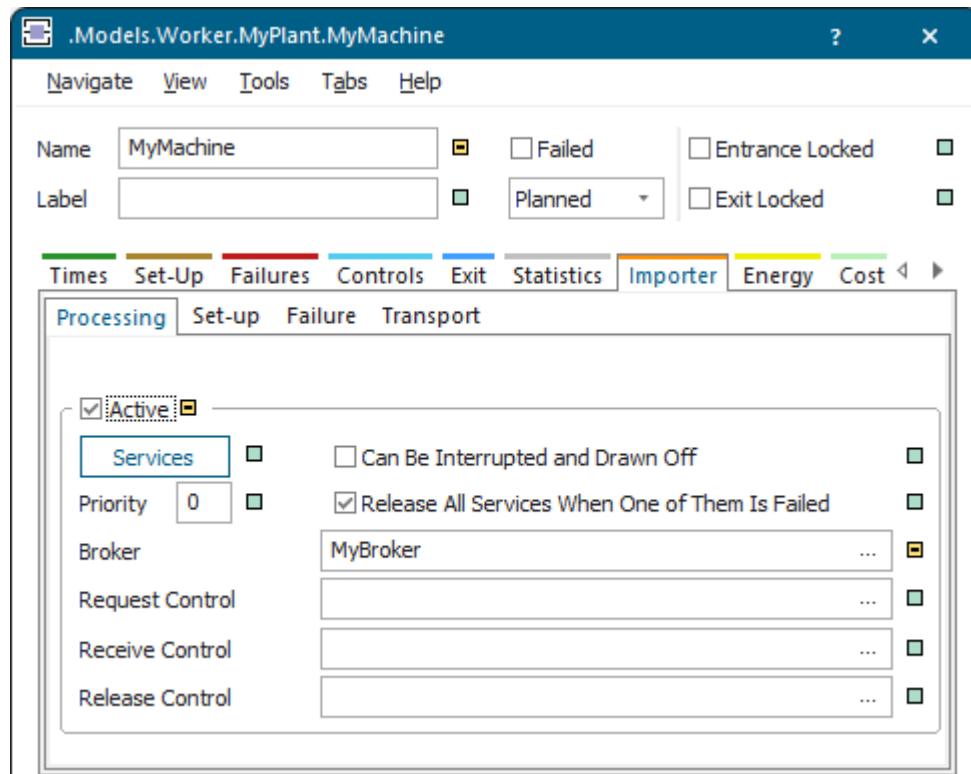
To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare [Relational Operators](#).

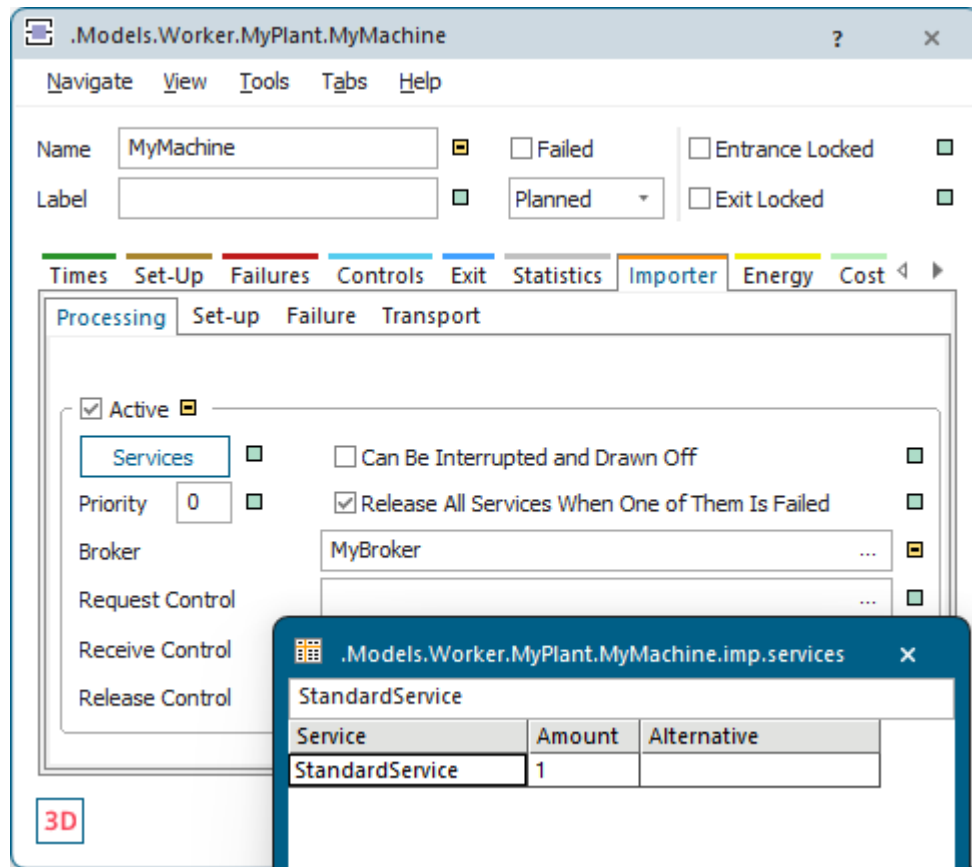


- Then, we have to tell the machine that it asks for the *Worker* to do a job: Select the check box **Active** on the sub-tab **Processing** of the tab **Importer** in the dialog of the machine.

Click the **Browse** button and add the path of the *Broker* to the sub-tab **Processing** of the tab **Importer** in the dialog **Select Object**.



- Next, we will insert the *FootPath* on which the *Worker* walks from the *WorkerPool* to the *WorkPlace*. Finally, we have to connect the *WorkerPool* and the *FootPath* with a *Connector*.
- Open the *EventController* and reduce the simulation speed. Then, start the simulation. The *Worker* walks on the *FootPath* from the *WorkerPool* to the *WorkPlace*, works on a single part, and then returns to the *WorkerPool*. This is because he performs **1 StandardService**.



You can also click the tab **Statistics** and view the most important data, which the object collected.

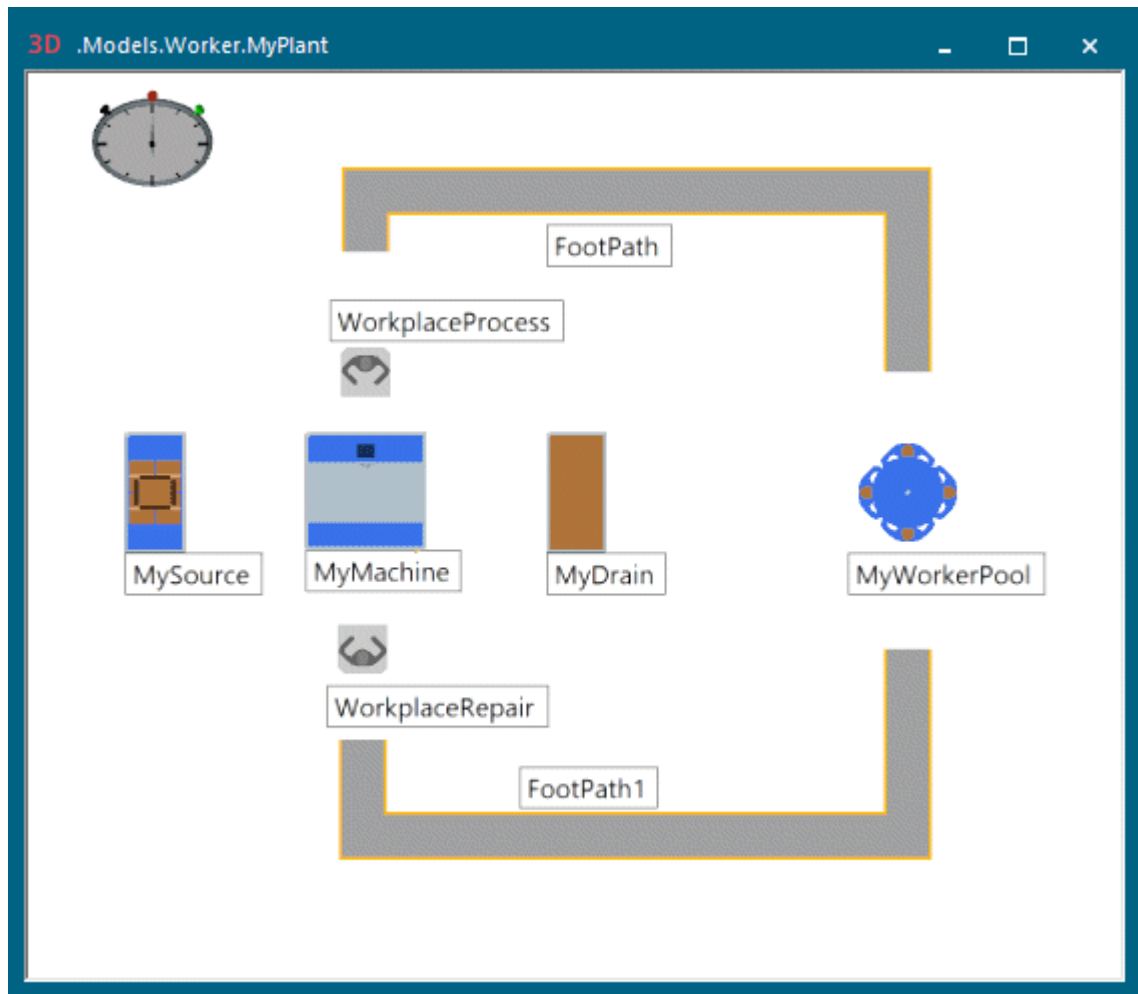
Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Repairs a Machine

You can model a *worker* who repairs a machine.

To model a **Worker**, who repairs a machine, we continue modeling with the simulation model we created above.

- Insert a *Footpath* from the *WorkerPool* to the machine on which the *repair worker* gets to it. Connect *Footpath* and *WorkerPool*.



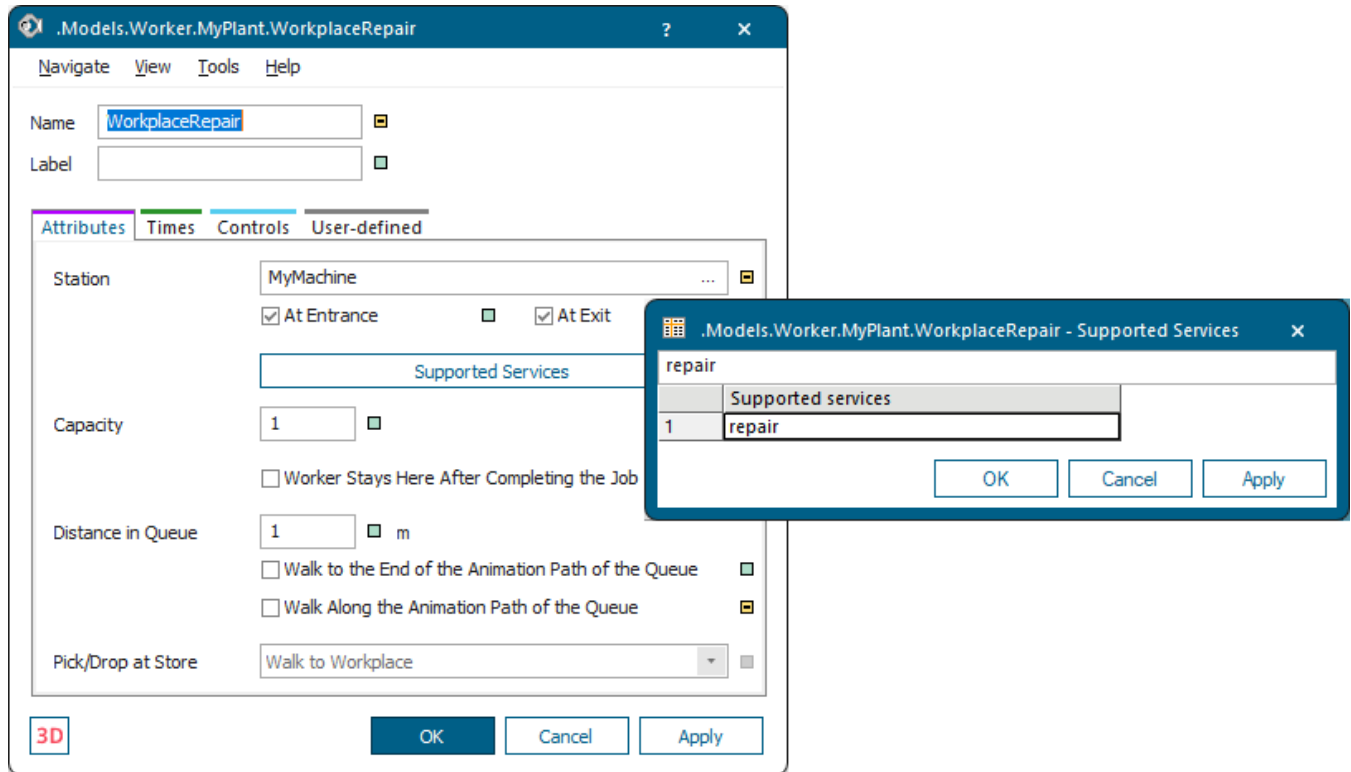
- Insert a *Workplace* on which the *Worker* stays, when he repairs the machine.

Turn inheritance off: Click the check box, so that it looks like this . Open the table **Supported Services**. Type in the service **repair**.

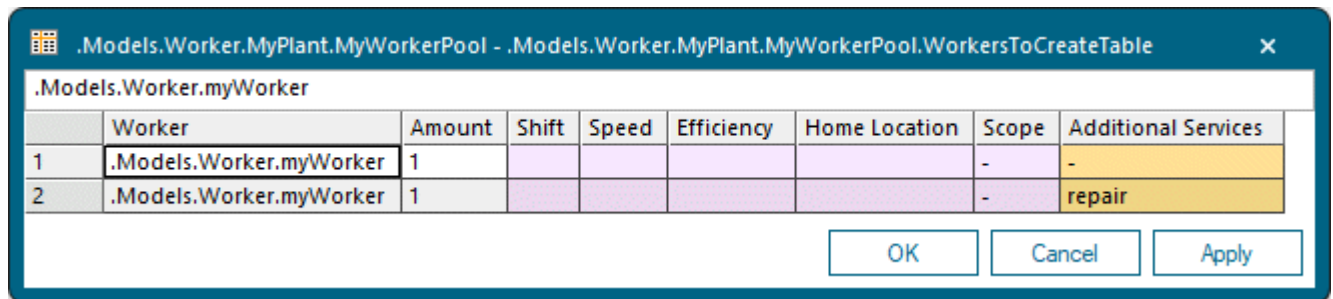
The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

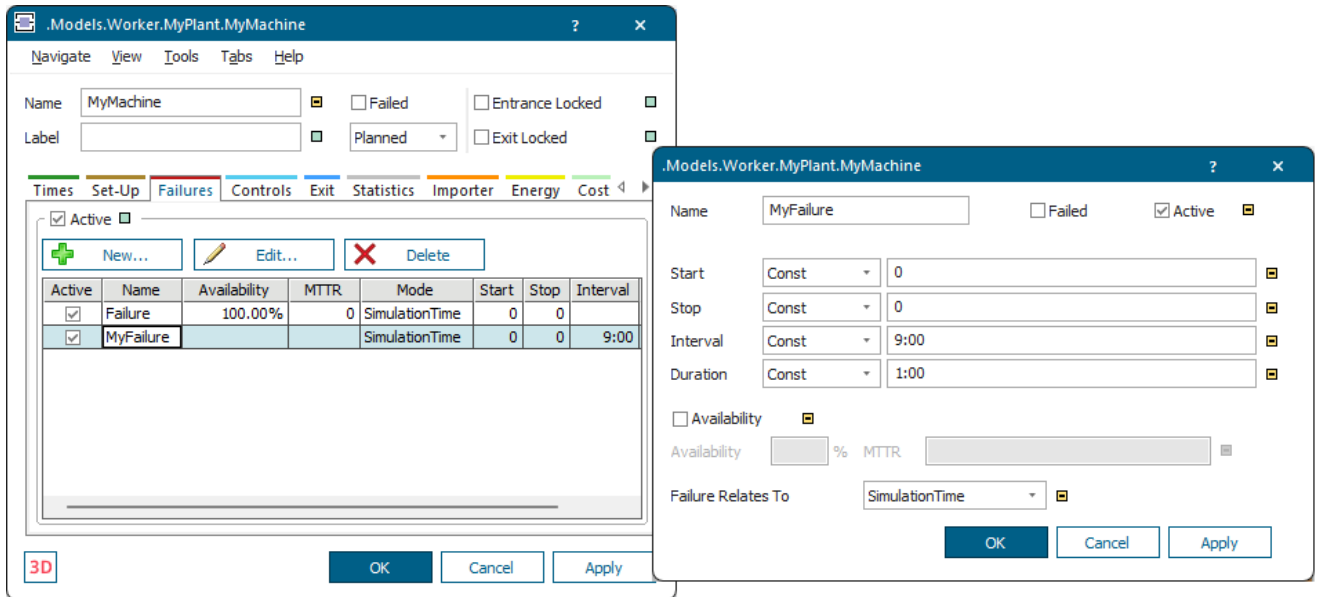
In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare [Relational Operators](#).



- Add a second *Worker* to the **Workers to Create** table of the *WorkerPool*: Right-click the table and select **Append Row** on the context menu. Drag the *Worker* from the *Class Library* to the cell **Worker**. Enter **repair** into the cell below **Additional Services**.

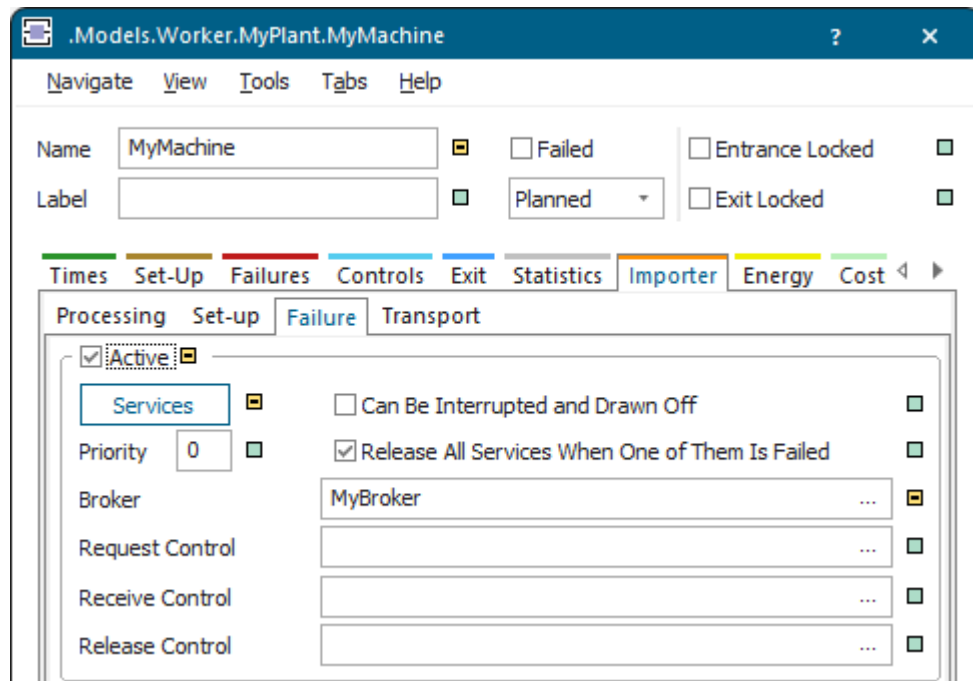


- Then, configure the machine, so that it creates failures and requests a *Worker* for fixing this failure.
 - Select the check box **Active** on the tab **Failures** in the dialog of the machine. Type **9**: into the text box **Interval**; this is the mean time between failures. Type **1**: into the text box **Duration**; this is the mean time to repair.



- Select the check box **Active** on the sub-tab **Failure** of the **Tab Importer**.

Click the  button and add the path of the *Broker* to the sub-tab **Failure** in the dialog **Select Object**.

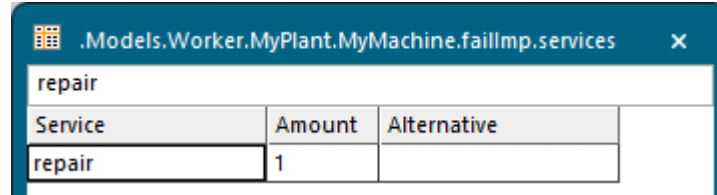


Deactivate inheritance: Click the **inheritance** check box, so that it looks like this , and open the table **Services for Repairing**. Replace the **StandardService** with the service repair.

The name of the service is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such an case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare **Relational Operators**.



The screenshot shows a window titled ".Models.Worker.MyPlant.MyMachine.failImp.services" with a close button. Inside, there is a table with the following data:

Service	Amount	Alternative
repair	1	

- Open the *EventController* and reduce the simulation speed. Then, start the simulation. You will see how the *Worker* on the *Workplace* at the top of the machine works on the part and how the *Worker* on the bottom *Workplace* repairs the machine, when it fails.

You can also click the tab **Statistics** and view the most important data, which the object collected.

Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Carries Parts Between Workplaces

Model a Worker Who Carries Parts Between Workplaces

The *Worker* can pick up and carry parts, which are ready to leave the station, to the destination object, and place them there. While the *Worker* is loading parts, he is brokered to the station. Procurement ends as soon as the *Worker* leaves the *Workplace* to carry the part to the **target**. While the *Worker* carries parts, the service still is assigned, but no longer brokered.

If you would like to define the **Loading Time** and/or the **Unloading Time** which the **Worker** 🧑 needs to pick the parts up at a station and to place them onto another station, you can enter them into the **Workplace** that is attached to the station.

The *Worker* evaluates the *part targets*, which you entered on the sub-tab **Transport**, and walks to the closest one first. At the target station, he deposits all parts destined for this station. If he carries additional parts, he evaluates the next target and walks to the closest one.

At the end of his shift the *Worker* walks back to his *WorkerPool* and deposits the parts he could not deliver in the **Parts Buffer**, which you assigned to the *WorkerPool*. If you did not assign a *parts buffer* to the *WorkerPool*, *Plant Simulation* shows a warning and stops the simulation.

If you activated the *transport importer* in the *parts buffer*, one of the available *Workers* carries the parts in the *parts buffer* to the target object, after the next shift has started. If you did not activate the *transport importer*, the parts in the *parts buffer* are moved on according to the **exit strategy** which you selected for the *parts buffer*.

You can, for example:

- [Model a Worker Who Walks on FootPaths](#)
- [Model a Worker Who Carries Several Parts](#)

- [Model a Worker Who Waits for a Free Target](#)

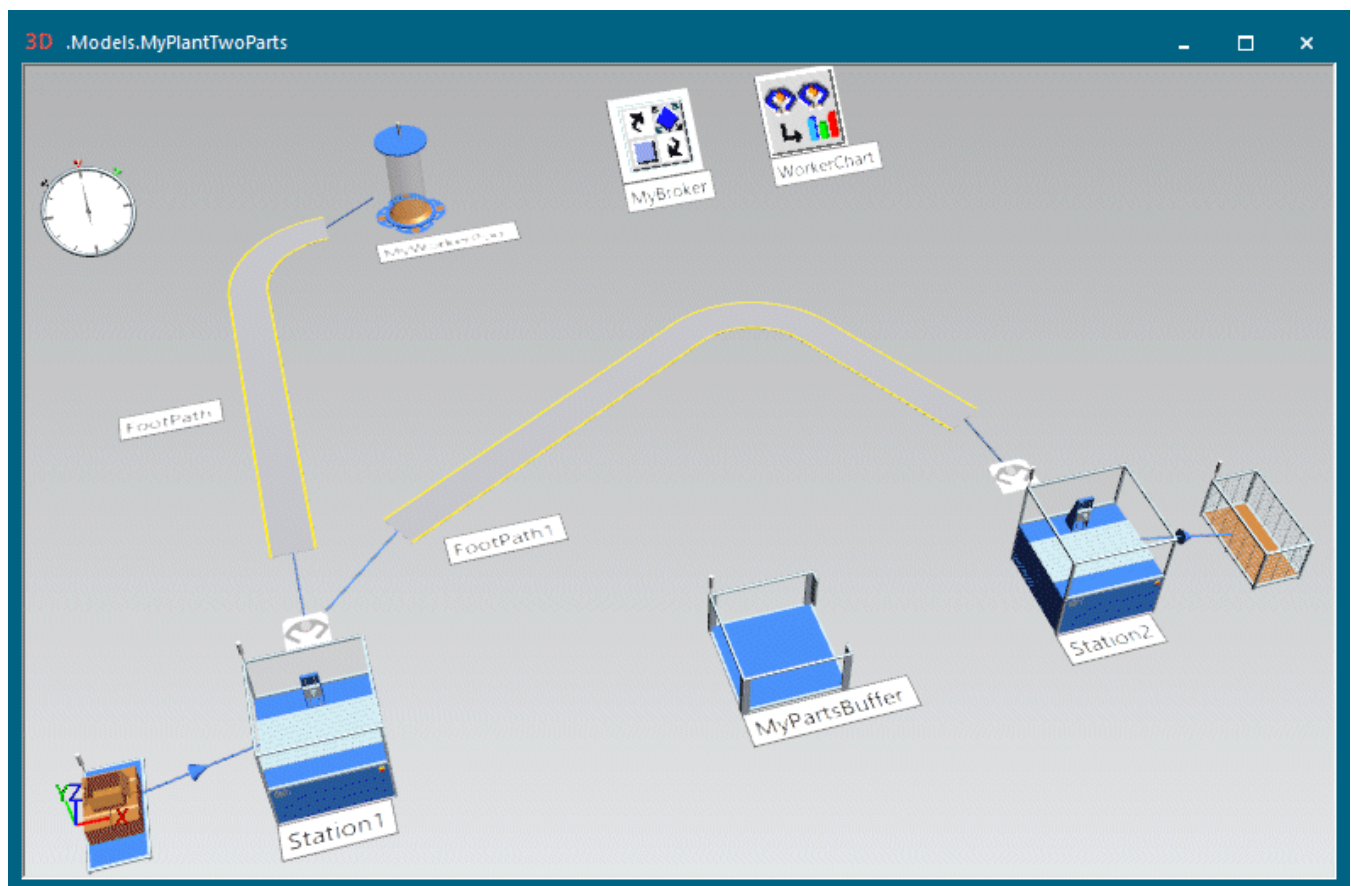
Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Walks on FootPaths

You can model a *Worker* who carries parts from *Workplace* to *Workplace* on *FootPaths*.

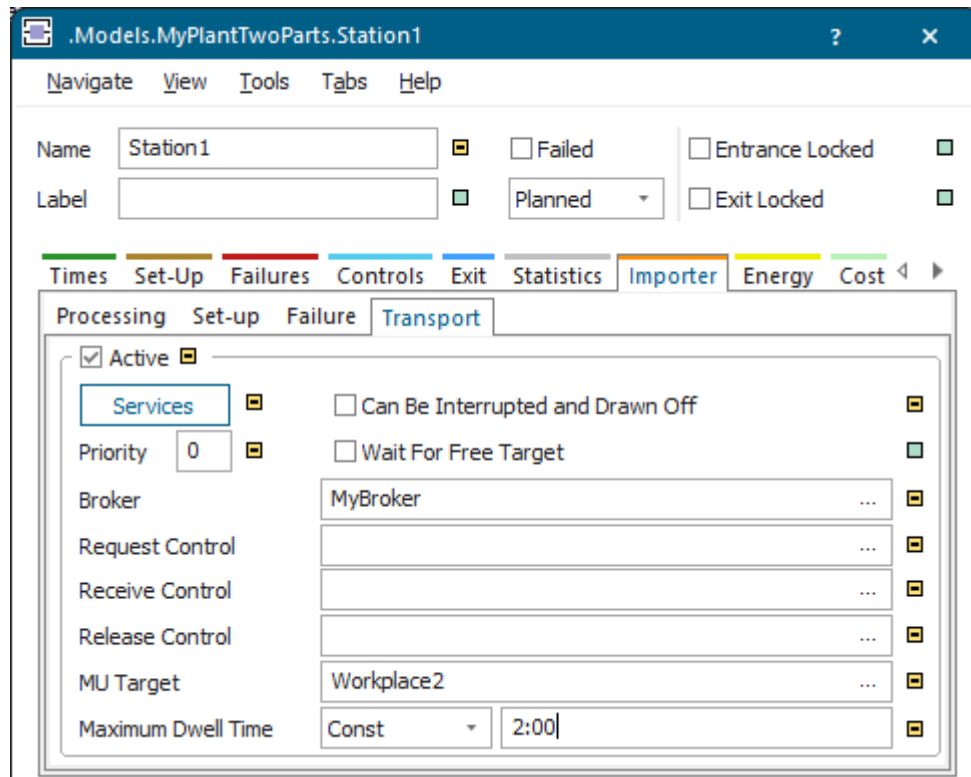
Proceed as follows:

- Attach a **Workplace**  to the work station at which the *Worker* picks up the part.
- Attach a *Workplace* to the target station at which he places the part. To automatically enter the **Station**, drag the *Workplace* close to one of the sides of the material flow object, which you want to use as the **Station**.
- Connect the *Workplaces* with a *Footpath*. The **Worker**  walks on this footpath from one *Workplace* to the next.
- Connect the **WorkerPool**  and the *Workplace*, at the work station at which the *Worker* picks up the part, with a *Footpath*.



- Activate the **transport importer** on the tab **Importer** of the station, which is ready to send the part on. The part then requests a *Worker*. The parts wait at the station until the *Worker* arrives and picks them up.

Select the *Broker* who manages the *Workers*. We selected *MyBroker*.

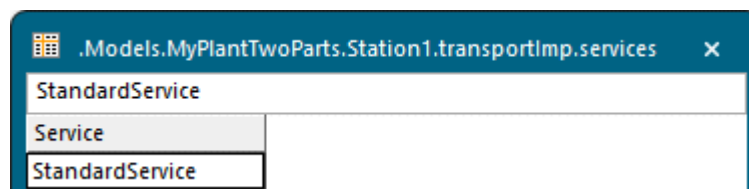


Enter the names of the **Services** which the *Broker* manages and which the *transport-importer* provides into the **services** table. We accepted the default setting, namely the *StandardService*.

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

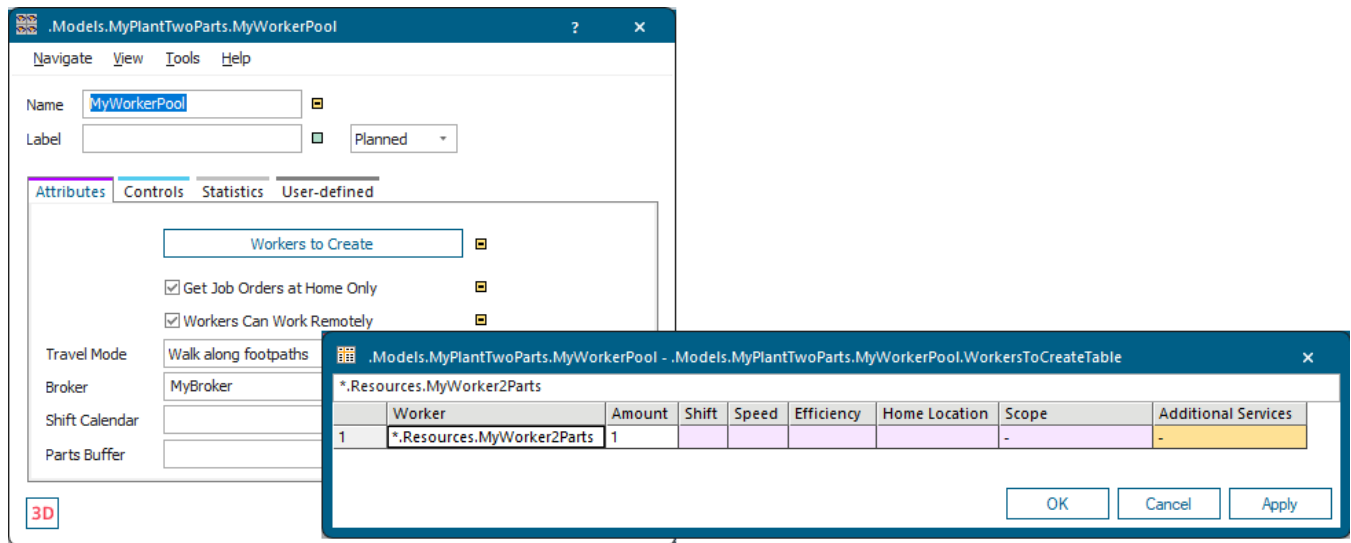
In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare **Relational Operators**.



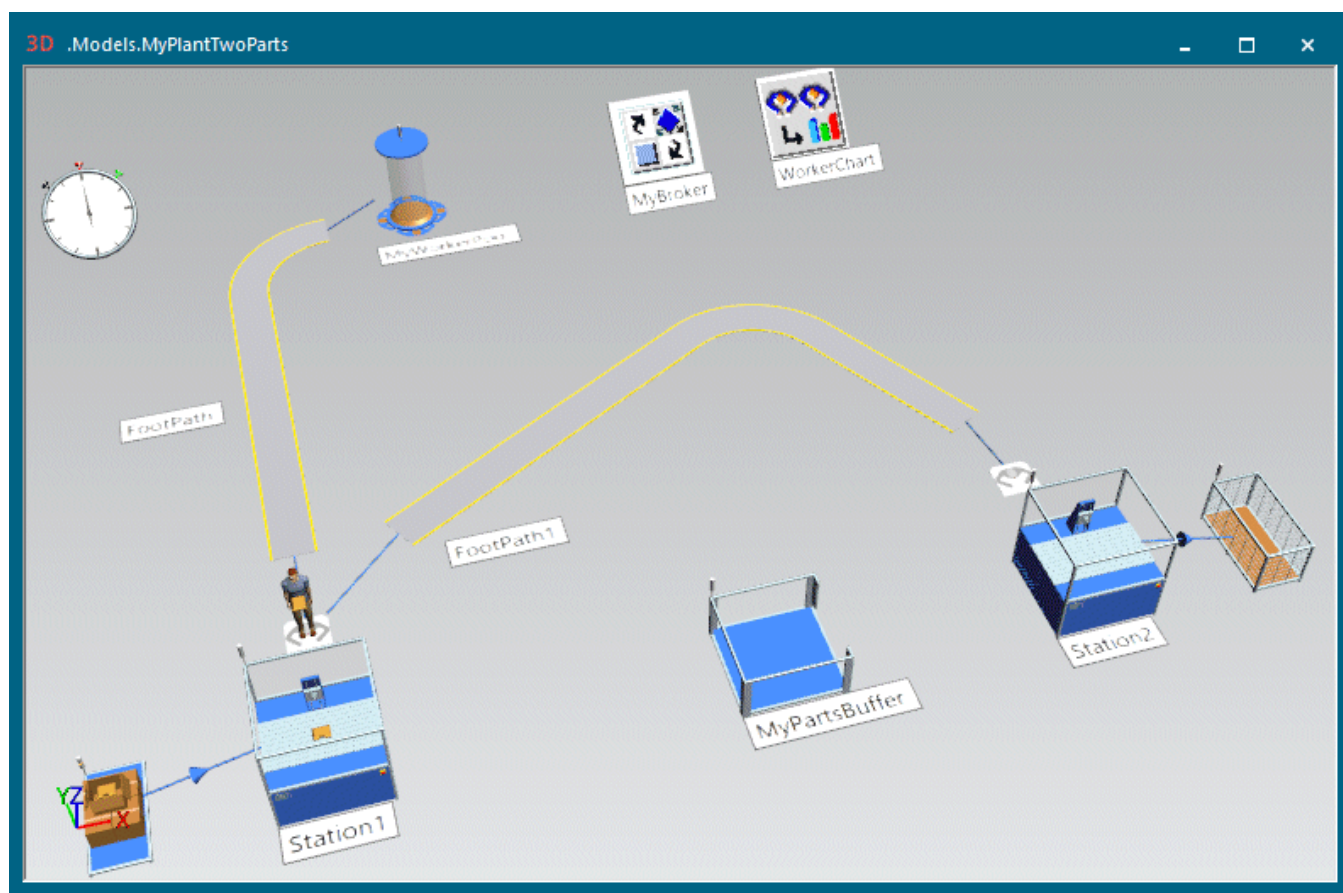
Select the **MU Target**, i.e., the target station, at which the *Worker* deposits the parts. We selected *Workplace2*.

To prevent the *Worker* from waiting too long for additional parts to arrive, you have to enter the **Maximum Dwell Time**. After it has passed, he walks to the target station, the **MU Target**.

- Select the *Worker*, who carries the parts from station to station in the *WorkerPool*. We selected the *Worker* named *MyWorker* which we created by duplicating the built-in *Worker*.



- Select the *Broker*, who manages the *Workers*, in the *WorkerPool*. We selected *MyBroker*.
- Start the simulation run and watch the *Worker* pick up the part at the first *Workplace*, walk on the *Footpath* to the target *Workplace*, put it down there, walk back to the first *Workplace*, pick up the next part, and so on.



Back to [Model a Worker Who Carries Parts Between Workplaces](#)

Model a Worker Who Carries Several Parts in 3D

Model a Worker Who Carries Several Parts

You can model a *Worker* who carries several parts between objects.

To model a *Worker*, who carries several parts, you can:

- Use the predefined [animation area](#).
- Use [animation paths](#) which you have to define.

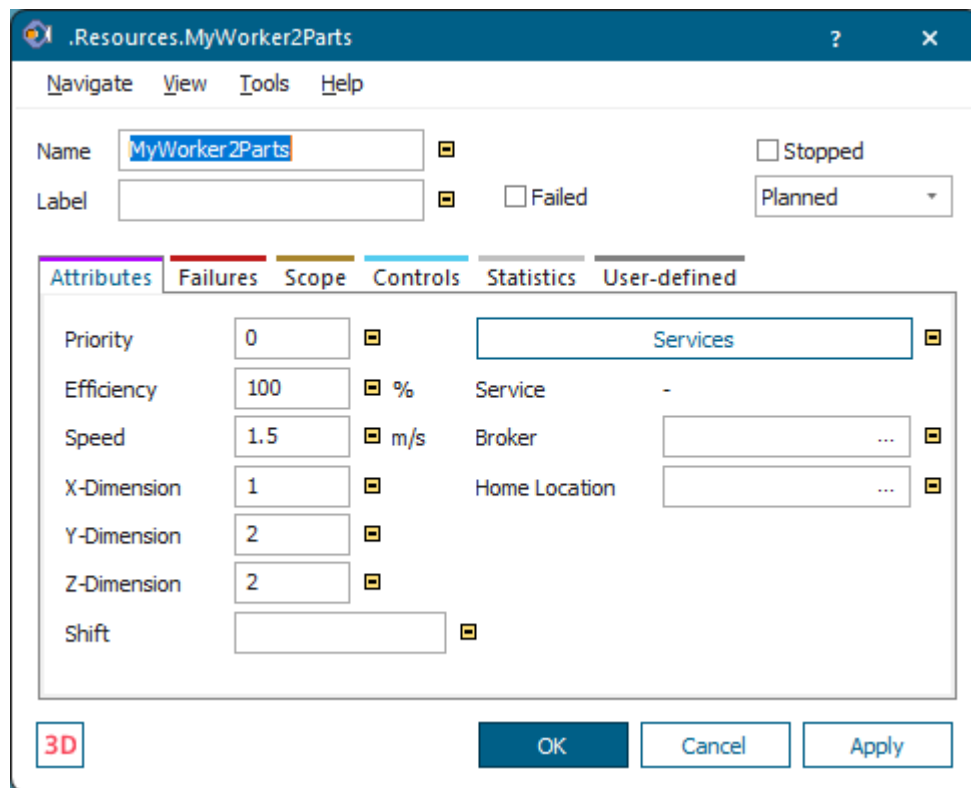
Back to [Model a Worker Who Carries Parts Between Workplaces](#)

Model a Worker Using the Animation Area

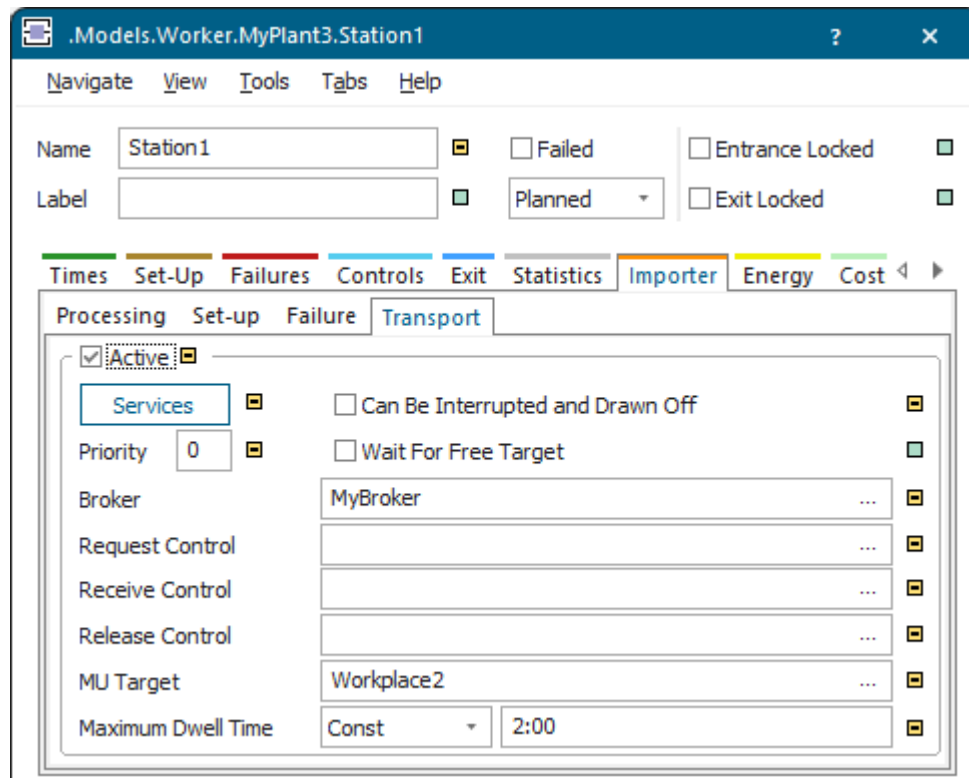
You can model a *Worker*, who carries several parts in 3D, who uses the predefined **animation area**.

Proceed as follows:

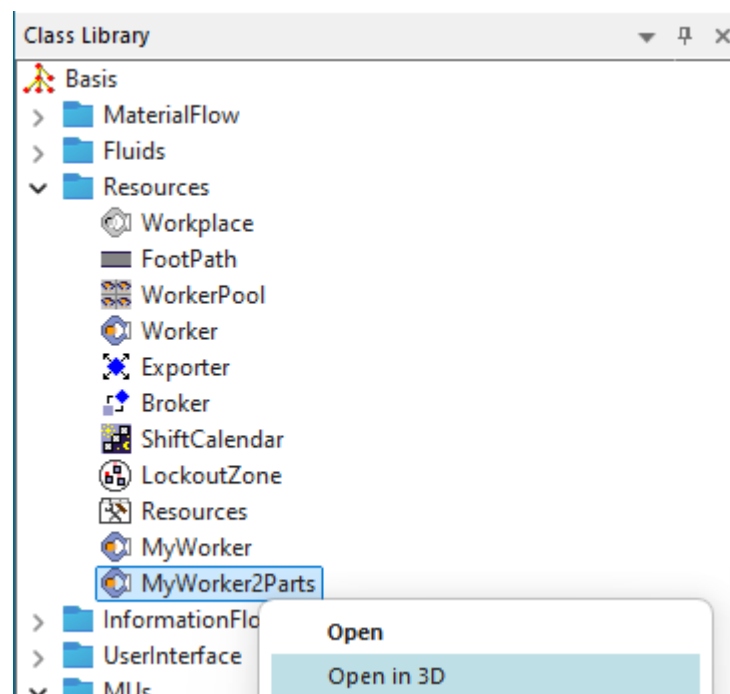
- Duplicate the *Worker* in the *Class Library* and rename it. We named our *Worker* *MyWorker2Parts*.
- Type the number of parts, which the *Worker*. We typed in 2 as the **Y-Dimension** and **Z-Dimension** as we want the *Worker* to carry two parts each in each hand, and leave the **X-Dimension** at 1 .



- To make the *Worker* wait for the second part, type the **Maximum Dwell Time** into the station, which wants to transfer the part. The **Dwell Time** has to be longer than the **Processing Time**, so we entered 2:00, i. e., two minutes.



- Click *MyWorker2Parts* on the tab **Class Library** with the right mouse button and select **Open in 3D**.

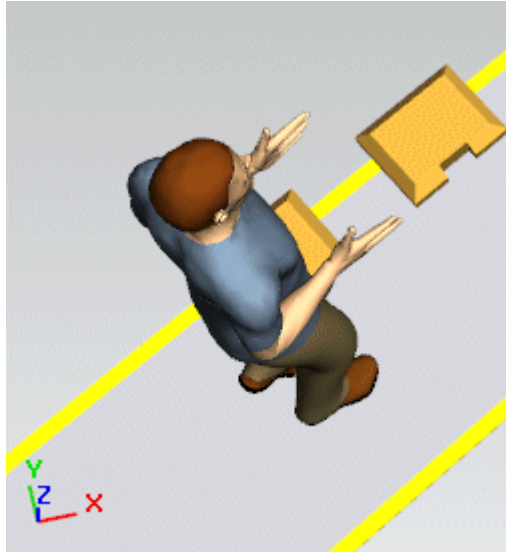


- Click into the background of the window with the right mouse button and select **Edit 3D Properties**. Click the **Tab MU Animation** and change to the **sub-Tab Animation Area**.

We want our *Worker* to carry several parts from station to station, depending on how many parts the delivering station has finished processing. To accomplish this we modified the model as follows:

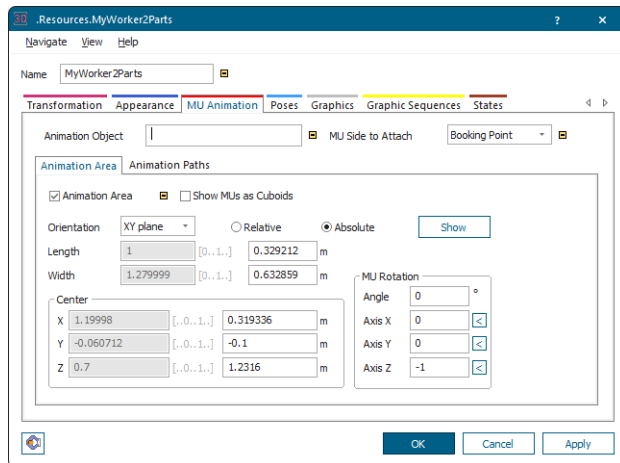
- We activated the **animation area** and set the settings for it on which *Plant Simulation* shows the parts.

The *Worker* looks like this in the model:

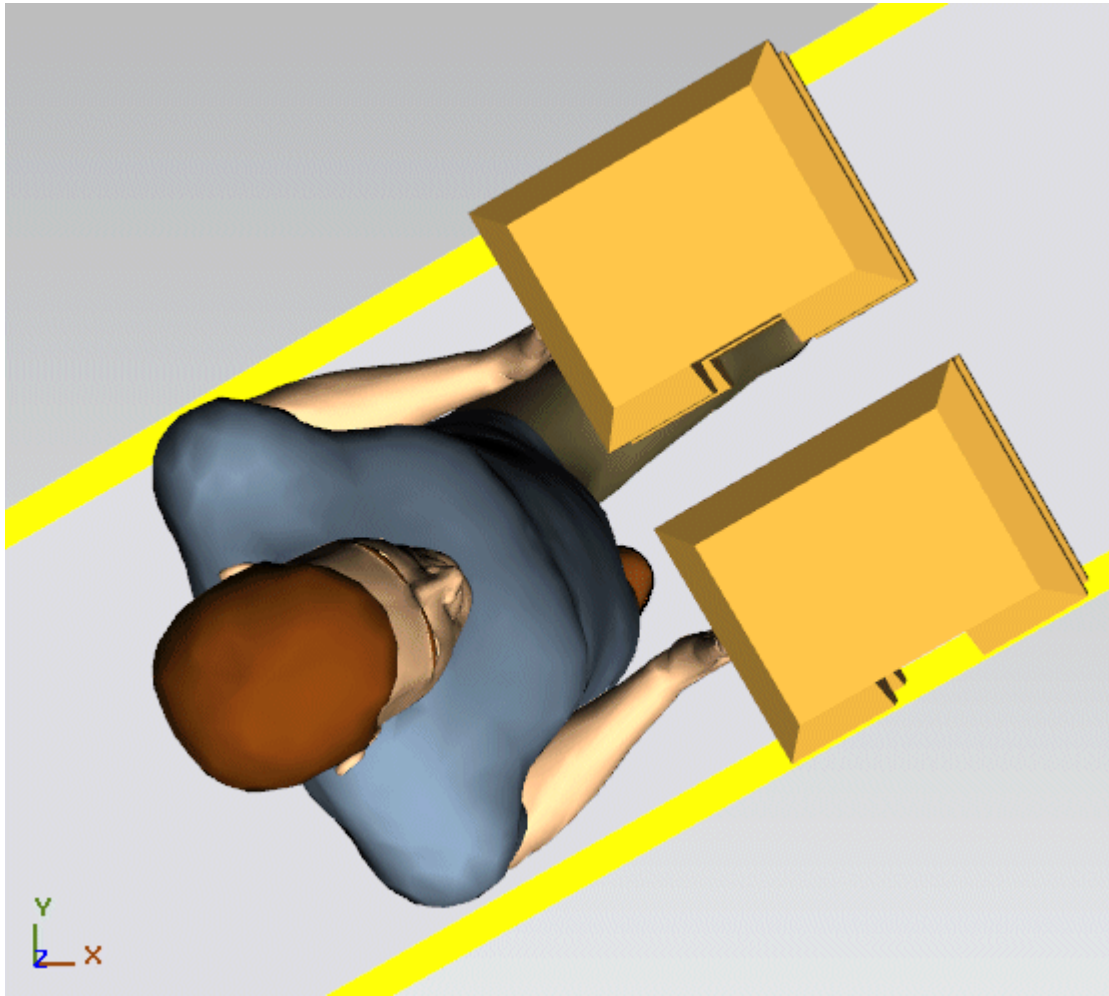


This is obviously not the way our *Worker* should look. For this reason we have to adjust the settings.

- ◊ First, we set the **X-position**, the **Y-position**, and the **Z-position** of the center of the animation area. After several attempts, we arrived at the settings below, which shows the parts at the correct positions on the hands of the *Worker*.

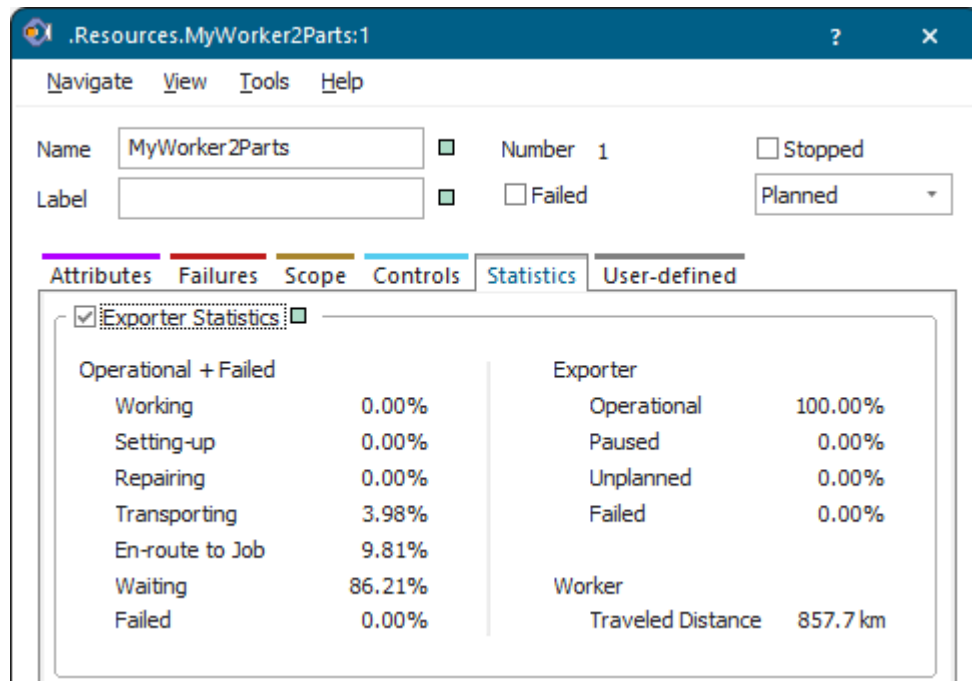


- ◊ As the parts are initially positioned too far on the outside of the hands of the *Worker*, we reduced the width of the animation area. This moves the parts closer to each other.
- Now everything looks like expected when we run the simulation.

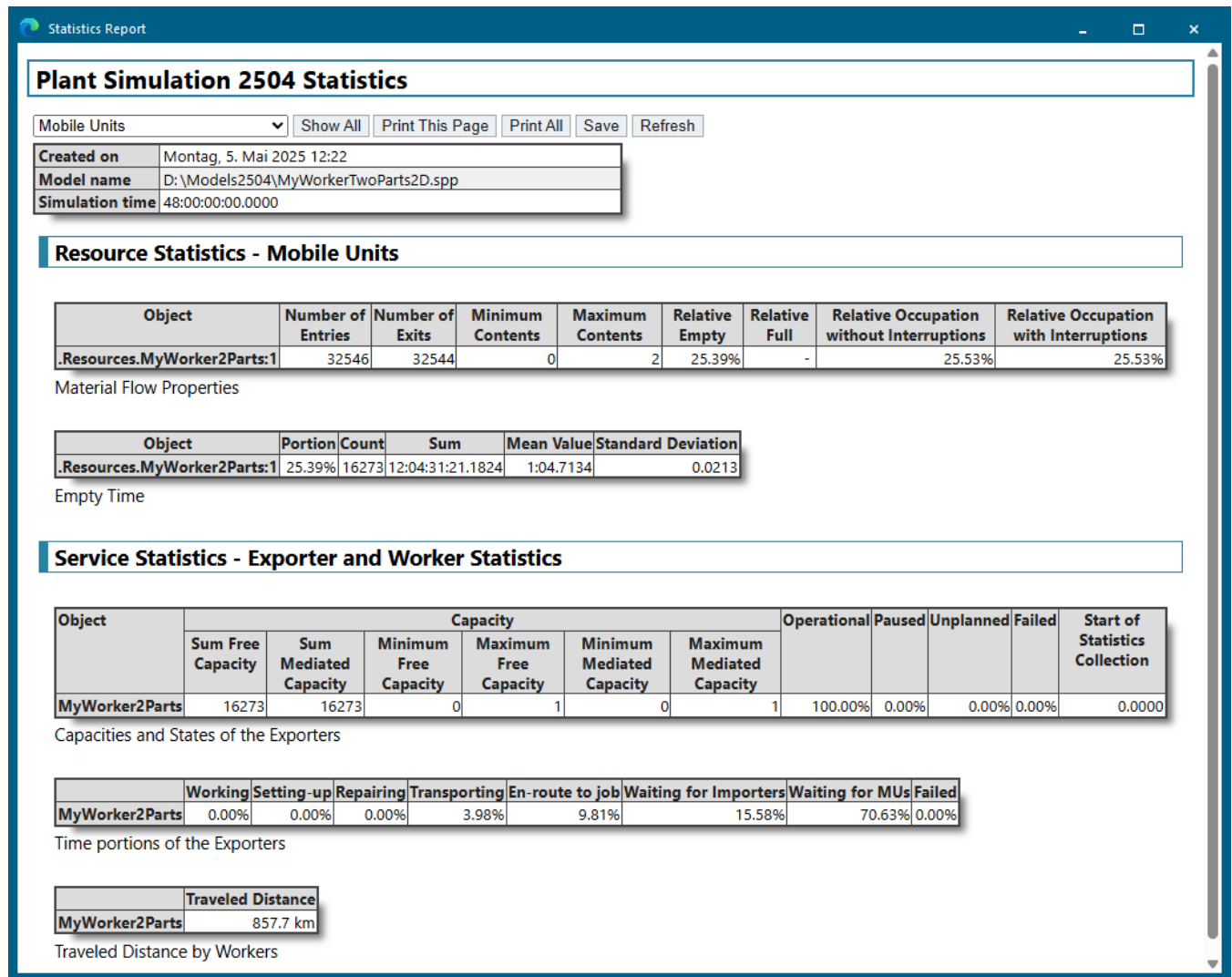


- Start the simulation and watch the *Worker* pick up the two parts at first *Workplace*, walk on the footpath to the target *Workplace*, put them down there, walk back to the first *Workplace*, pick up the next two parts, and so on.

The *Worker* shows the percentages of the carrying operations and of the en-route-times of the statistics collection time on the [Tab Statistics](#) below **Operational + failed** next to **Transporting** and **En-route to job**.



To show additional information in the *statistics report*, select the *Worker* and press **F6**.



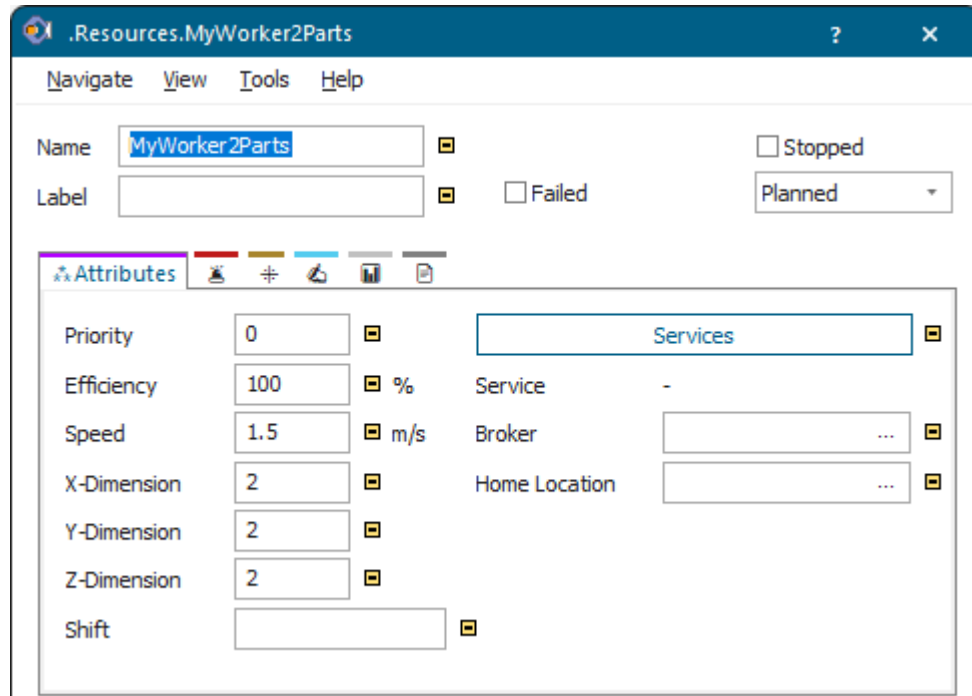
Go to [Model a Worker Using Animation Paths](#)

Back to [Model a Worker Who Carries Several Parts](#)

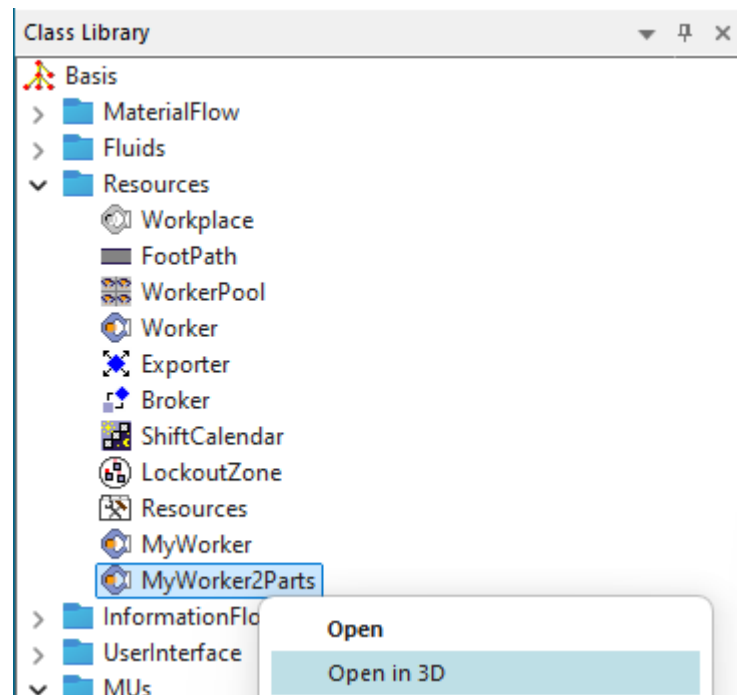
Model a Worker Using Animation Paths

You can model a *Worker*, who carries several parts in 3D, who uses an **animation path** each for his right hand and his left hand.

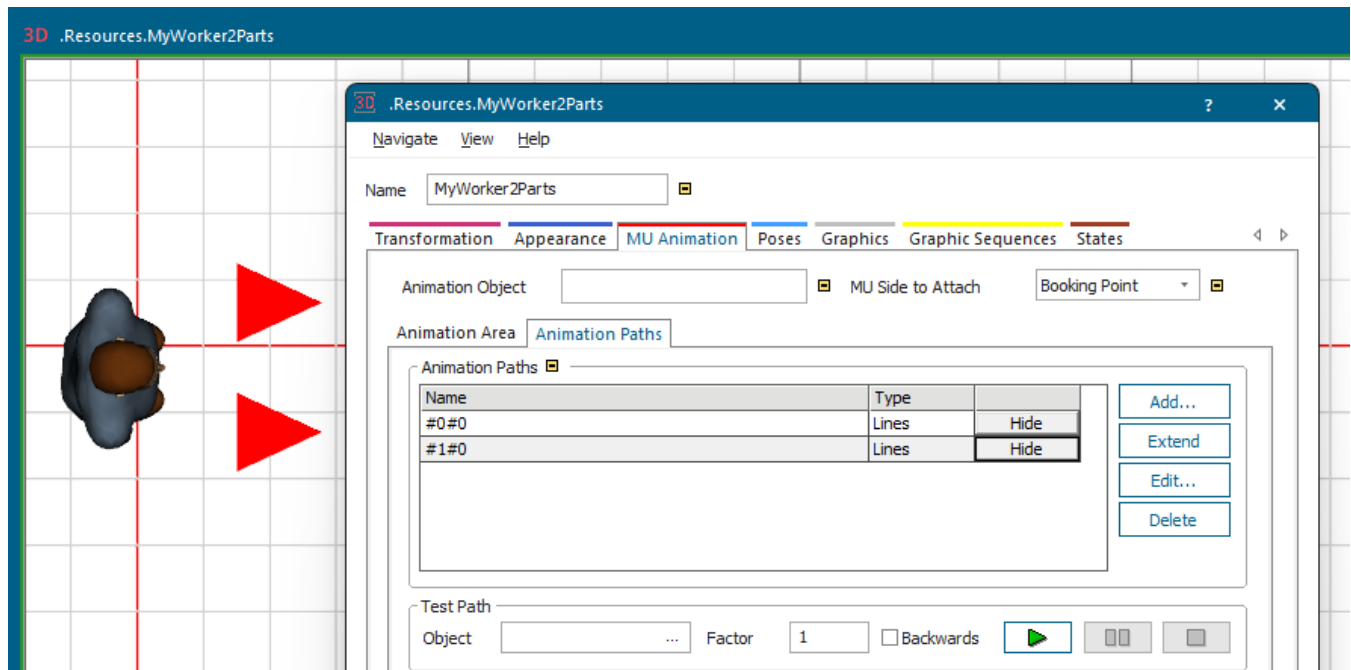
- Double-click *MyWorker2Parts* in the **Class Library**. Change the **X-Dimension**, the **Y-Dimension**, and the **Z-Dimension** to 2.



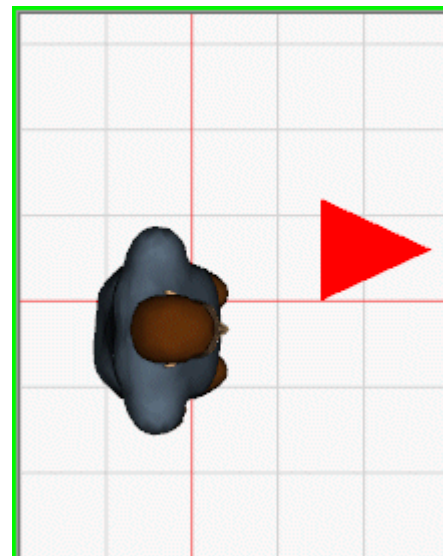
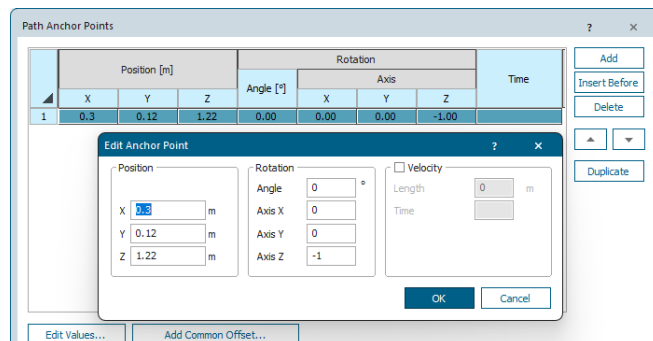
- Click *MyWorker2Parts* on the tab **Class Library** with the right mouse button and select **Open in 3D**.



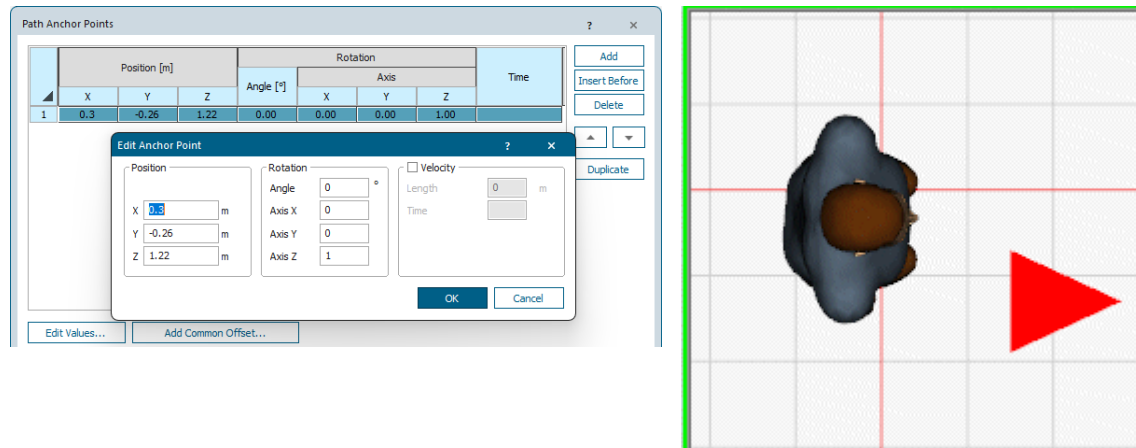
- Click into the background of the window with the right mouse button and select **Edit 3D Properties**. Click the **Tab MU Animation** and change to the **sub-Tab Animation Paths**.
- Add two animation paths of type **Lines** and name them #0#0 and #1#0.



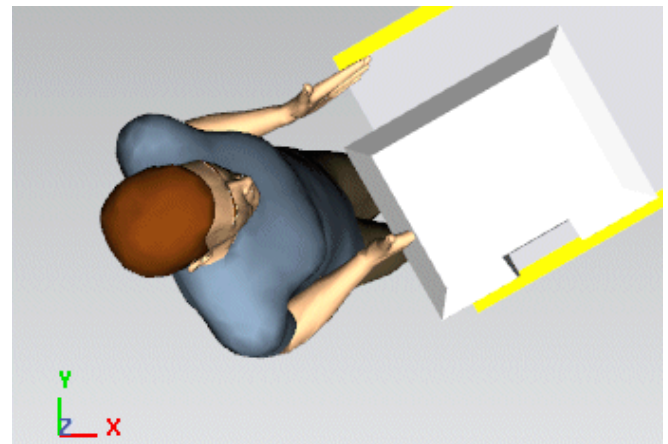
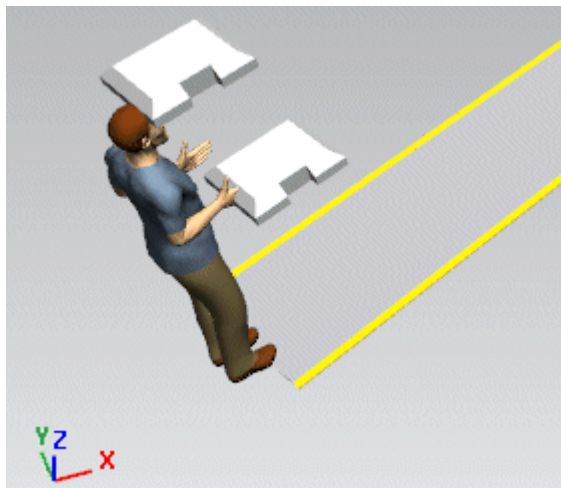
- **Show** the animation path named **#1#0** (right hand) and move it to the right by clicking **Edit** and by changing the **Position**. Click **OK** in the dialog **Edit Anchor Values**. Click **Apply** in the dialog **Path Anchor Points** to show the position of the path in the window of the *Worker*.



Show the animation path named **#0#0** (left hand) and move it to the left by clicking **Edit** and by changing the **Position**. Click **OK** in the dialog **Edit Anchor Values**. Click **Apply** in the dialog **Path Anchor Points** to show the position of the path in the window of the *Worker*.

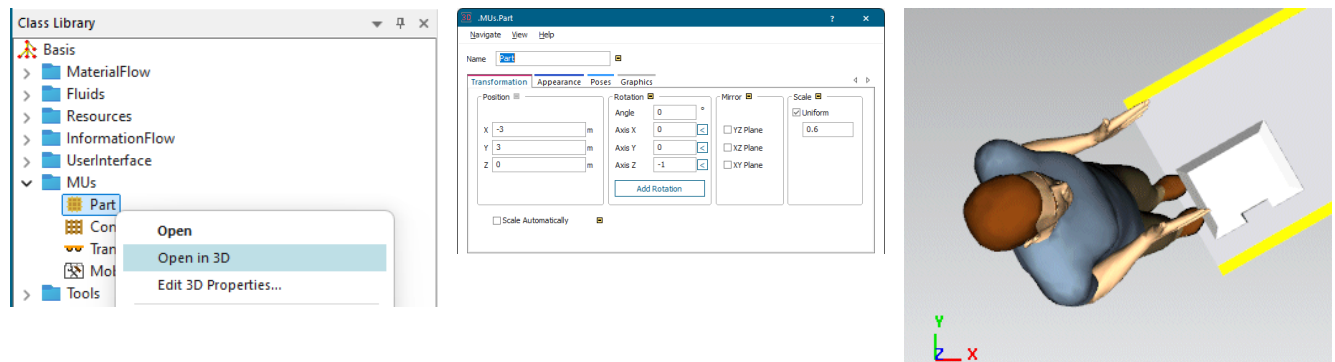


- Run the simulation to check if the settings meet our needs.



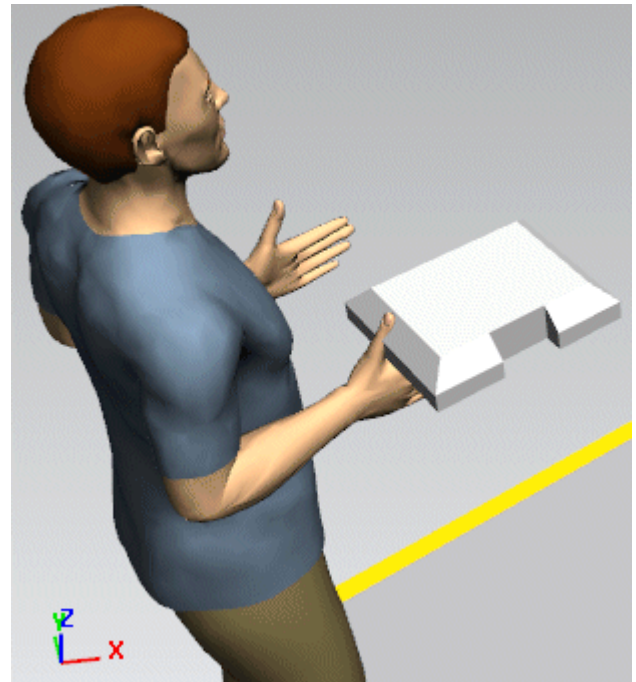
- The part on the right hand side is located at the correct height, but it is too big and too far to the left. To fix this, we decrease the size of the part by selecting a **uniform scaling factor** of 0.6 or select the check box **Scale automatically**.

Click the *Part* in the *Class Library* with the right mouse button and select **Open in 3D**.



Now the size and height of the part are correct, but it is located too far forward. To fix this, we have to move the **X-Position** toward the body of the *Worker*. We used the settings below.

Position		
X	0.3	m
Y	-0.26	m
Z	1.22	m



- Repeat this for the animation path named #0#0 (left hand). We selected the settings below:

Position		
X	0.3	m
Y	0.12	m
Z	1.22	m



- Now everything looks like expected when we run the simulation.
Our *Worker* carrying four parts in 3D looks like this:



As defining animation paths for the *Worker* who carries parts is quite involved, we recommend to use the pre-defined animation area.

Back to [Model a Worker Using the Animation Area](#)

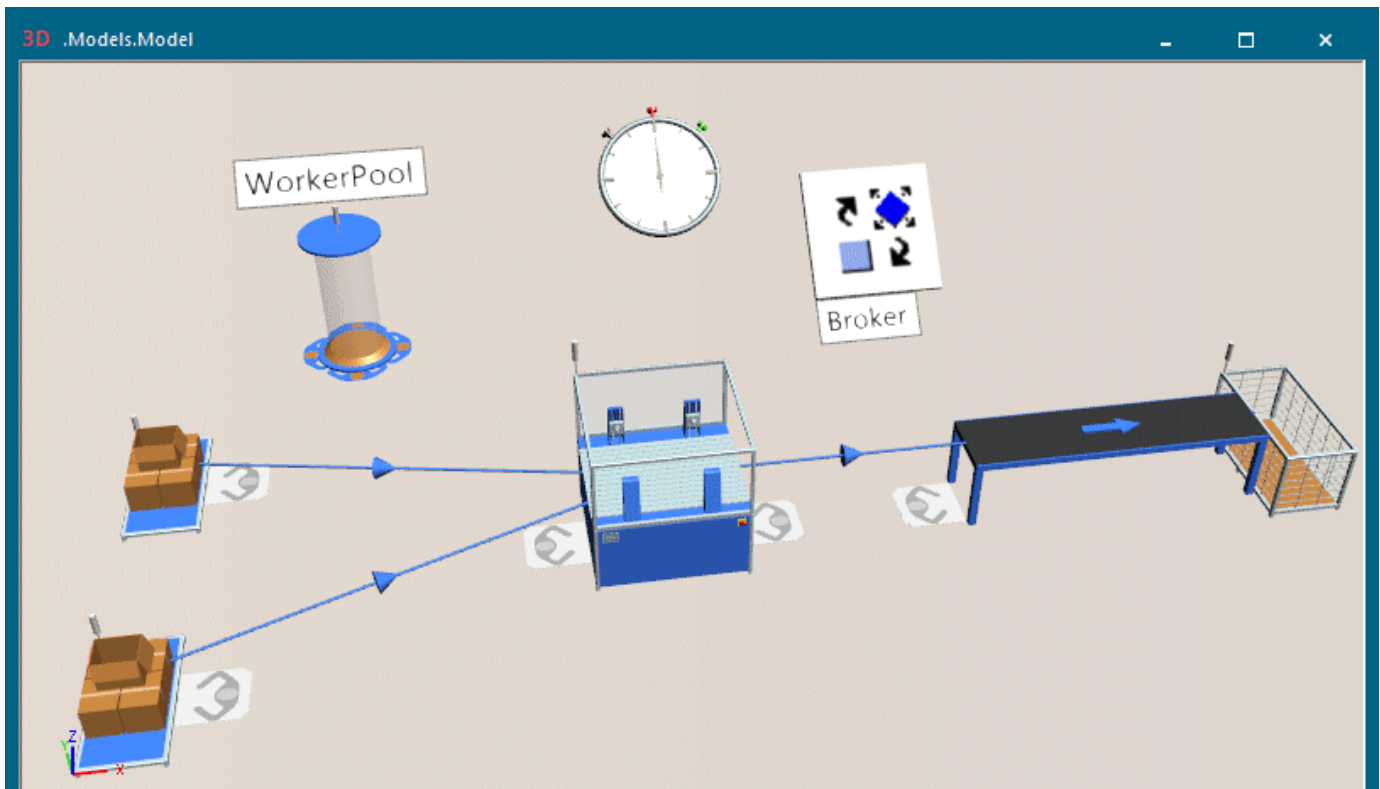
Back to [Model a Worker Who Carries Several Parts](#)

Model a Worker Who Waits for a Free Target

You can model two *Workers* who pick parts up from a *Workplace* attached to a *Source*.

The **Workers**  then carry the parts to a *Workplace* attached to the **entrance** of the *ParallelStation*, and place them onto one of the four processing places. Once all processing places are stocked with parts, the *Workers* walk around the *ParallelStation* to the *Workplace* attached to its **exit**. They finally pick a processed part up and place it onto the *Workplace* attached to the *Conveyor*.

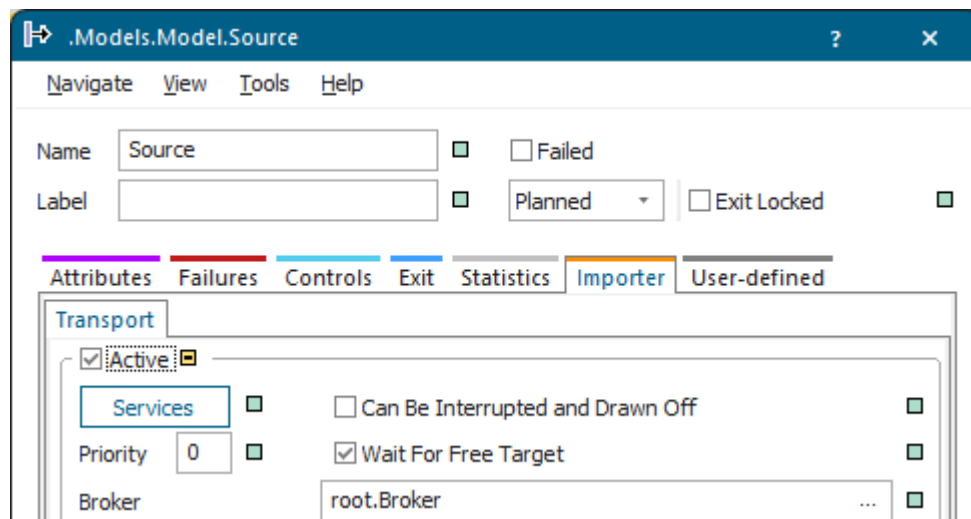
The finished simulation model looks like this:



Proceed as follows to create the simulation model:

- Insert the two *Sources*. We used them with their default settings.

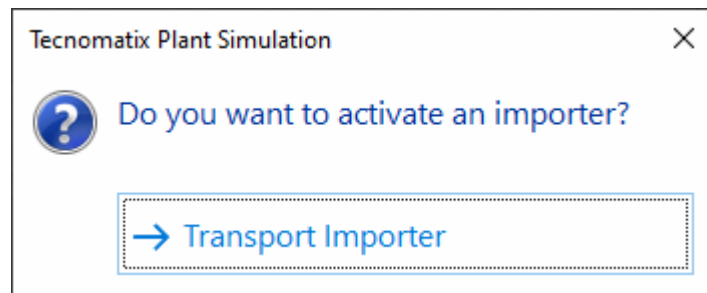
Make sure that **Wait for Free Target** on the **tab Importer > sub-tab Transport** is turned on. This prevents that a *Worker* is requested if the target is not free. A *Worker* only walks to the station on which the part is located once the target is free. He then picks the part up and carries it to the target.



- Insert a **Workplace** next to each of the *Sources* where the *Workers* picks up the parts. Drag the *Workplace* close to one of the sides of *Source*, which you want to use as the **Station**.

Plant Simulation then

- Opens a window asking you if you want to activate a required *importer*, a **Transport Importer** in our case. Click the button to activate it.



- Enters the *Source* as the **Station** into the *Workplace*.
- Selects the check boxes **At Entrance** and **At Exit** of the *Workplace*.

The *Workers* carry the produced parts to the *Workplace* attached to the **entrance** of the *ParallelStation* and place them there.

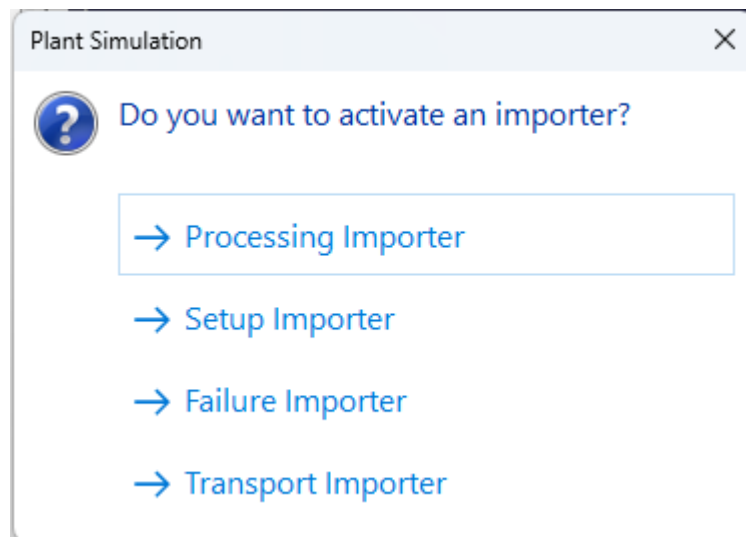
They pick the processed parts up at the *Workplace* attached to the **exit** of the *ParallelStation*.

- Insert the *ParallelStation* that processes a part on each of its four processing places.

Insert a *Workplace* close to the left side of the *ParallelStation* where the *Workers* place the part. Drag the *Workplace* to the *ParallelStation*.

Plant Simulation then

- Opens a window asking you if you want to activate a required *importer*, a **Transport Importer** in our case. Click **Transport Importer** to activate it.



- Enters the *ParallelStation* as the **Station** into the *Workplace*.
- Selects the check boxes **At Entrance** and **At Exit** of the *Workplace*.

As we want this *Workplace* to be at the entrance of the *ParallelStation*, we clear **At Exit**.

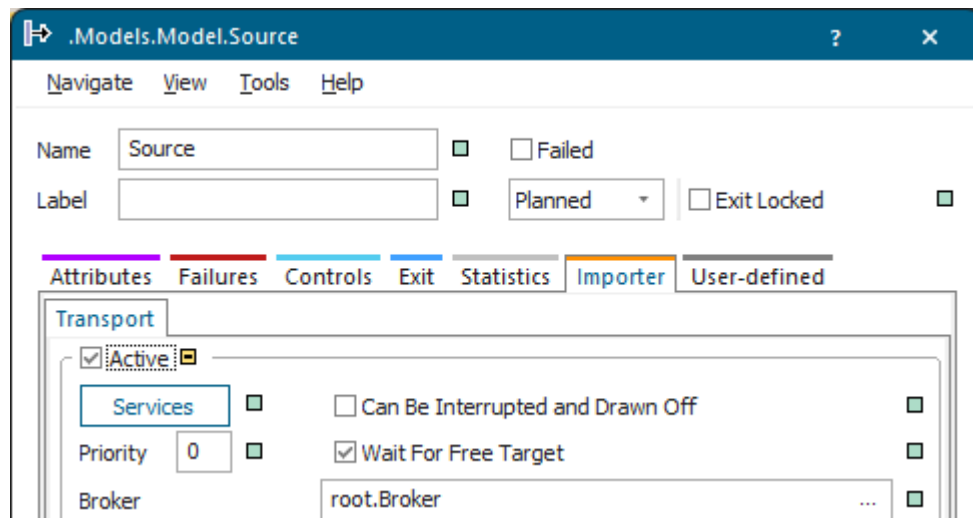
The *Workers* carry the produced parts to the *Workplace* attached to the **entrance** of the *ParallelStation*.

- Insert a *Workplace* close to the right side of the *ParallelStation* where the *Workers* pick up the parts to carry them to the *Conveyor*. This enters the *ParallelStation* as the **Station** into the *Workplace* and selects the check boxes **At Entrance** and **At Exit**.

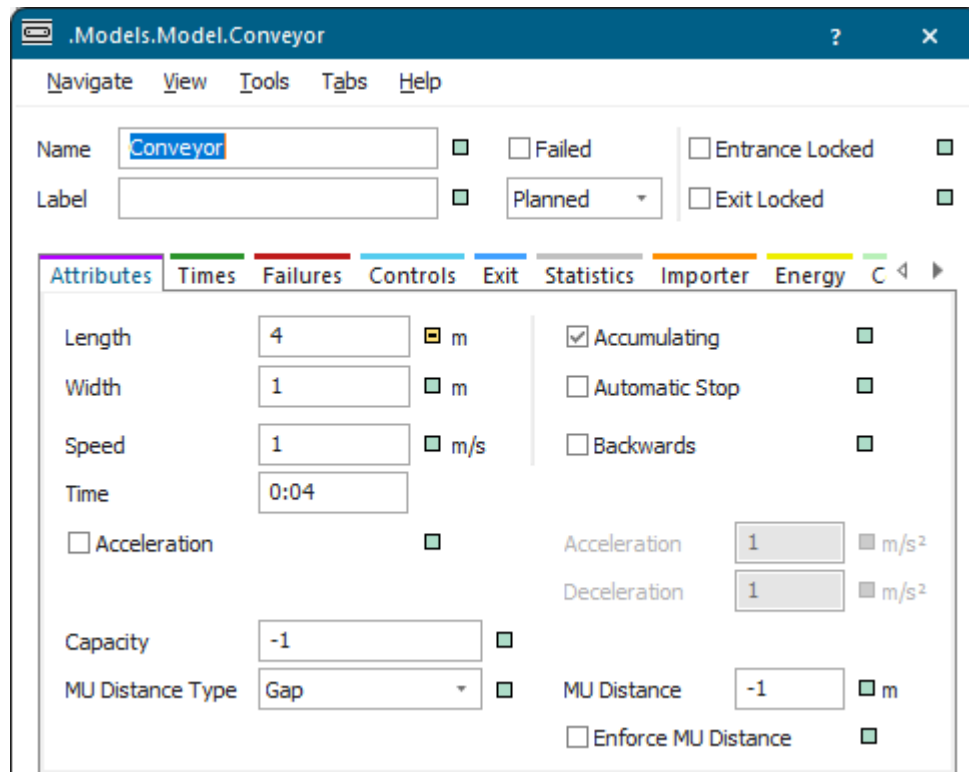
As we want this *Workplace* to be at the exit of the *ParallelStation*, we clear **At Entrance**.

The *Workers* pick the processed parts up at the *Workplace* attached to the **exit** of the *ParallelStation*.

- Configure the *ParallelStation*.
 - We entered a **Processing Time** of 5 seconds
 - Make sure that **Wait for Free Target** on the **tab Importer > sub-tab Transport** is turned on. This prevents that a *Worker* is requested if the target is not free. A *Worker* only walks to the station on which the part is located once the target is free. He then picks the part up and carries it to the target.



- Insert a *Conveyor*. Our *Conveyor* has these settings.



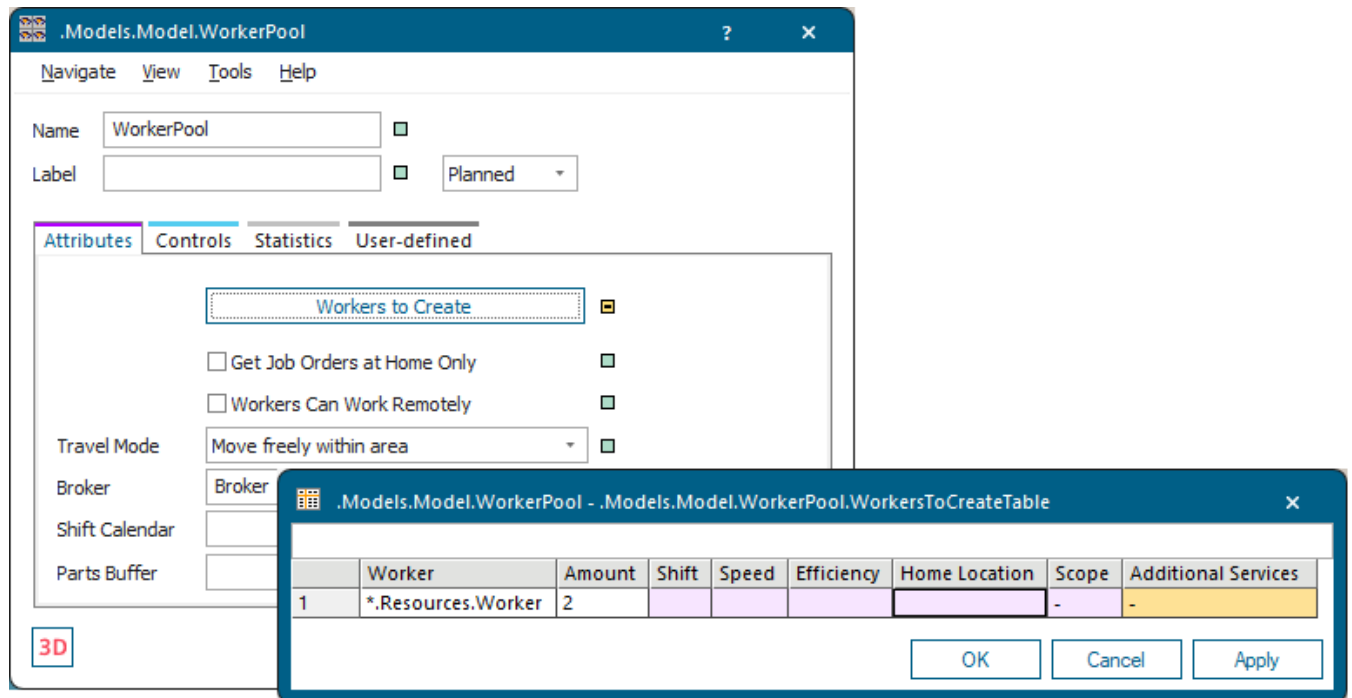
Insert a *Workplace* next to the left side of the *Conveyor*. Drag the *Workplace* to the *Conveyor*.

Plant Simulation then

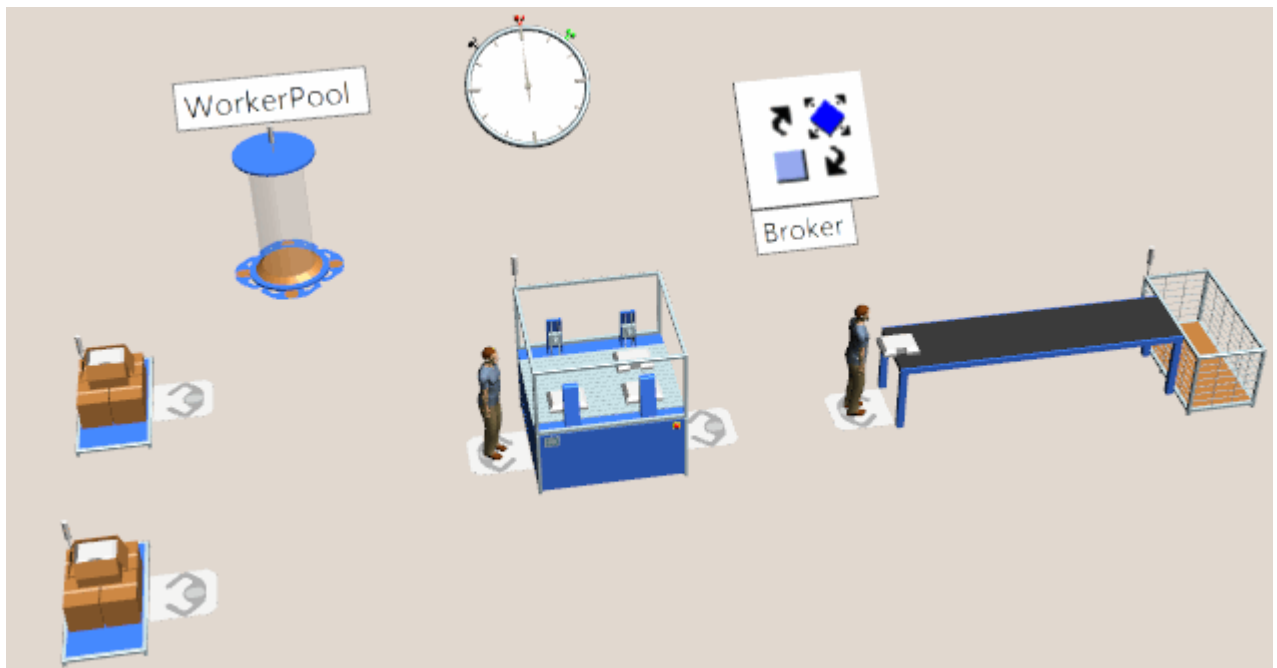
- Enters the *Conveyor* as the **Station** into the *Workplace*
- Selects the check box **At Entrance** of the *Workplace*.

The *Workers* carry the processed parts from the *Workplace* attached to the **exit** of the *ParallelStation* to the *Workplace* attached to the **entrance** of the *Conveyor*.

- Insert a **WorkerPool**  and configure it. We selected these settings.



- Select the *Broker*, who manages the *Workers*, in the *WorkerPool*. We selected *Broker* and used it with the default settings.
- Connect the material flow objects with *Connectors*. The *Workers* walk along the *Connectors* to the *ParallelStation* and to the *Conveyor*.
- Start the simulation run and watch the two *Workers* pick up a part at one of the *Sources* and carry it to the respective *Workplace*. Notice how they **wait until a processing place becomes available** before starting to carry a part to it.



You can check the following on the **View Menu** of the respective objects:

- The name of the **Exporters** exporting services.

	object 1	object 2	object 3	object 4
string	Failure Exporter	Set-Up Exporter	Processing Exporter	Transport Exporter
1				*.Models.worker.SpecialObjects.WorkerCap3:1
2				

- The name of the **Imported Services**.

	table 1	table 2	table 3	table 4
string	Failure Service	Set-Up Service	Processing Service	Transport Service
1				StandardService
2				

- The name of the **Associated Workplaces** of the station.

	object 1	string Workplace
1		*.Models.Transportimporter.WaitForFreeTarget.Workplace2
2		*.Models.Transportimporter.WaitForFreeTarget.Workplace3
3		

- The **Contents and Reserved Places** list. It shows the names of the parts on the object for which a place is reserved.

	Object	From	To
1	*.MUs.Part:2	0	0
2	*.MUs.Part:1	1.06767	1.56767

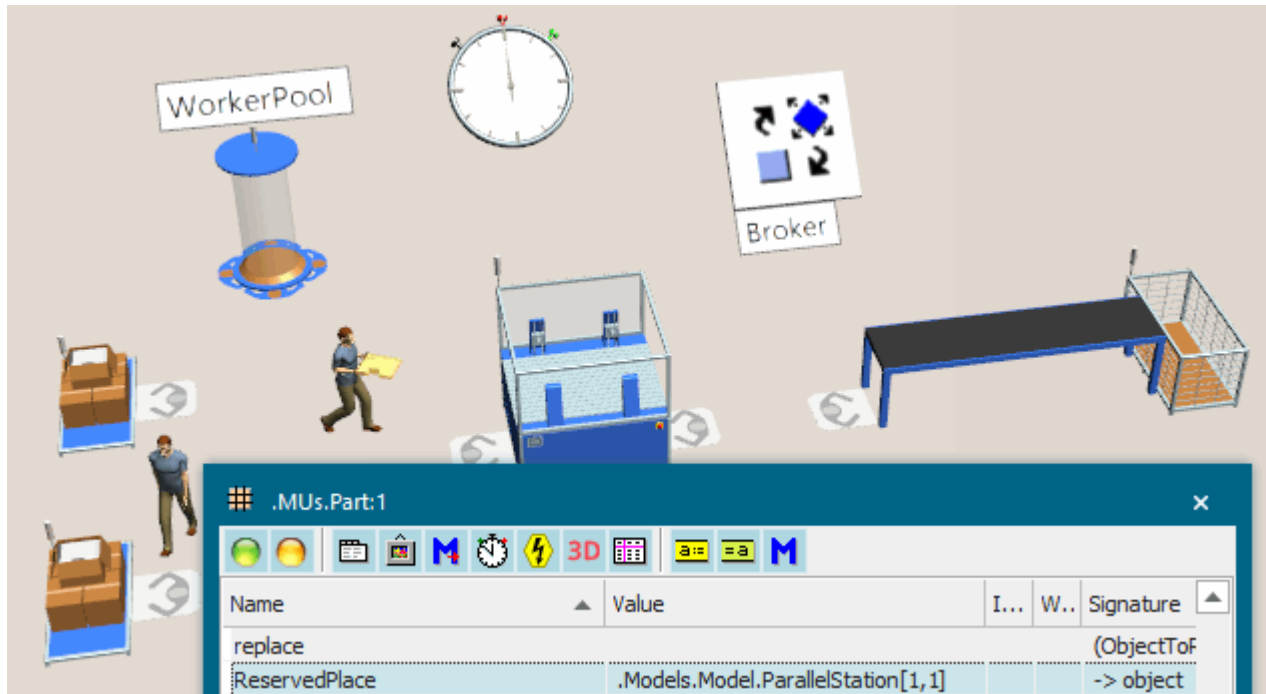
	1	2	3
1	*.MUs.Part:1	*.MUs.Part:2	
2	*.MUs.Part:3		

You can also check the following information:

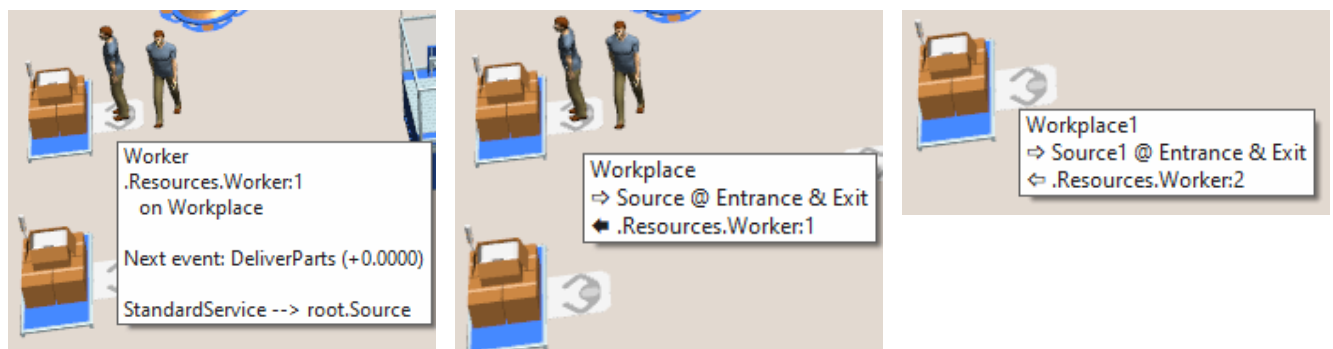
- The read-only attribute **ReservedFor** of the *Conveyor* shows the part for which a place is reserved.

.Models.Model.Conveyor				
Name	Value	I...	W..	Signature
ResCurrentState	"Working"		*	-> string
ReservedFor	["*.MUs.Part:2"]			any

- The read-only attribute **ReservedPlace** of the *Part* shows the place which is reserved for it on a material flow object.



- The *tooltips* of the *Worker* and the *Workplace* show additional information.



Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Walks Freely

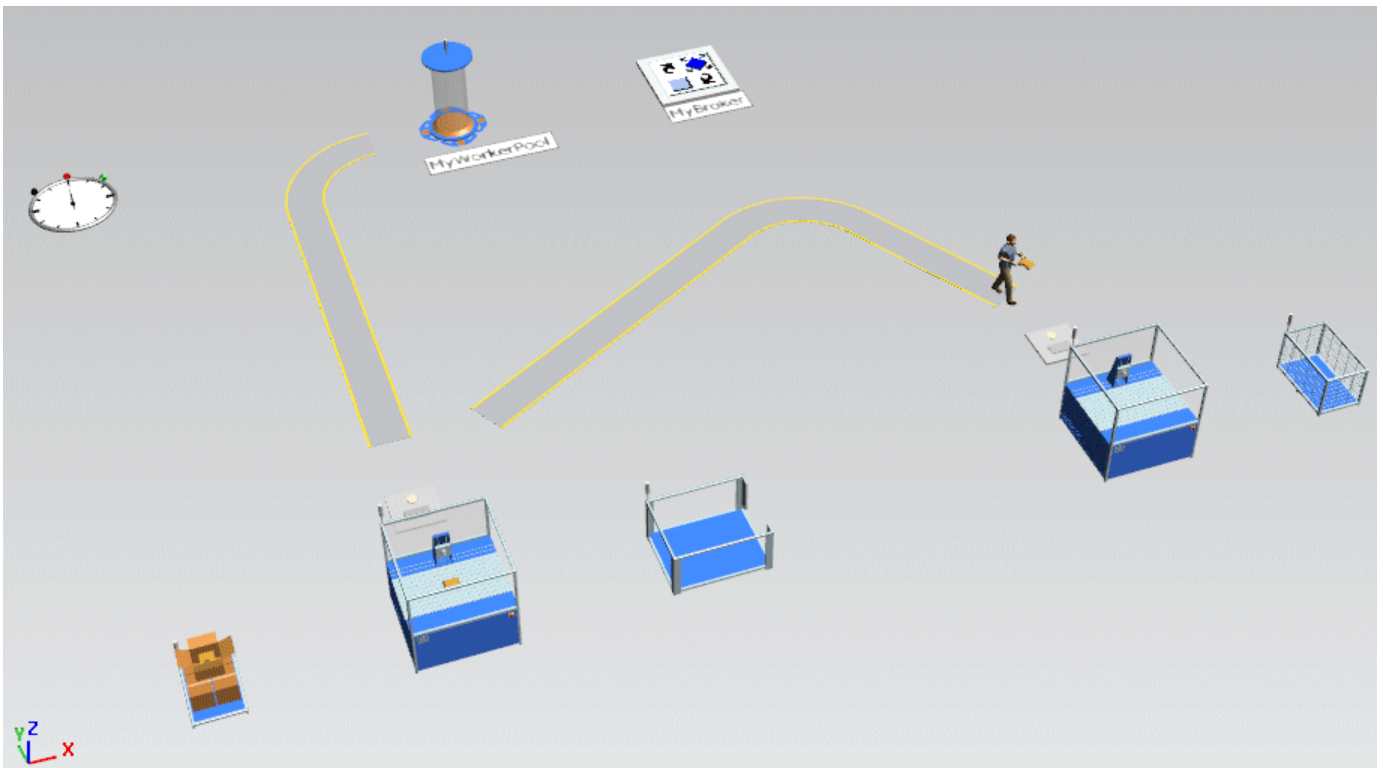
Model a Worker Who Walks Freely

You can model a *Worker* who **walks freely within the plant**.

This way the **Worker**  is not forced to walk on the *FootPaths*, which you inserted between the *WorkerPool* and the *Workplaces* attached to the work station. For this reason he has to walk around obstacles, which he cannot enter or cross. The *obstacles* can be machines, safety fences around robots, paths of automated guided vehicles, or structural factors in the plant, such as columns, stairways, etc.

As you can only set some of the functions of the *Worker*, who **walks freely within the plant**, in 3D, select **3D** when you create a new model.

To show the difference between a *Worker walking on FootPaths* and a **Worker, who walks freely within the plant**, we use an existing model with *FootPaths*. In the screenshot below the *Worker* walks from the *WorkerPool* via the inserted *FootPaths* to the *Workplaces*, which are assigned to the stations.



We modify this model by inserting a **barred area** and demonstrate how to:

- [Model a Worker Who Walks Freely Within the Model](#)
- [Model a Worker Who Walks One-Way](#)
- [Model a Worker Who Walks Through Doors](#)

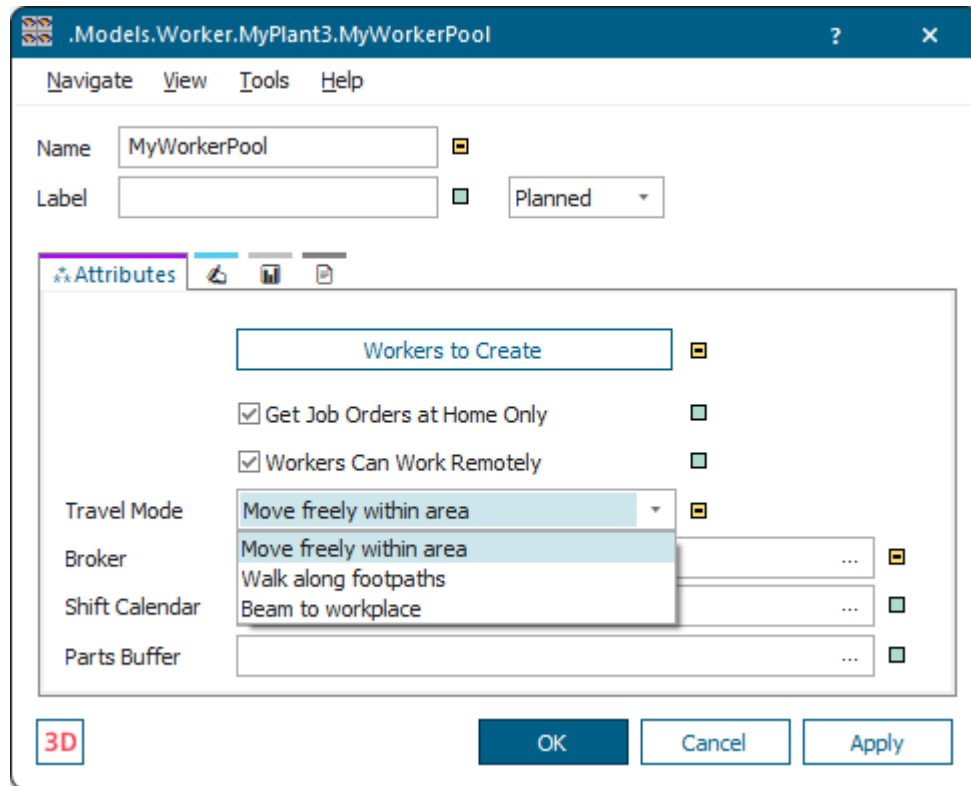
Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Walks Freely Within the Model

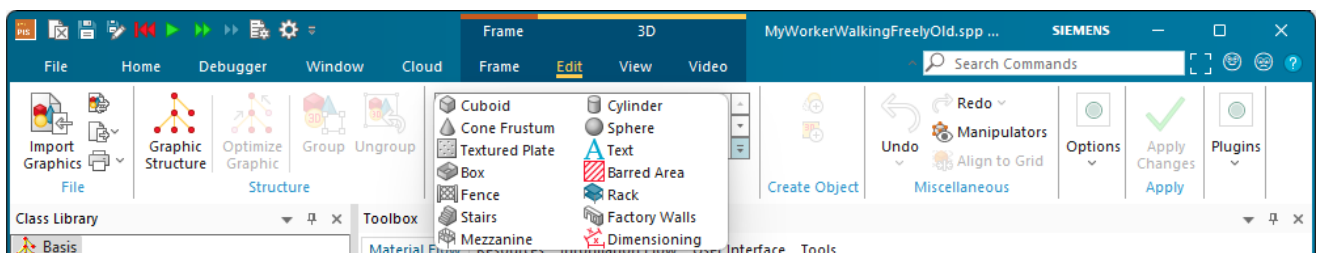
You can model a *Worker* who walks freely within the model.

Proceed as follows:

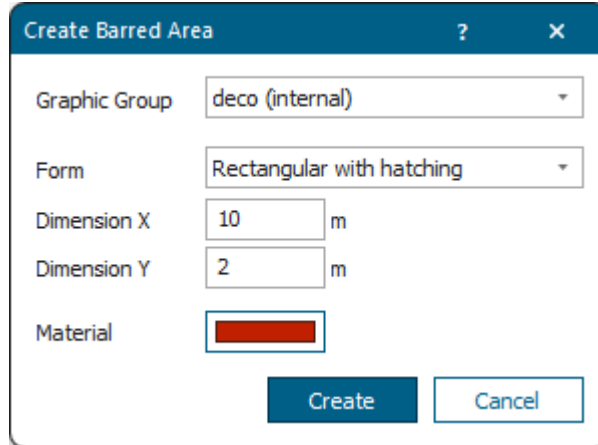
- Double-click the [WorkerPool](#) in the 3D model and select **Travel Mode > Move freely within the area** in the drop-down list.



- Insert an **obstacle** around which the *Worker* has to walk when he walks to the *Workplaces* at which he performs his job:
 - Change to the ribbon tab **Edit**.
 - Click **Barred Area**.



- Select the **graphic group**, the **Form**, the **Dimensions** and the **Material** of the **barred area**. We selected the following settings:



Create Barred Area

Graphic Group: deco (internal)

Form: Rectangular with hatching

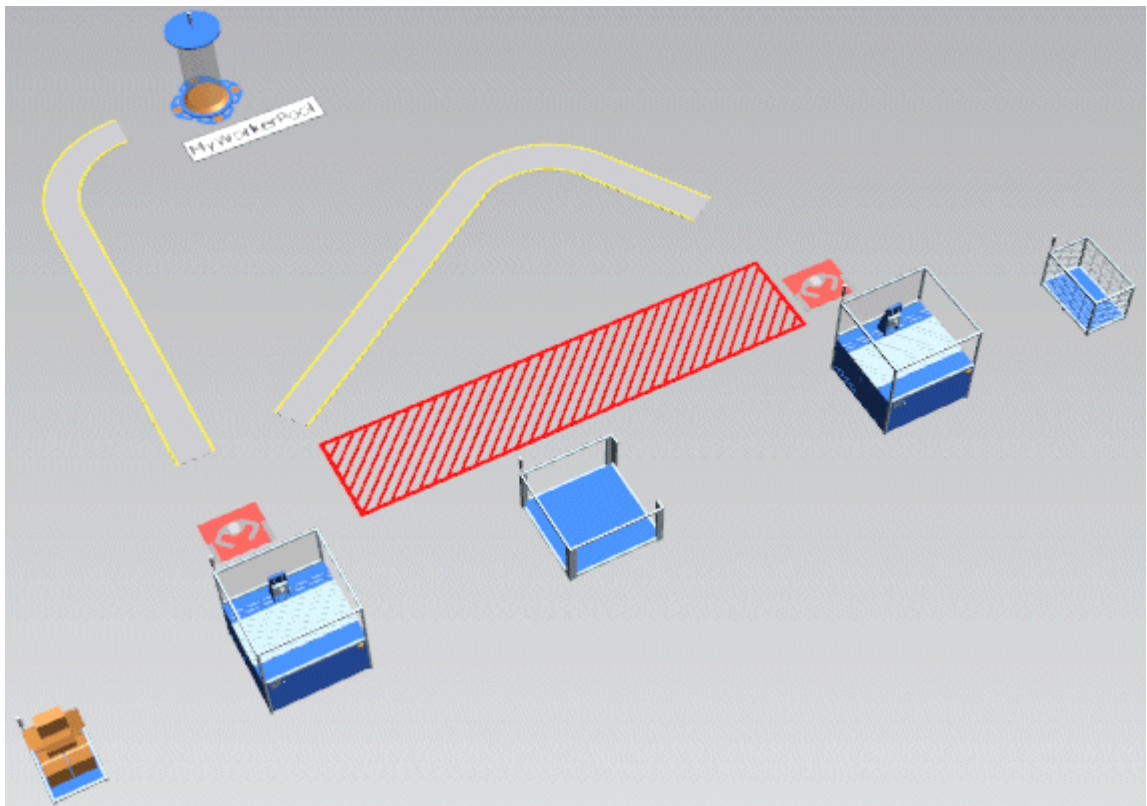
Dimension X: 10 m

Dimension Y: 2 m

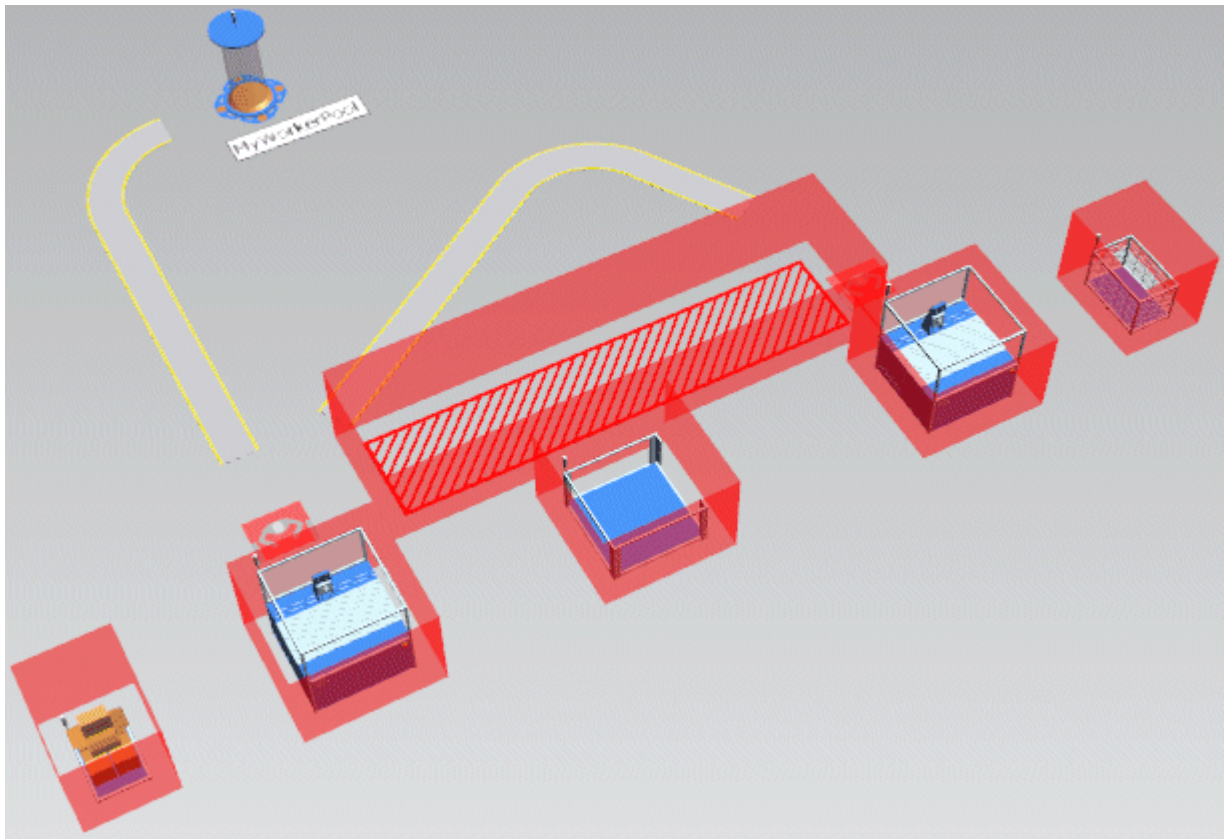
Material: [Red hatched box]

Create Cancel

- The inserted **barred area**, our **obstacle for the worker**, looks like this:

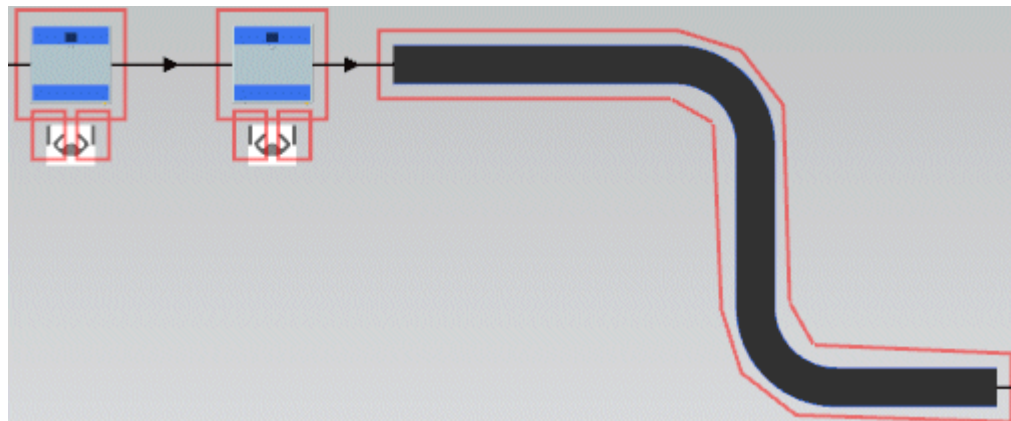


By default the **barred area** is an **obstacle for the worker**. Press the **o** key on the keyboard to show **all obstacles for the worker** in the model. The *Worker* has to walk around these obstacles, he cannot cross them.

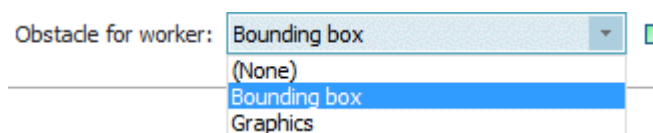


For the *Workplace Plant Simulation* shows two bounding boxes with a gap in between. This ensures that the *Worker* only stays on the middle of the *Workplace*. The *Worker* first has to leave the *Workplace* before he can change his walking direction toward the next target object.

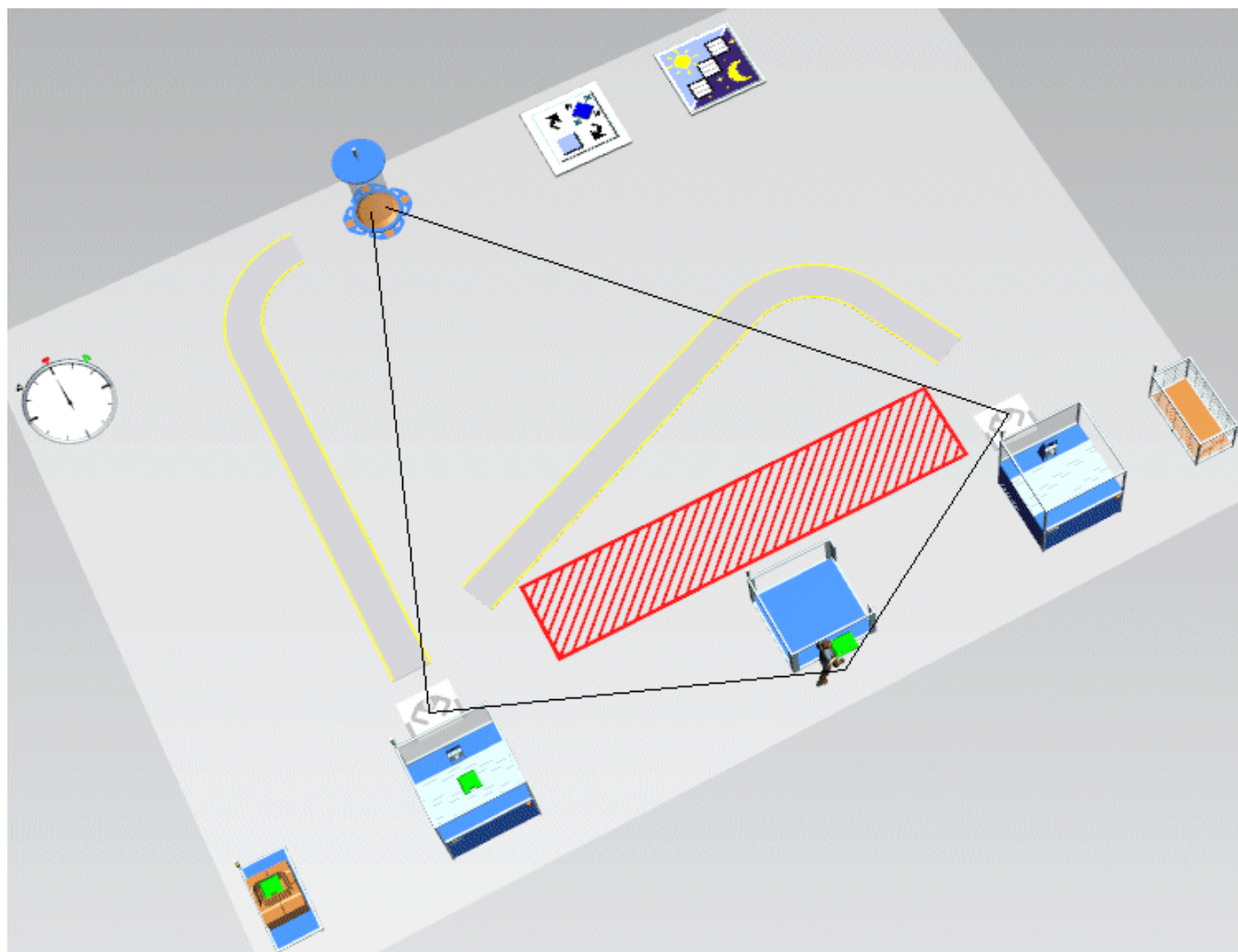
For the length-oriented objects *Plant Simulation* approximates the display of the bounding box only roughly to the curve, i.e., it does not show the curve of the conveyor as a smooth curved line.



For most of the 3D objects you can select if they are an **Obstacle for the Worker** on the **Tab Graphics** or on the **Tab Graphic Settings** in the dialog **Edit 3D Settings**. For the built-in material flow objects the **bounding box** of the graphic is an **obstacle for the worker**.



- Change to the **Home** ribbon tab, run the simulation and watch how the *Worker* walks from the *WorkerPool* to the *Workplaces*, how he ignores the *FootPaths* and walks around our **barred area**. In the screenshot below we marked his route in black for clarification purposes.

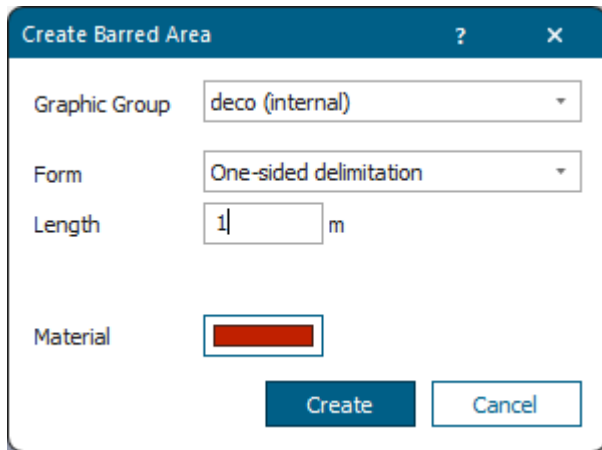


Back to [Model a Worker Who Walks Freely](#)

Model a Worker Who Walks One-Way

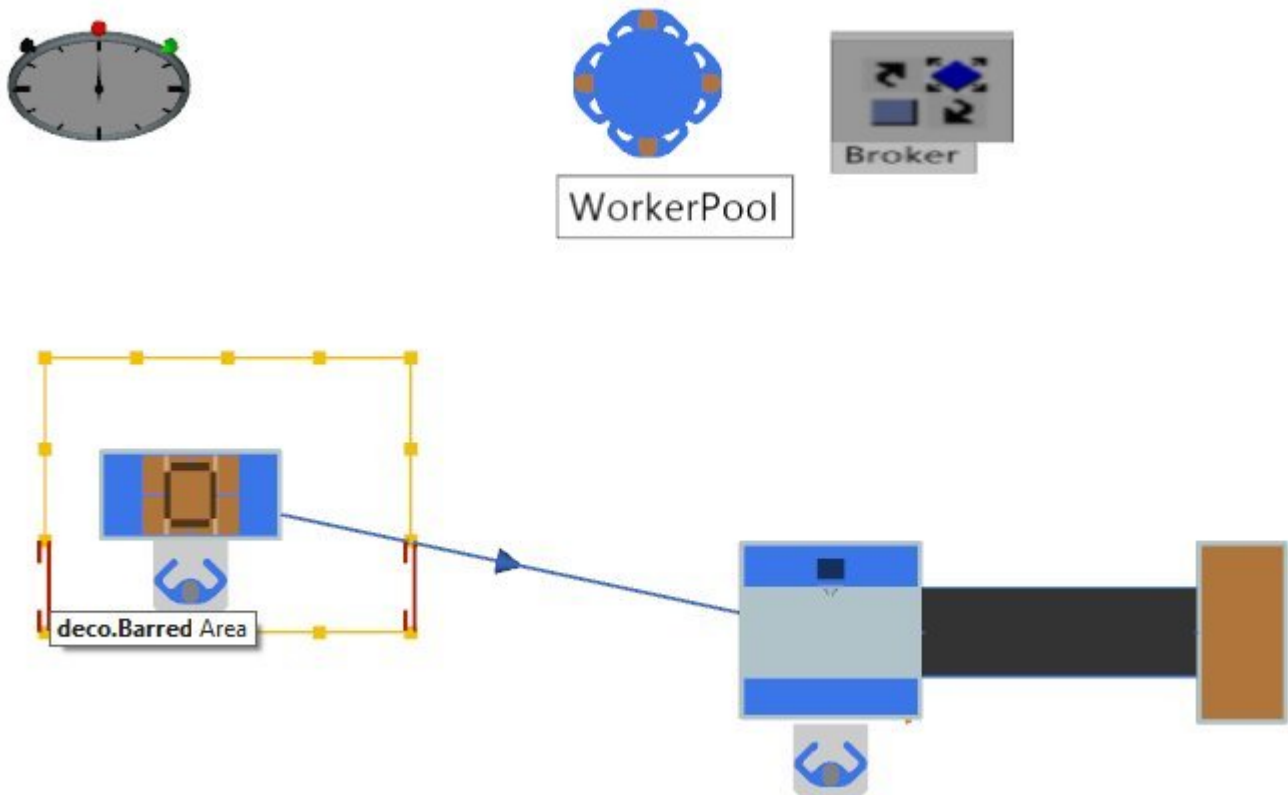
You can model a *Worker*, who can only enter the *Workplace* from one direction only.

To do so, we insert a **Barred Area** with the **Form > One-sided delimitation** into our model.



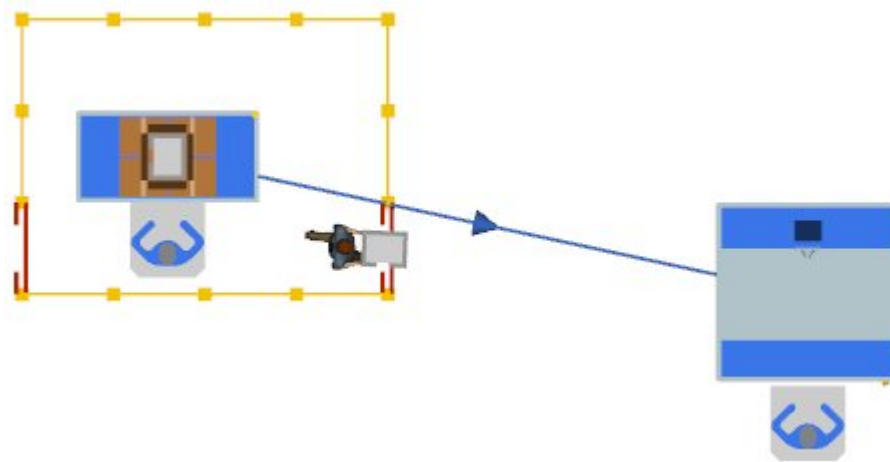
The *Worker* then can cross the line on the side with the dashed line but not on the side with the solid line. Depending on the insertion direction of the barred area, he can walk from left to right, but not from right to left.

Our finished model looks like this. We used all objects with their default settings.

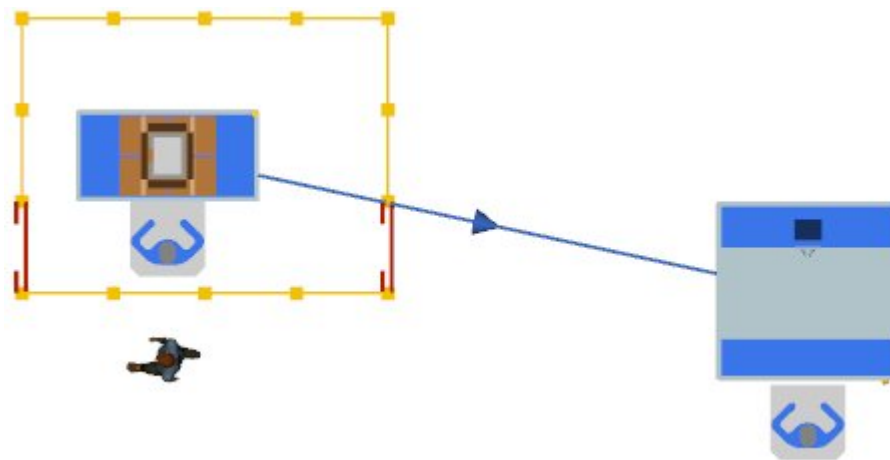


We inserted a *WorkerPool*, a *Broker*, and a *Source* with an attached *Workplace* surrounded by a safety fence. The *Worker* walks toward the *Source*, enters the **one-way barred area**, steps onto the *Workplace*,

picks the produced part up, crosses the second **one-way barred area**, carries the part to the *Workplace* attached to the *Station*, and places it there.



Then the *Worker* walks back to the *Source* to pick up the next part. The **barred area** on the right is **one-way** as well, meaning he cannot walk through it from right to left, which would be the shortest way. Instead, he has to walk around the safety fence and walk through the **one-way barred area** on the left to continue working.



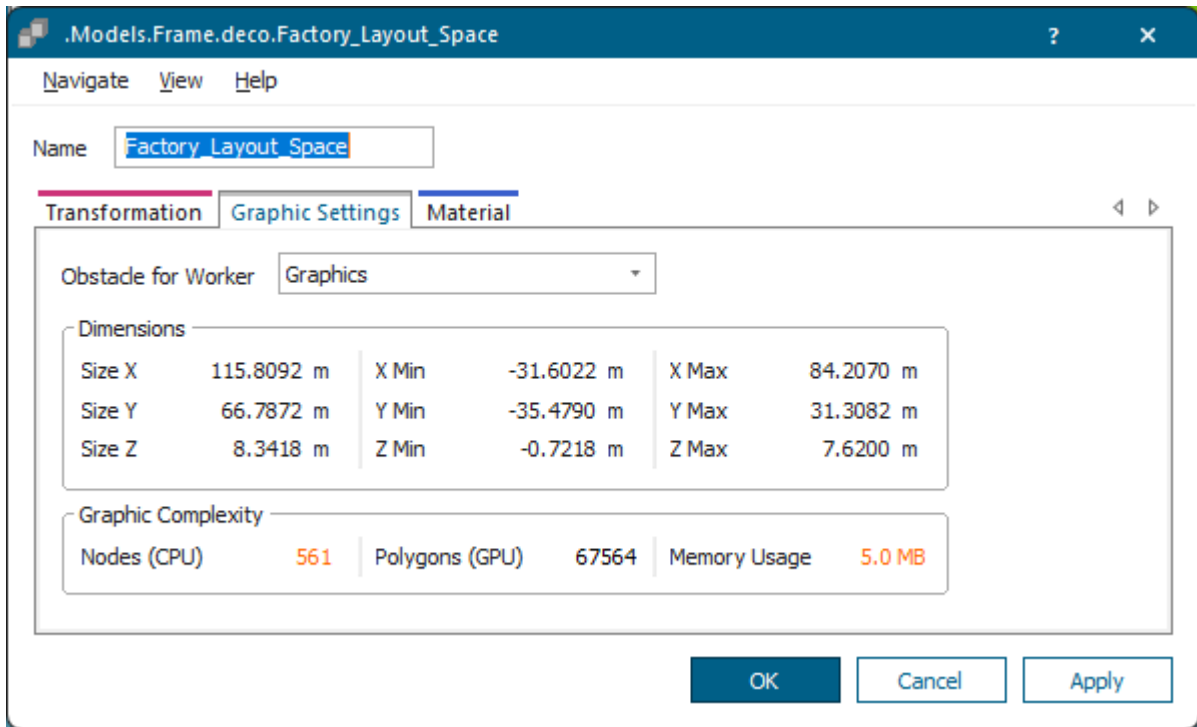
Back to [Model a Worker Who Walks Freely](#)

Model a Worker Who Walks Through Doors

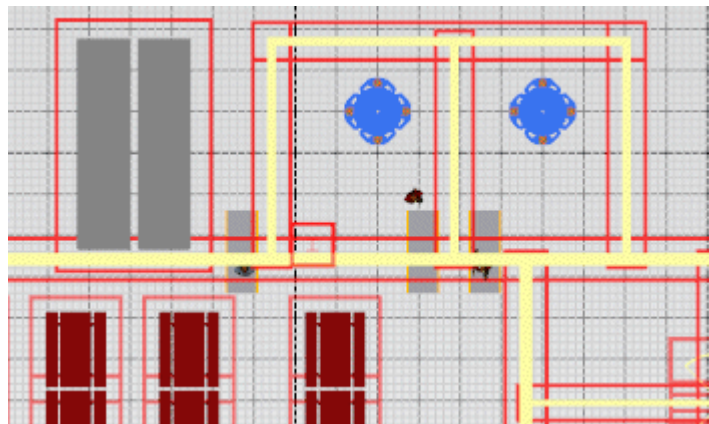
You can model a *Worker*, who has to overcome obstacles in your model, you have to use *FootPaths* even if the *Worker* walks freely within the model.

In the example below we want the *Worker* to walk through the doors from the break room onto the actual shop floor and back. For displaying the shop floor in our model we use the layout file of the actual factory.

The layout file is a graphics file which we added to the scene in 3D as the background. Although the graphics file contains all of the doors, 3D treats the factory as a single unit, whose graphics is an obstacle for the *Worker*, which the *Worker* cannot enter and cross.



When we display the **barred areas**, by pressing the **o** key, we can see this clearly by the continuous red border.



To allow the *Worker* to walk through the doors, we laid *FootPaths* through the doorways. The *FootPaths* always have to be a little longer than the depth of the obstacle.

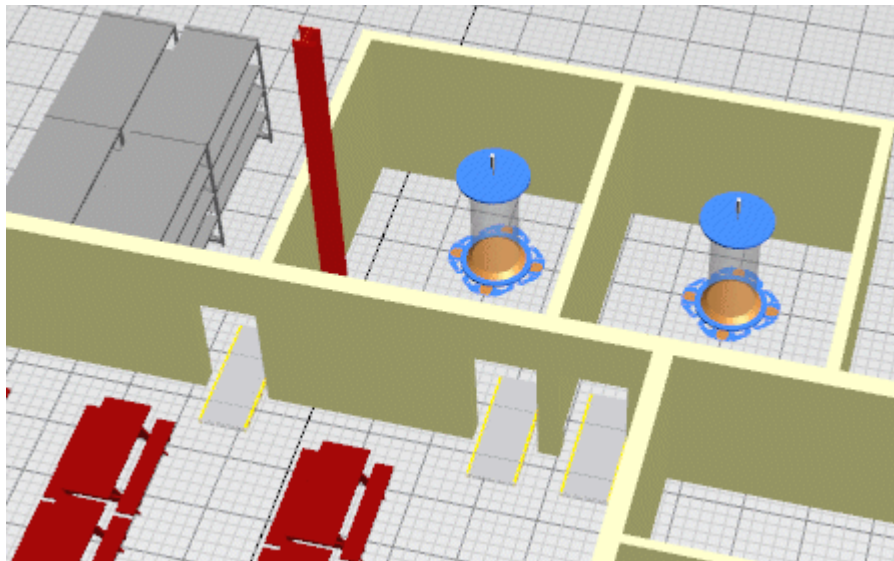
For the *FootPaths* not the length, which you enter on the tab **Attributes** is decisive, but the **length** of the *FootPath* in the 3D scene. The distance covered of the *Worker* is determined by the starting point and the end point of the *FootPath* as well as by the anchor points.

If the *FootPath* includes an **incline**, the *Worker* moves slower on the incline than on a flat surface. *Plant Simulation* achieves this by extending the imagined distance for the incline segments. When computing the **length** of an inclined segment, *Plant Simulation* triples the height difference.

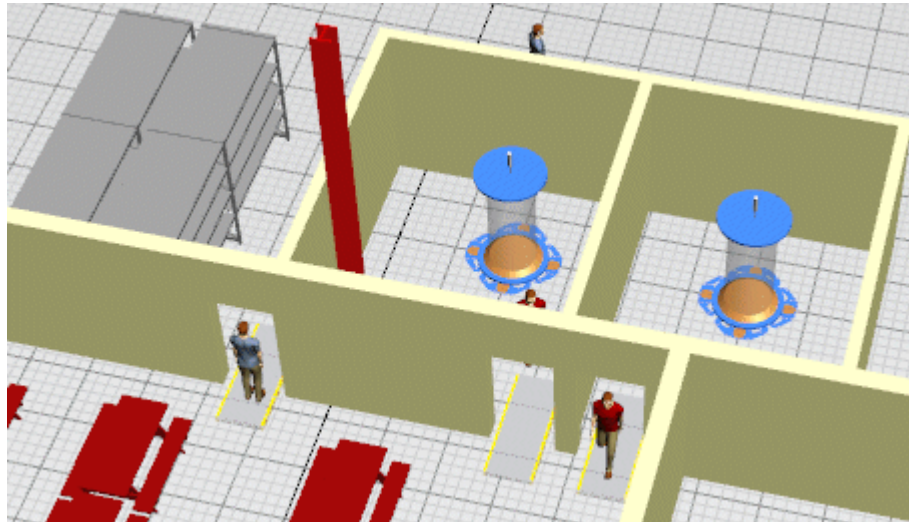
Compare this example: A segment of a *FootPath* has a dimension of 1 m in the x-direction and 1 m in the z-direction, thus actually is 1.414 m long. The imagined dimension however is 1 m in the x-direction and 3 m in z-direction, so that *Plant Simulation* computes an effective length of 3.162 m.

The *Worker* always walks on the route, which has the shortest effective length and which he can thus cover in the shortest time. This can cause the *Worker* to take an actually longer but flatter route to the destination instead of a shorter route which contains inclines and gradients.

For computing the effective route length *Plant Simulation* triples the height difference. The resulting speeds of the *Worker* are realistic in general. The speed of the *Worker* decreases to a third, when the *Worker* moves vertically up or down, for example when climbing a **ladder**. The speed decreases to about half, when the *Worker* negotiates an incline of 38 degrees, for example when walking on a **stairway**.



When we run the simulation, we can watch how the *Workers* walk through the doors of the break rooms into the anteroom and on onto the shop floor. On the shop floor itself, the *Workers* walk freely again.



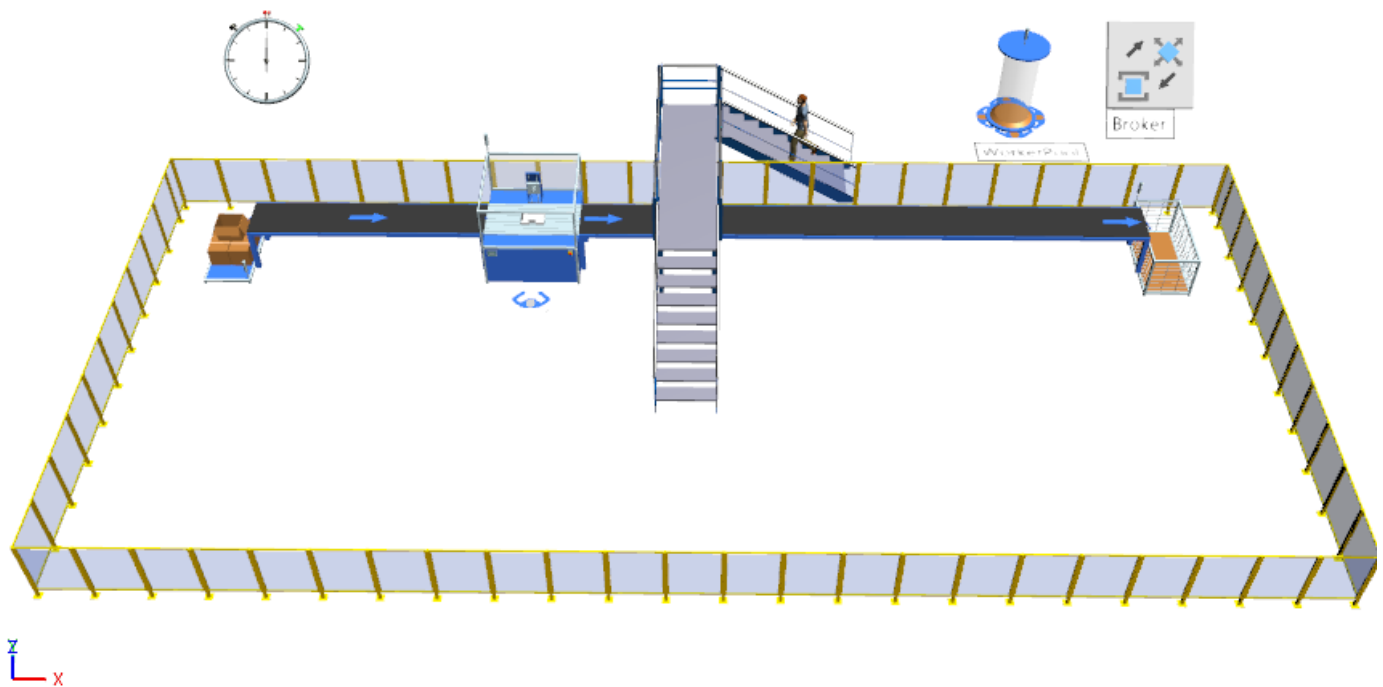
Back to [Model a Worker Who Walks Freely](#)

Model a Worker Who Climbs Stairs

In this example we model an overpass to allow *Workers* to safely cross the conveyors of a production line.

To model the overpass, we insert a set of **Stairs**, a **Mezzanine**, and a second set of **Stairs**. The *Worker* climbs the first staircase onto the platform spanning the conveyor and then climbs down the second staircase to return to the factory floor. The routing mechanism automatically sends the freely walking *Worker* on the correct path to this destination, the *Workplace* attached to the *Station* in our case.

The finished model looks like this:



Insert and Configure the Mezzanine

First, insert the platform on which the *Worker* crosses the production line.

- Click the **Edit** ribbon tab in **3D**. Click **Edit > Insert Shape > Mezzanine**.
- Select the settings for the mezzanine. We selected these settings.

Create Mezzanine ? x

Graphic Group

Dimension X m

Dimension Y m

Dimension Z m

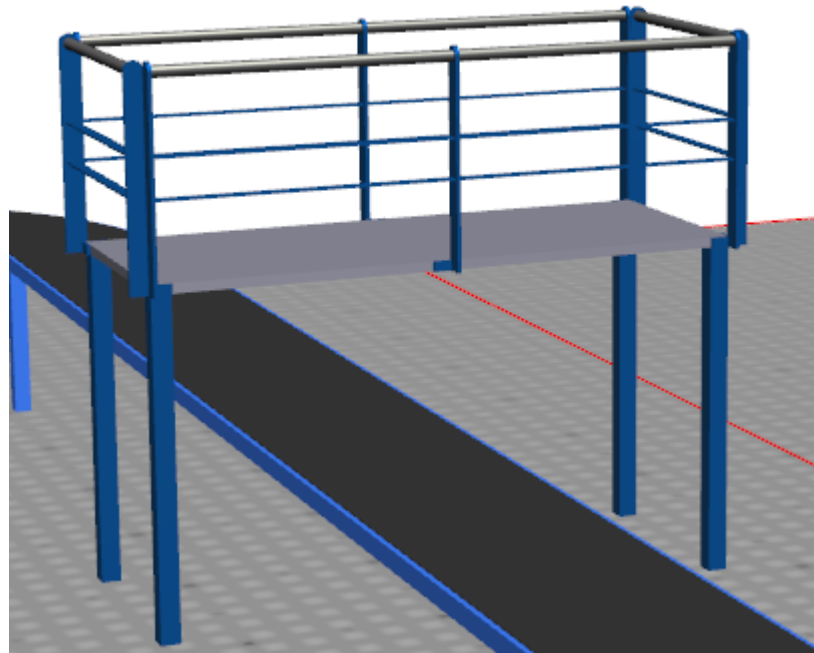
Floor Thickness m Floor Material

Post Diameter m Post Material

☒ Railings Segment Width m Material

☐ Border Line

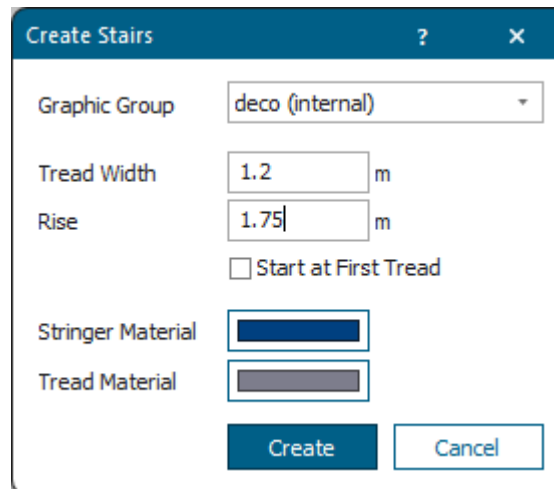
- The mezzanine looks like this:



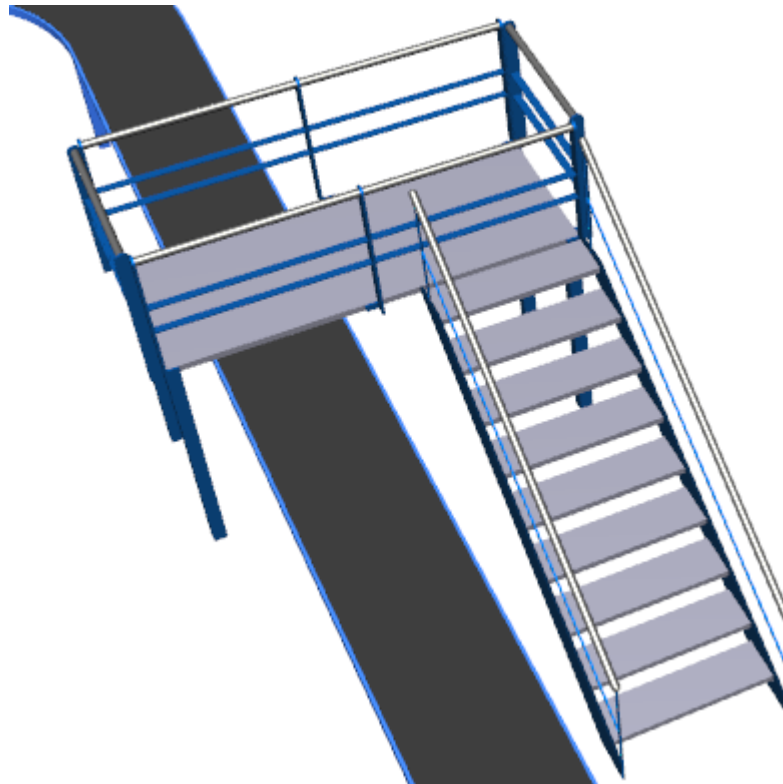
Insert and Configure the Staircases

Then, we insert the staircases, which the *Worker* climbs to get onto and off the overpass.

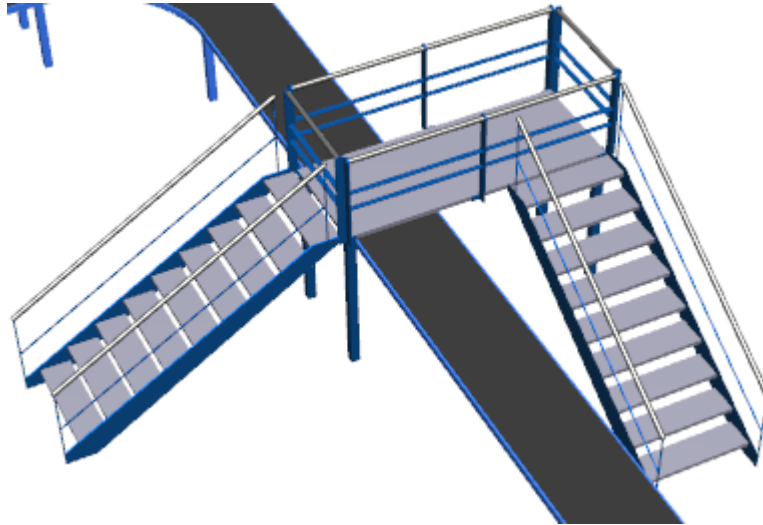
- Click the **Edit** ribbon tab in *3D*. Click **Edit > Insert Shape > Stairs**.
- Select the settings for the stairs on which the *Worker* gets onto the overpass. We selected these settings.



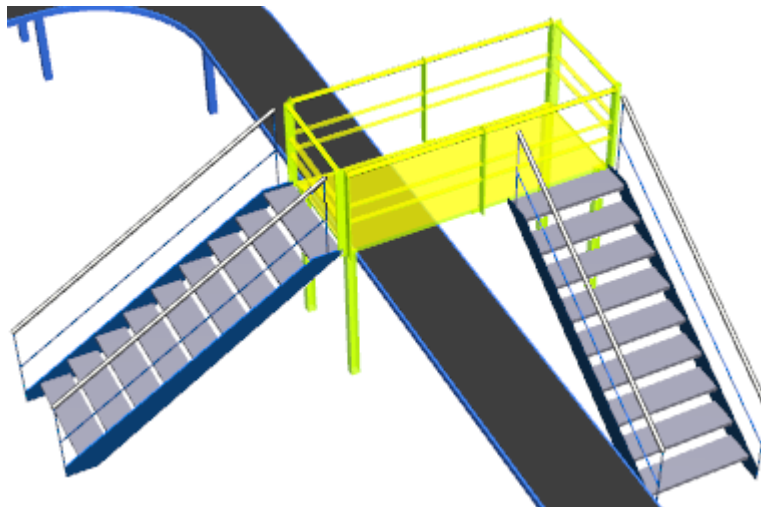
- Rotate the stairs and move it to the mezzanine until they touch. The model then looks like this:



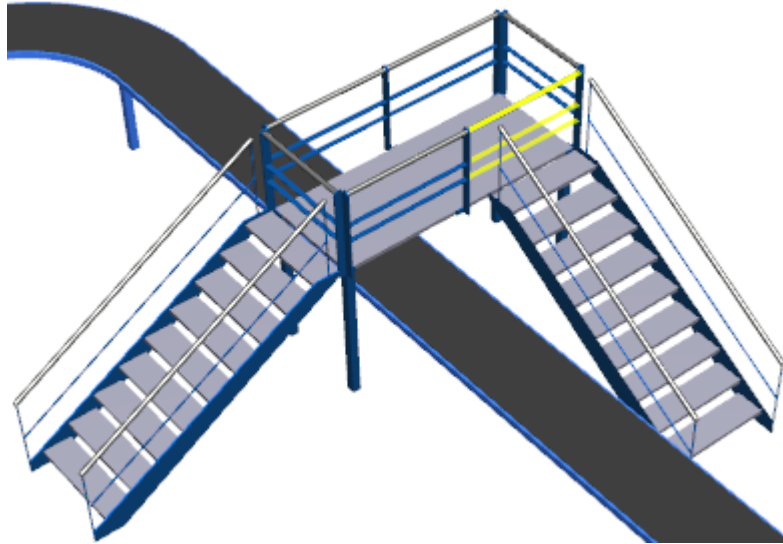
- Repeat the steps to insert a second set of stairs with the same settings. Move it to the mezzanine until they touch. The model then looks like this:



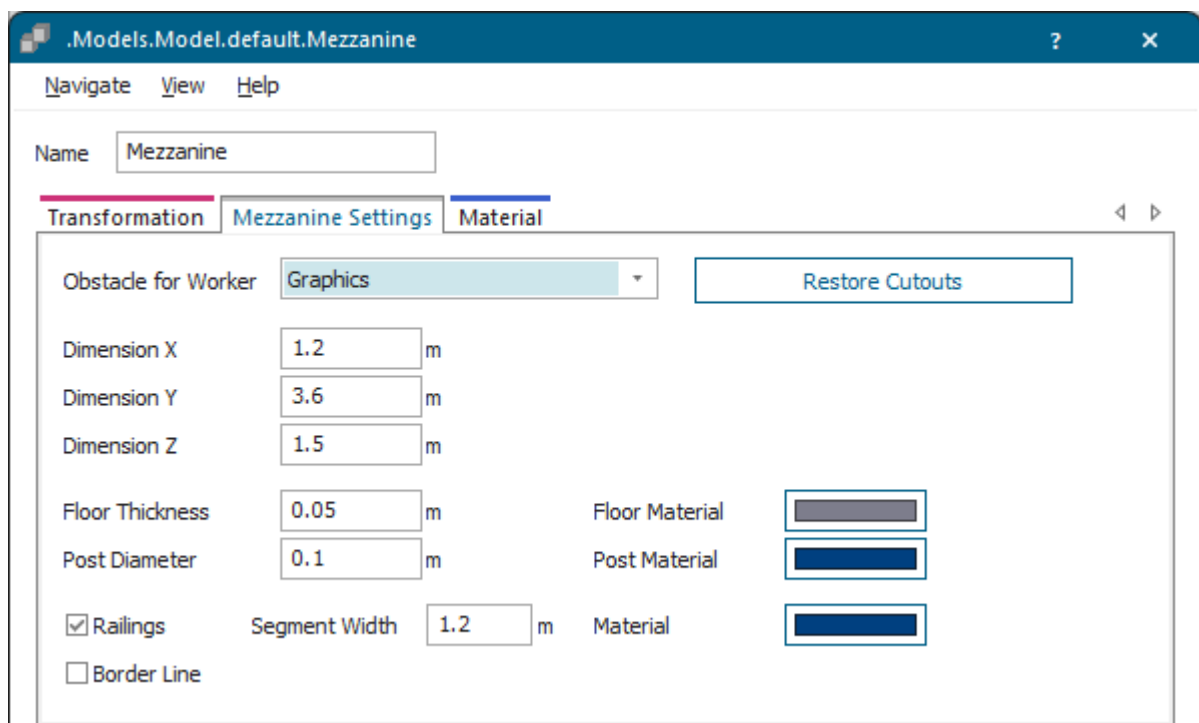
- Then remove the section of the mezzanine railing where the stairs connect to them. Otherwise our *Worker* cannot use the overpass.
 - Click the mezzanine with the left mouse button once. *Plant Simulation* selects the entire graphic of the mezzanine.



- Press the + key until only the part of the railing is selected where the stairs connect to it.



- Press **Del** to delete this section of the railing.
- Repeat this for the section at the other end of the mezzanine.
- While modeling we noticed, that our mezzanine is too short. To fix this, we selected it and pressed the spacebar. We changed the **Dimension Y** to 4.5 meters.

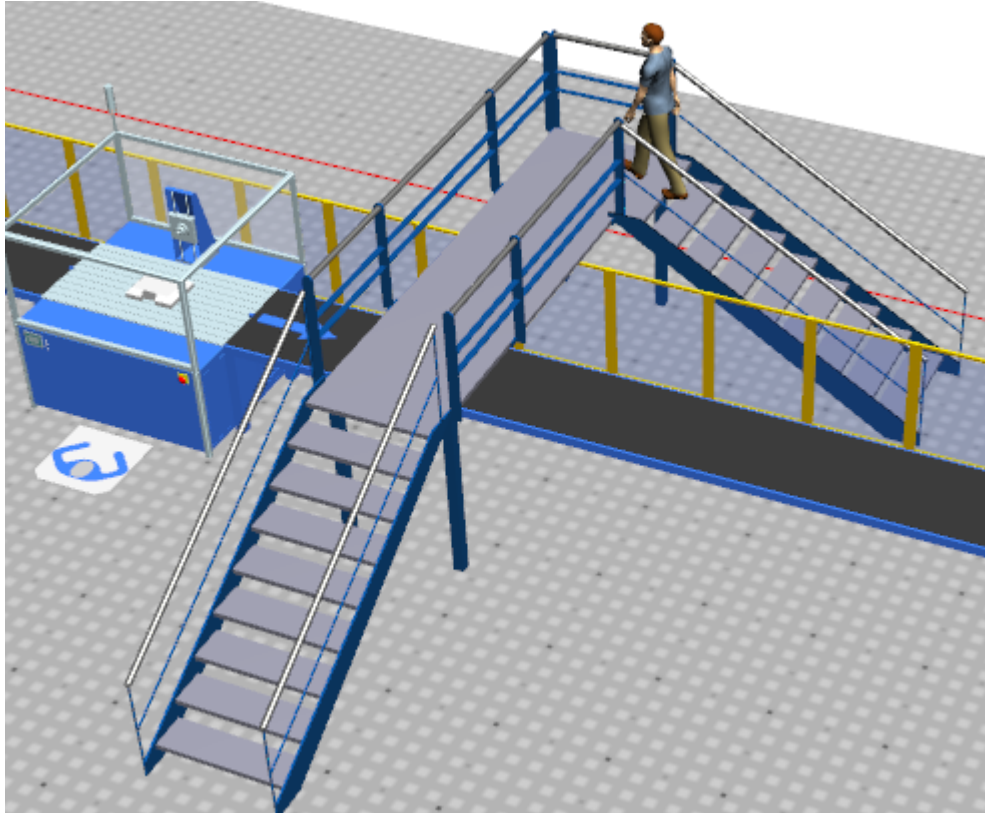


Note

When you view the **Obstacles for the Worker**, you will notice that there is a gap of 40 cm between the obstacles and the objects/graphics. This is because we supposed the width which a *Worker* needs to pass through to be 80 cm. As our *Workers* do not have width as such, you see this distance between the obstacles instead.

To modify this gap, change the setting **Worker Width** in the **Model Settings** or the **Preferences**. Reducing this gap too much can lead to unrealistic simulations due to existing safety regulations or to the *Workers* carrying parts.

- When we run the simulation, we can watch the *Worker* walk from the *WorkerPool* to the staircase, climb it, cross the overpass, and climb down the stairs on the other side.

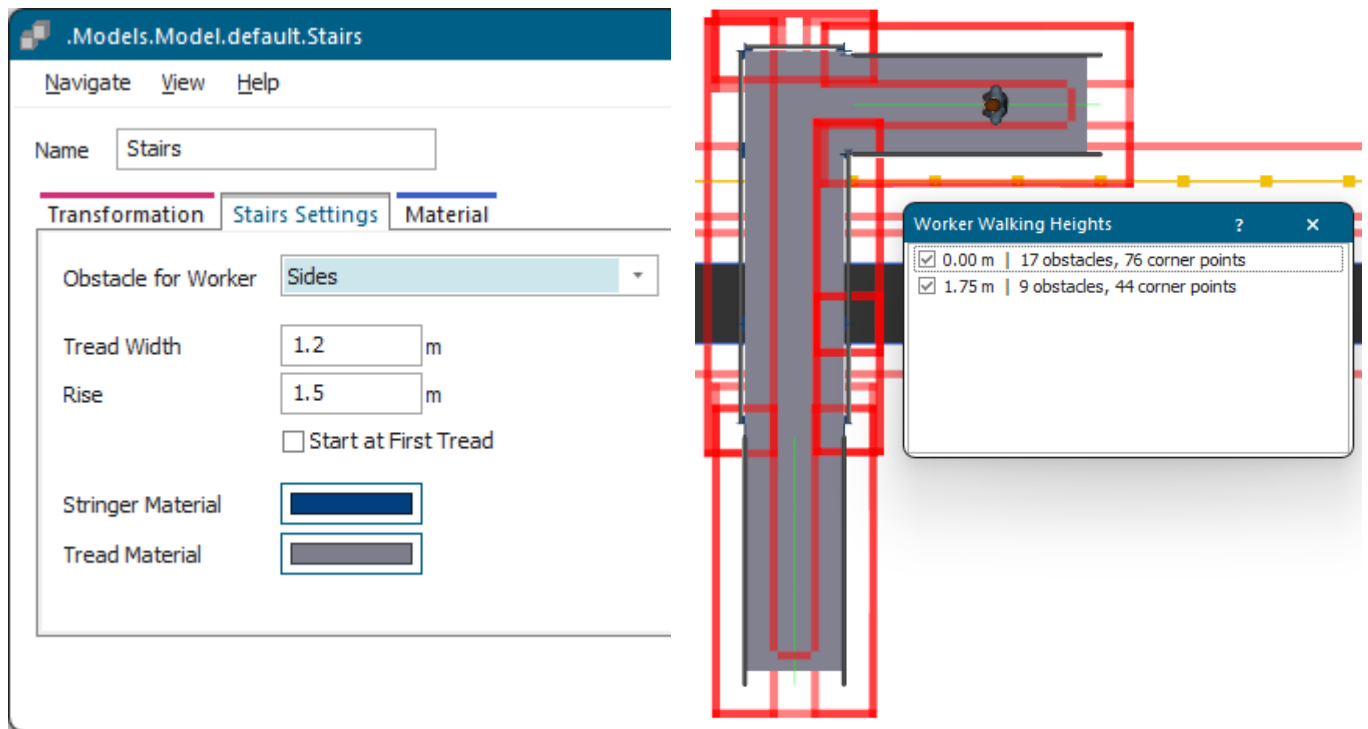


Check the Area Where the Worker Walks

For the default setting **Obstacle for Worker > Sides**, we see the way the *Worker* can take, symbolized by the green line.

Note

When you view the **obstacles for the Worker**, you will notice that there is a gap of 40 cm between the obstacles and the objects/graphics. This is because we supposed the width which a *Worker* needs to pass through to be 80 cm. As our *Workers* do not have a width as such, you see this distance between the obstacles instead.



Another view of the **Obstacle for the Worker** looks like this:



Back to [Model a Worker Who Walks Freely](#)

Model a Worker Who Waits in Line

Model a Worker Who Waits in Line

You can model a *Worker*, who **walks freely within the area** of the simulation model and who **stands in line** in front of a *Workplace* which he cannot enter.

We demonstrate how to:

- [Model a Worker Who Queues Up](#)
- [Model Workers Who Walk to the End of the Queue](#)
- [Model Workers Who Walk Along the Queue](#)

Worker 

[Back to Model Workers and the Jobs They Do](#)

Model a Worker Who Queues Up

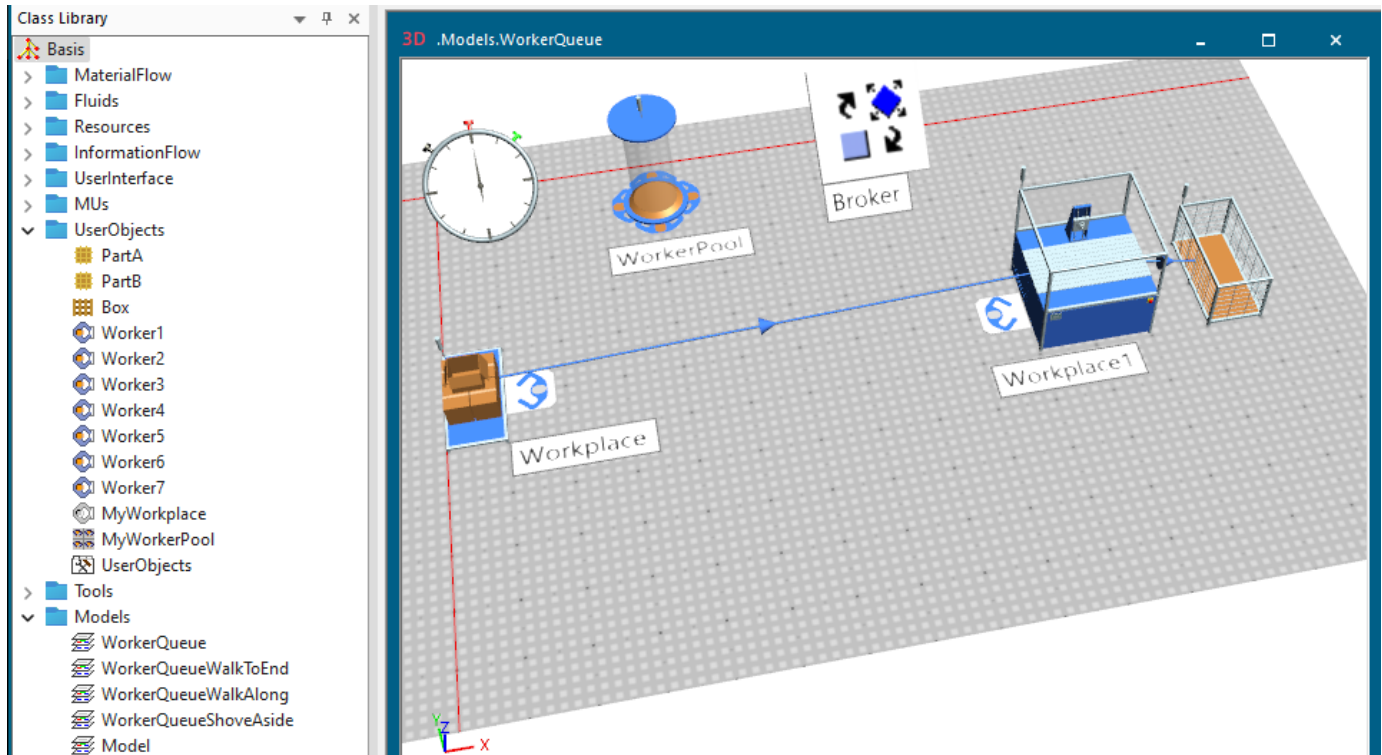
You can model *Workers* who **walk freely within the plant** and who **stand in line** in front of a *Workplace* which they cannot enter because it is still occupied.

With the default setting of the *Workplace* the *Workers* first walk to *Workplace1*, then jump into the queue in front of *Workplace1*, and stand in line until they can step onto it to deliver the *Part*.

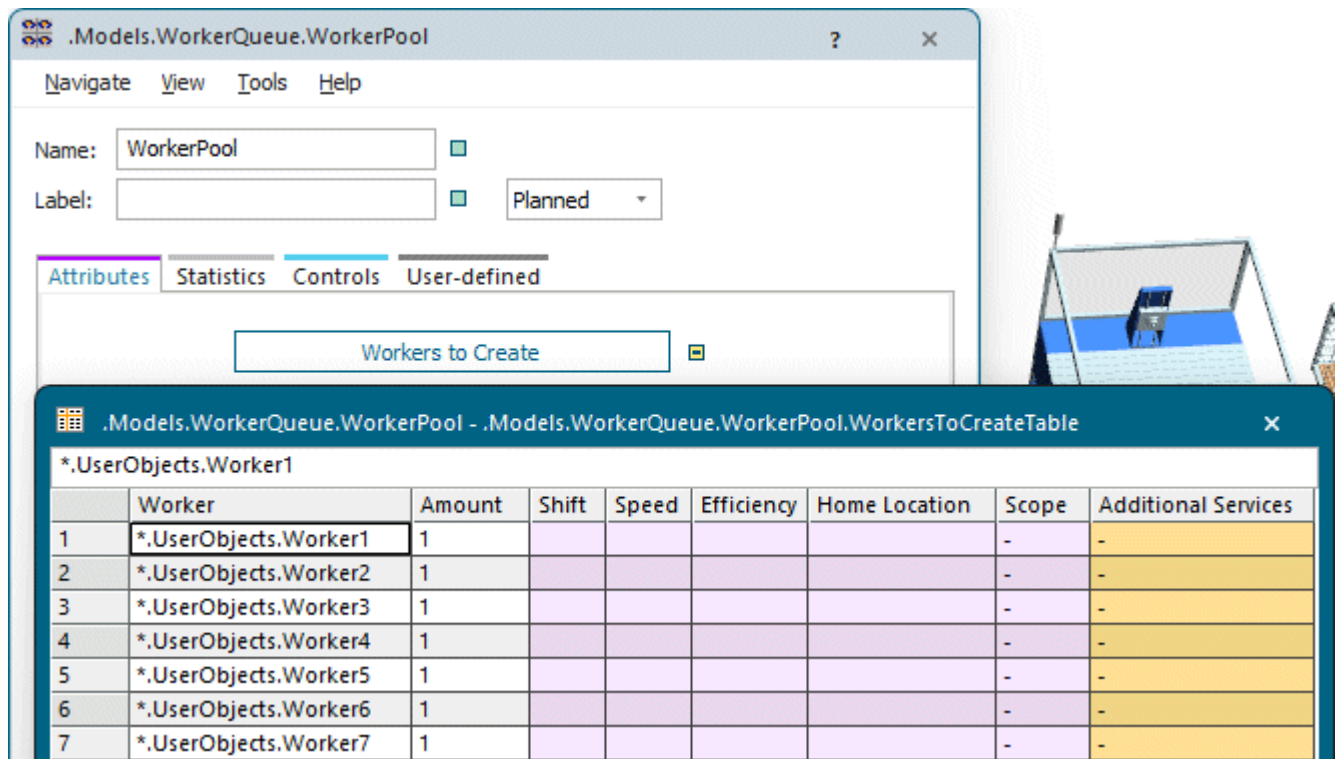
Capacity	1	☐
	☒ Worker Stays Here After Completing the Job	☐
Distance in Queue	1	☐ m
	☐ Walk to the End of the Animation Path of the Queue	☐
	☐ Walk Along the Animation Path of the Queue	☐

[How Workers Queue Up in Front of the Workplace](#) describes the general mechanism in detail.

Insert the objects into the simulation model so that they match the screenshot below:

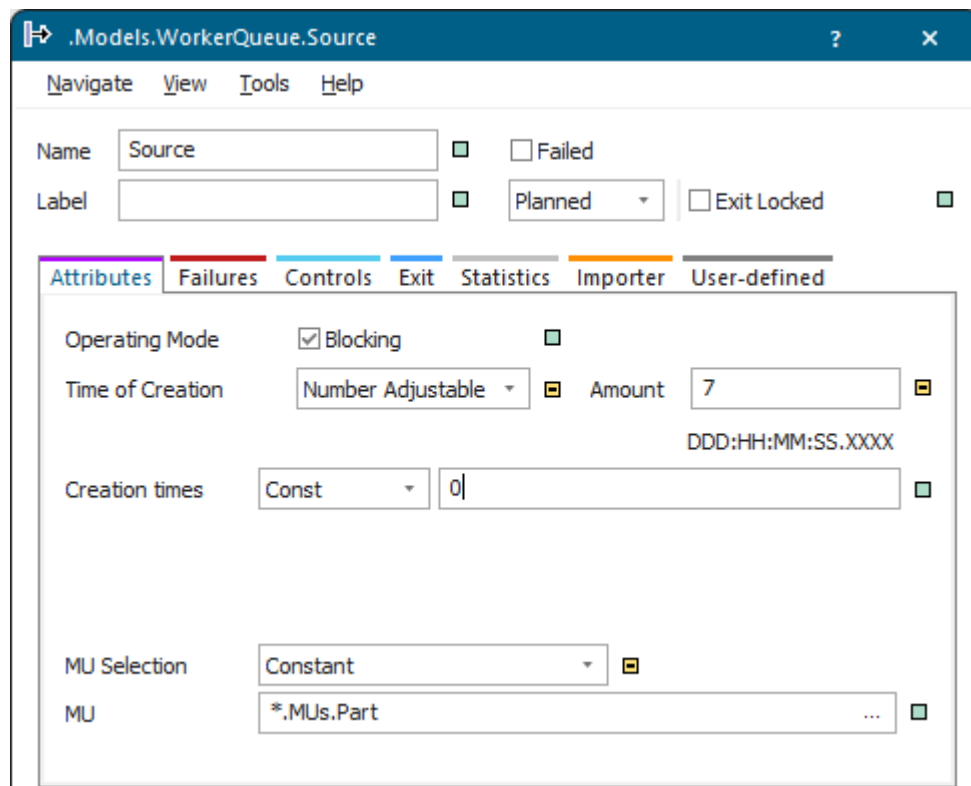


- Our *WorkerPool* provides 7 *Workers*. To distinguish the *Workers* from each other, we assigned a different color each to their T-Shirts.

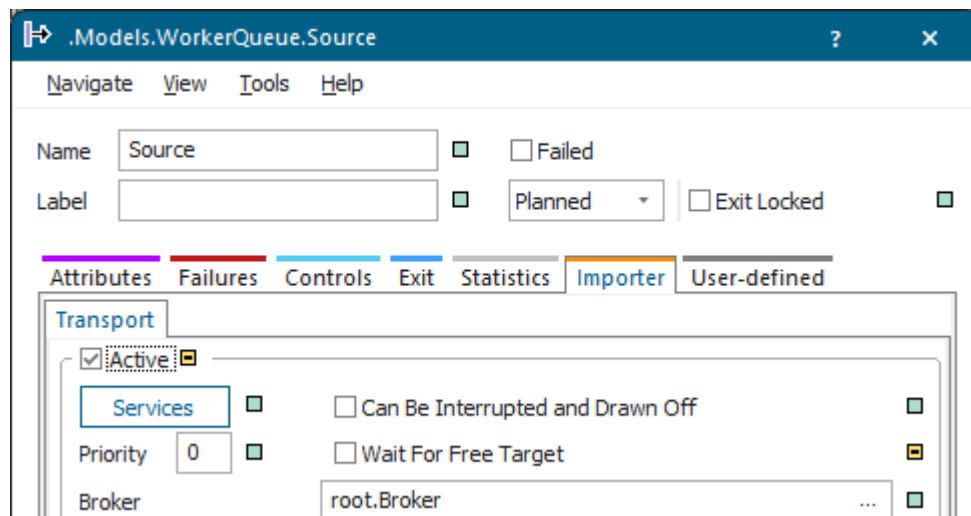


- The *Broker* brokers the services. We use the *Broker* with its default settings.

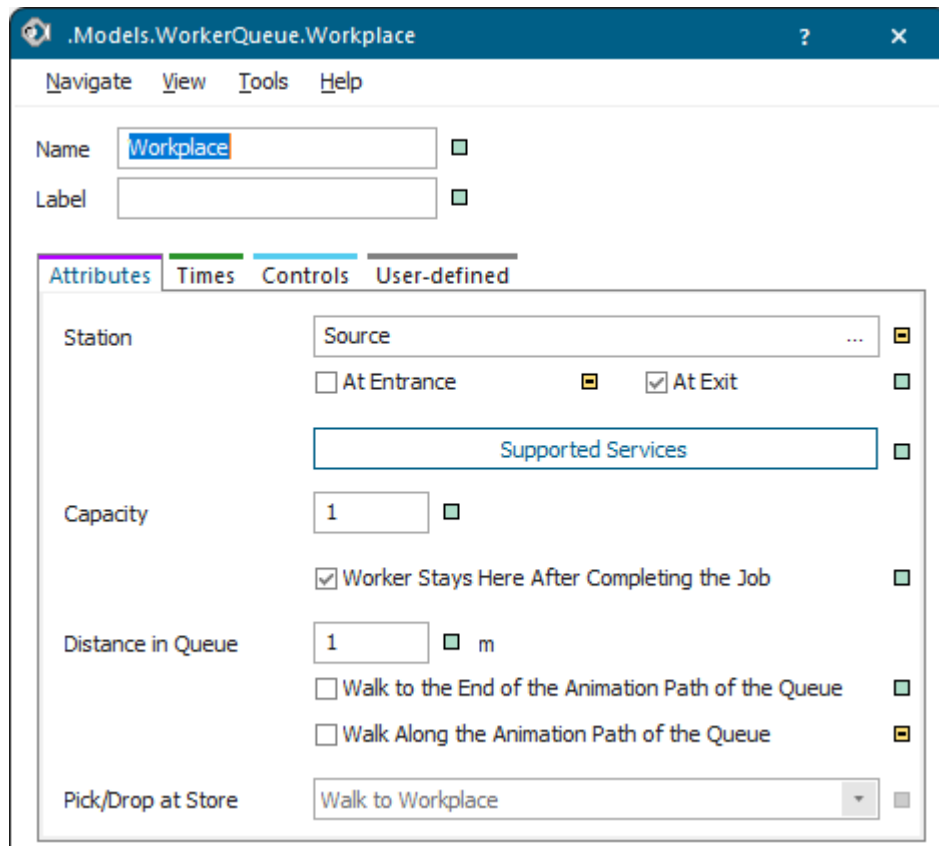
- The *Source* produces 7 *MUs* of type *Part*.



To enable the *Worker* to carry the parts away, activate the *Transport importer*. Clear **Wait for Free Target** to immediately request a *Worker*.



- As we want the *Worker* to pick the produced *Parts* up at the *Source*, we need a *Workplace*. Drag the *Workplace* onto the *Source* and drop it there.

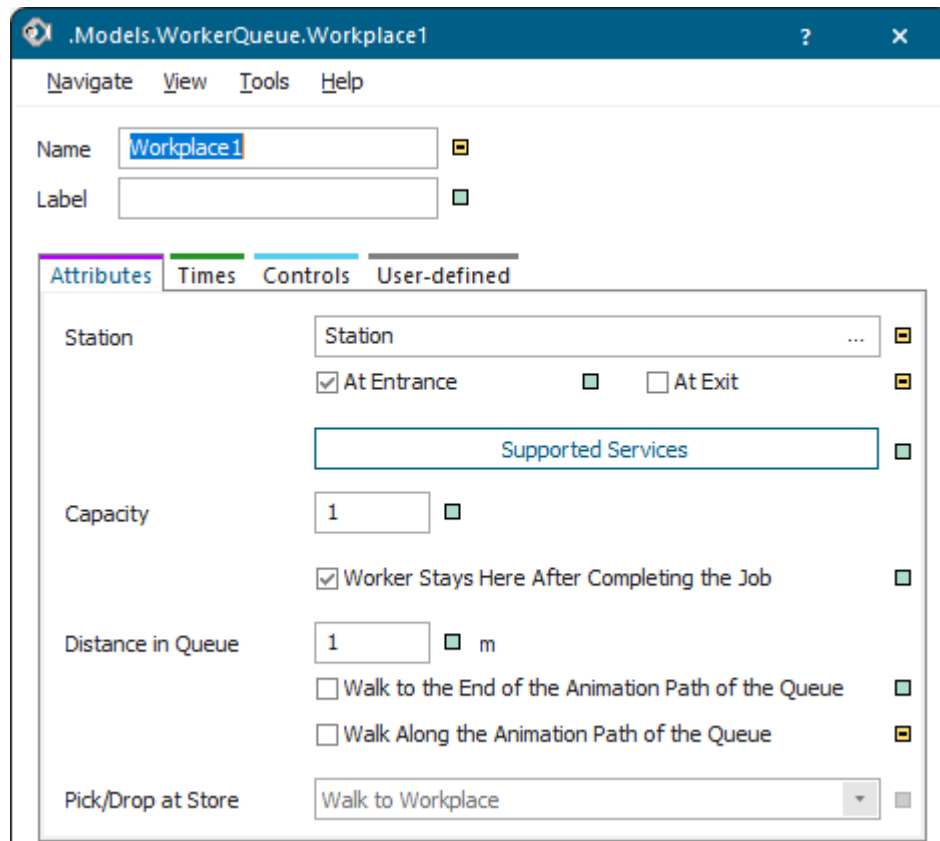


- The *Station* processes the *Parts*. We use the *Station* with its default settings.
- As we want the *Worker* to place the *Parts* it carried from the *Source*, we need a second *Workplace*. Drag the *Workplace* onto the *Station* and drop it there.

Select **Transport Importer** in the window that opens.

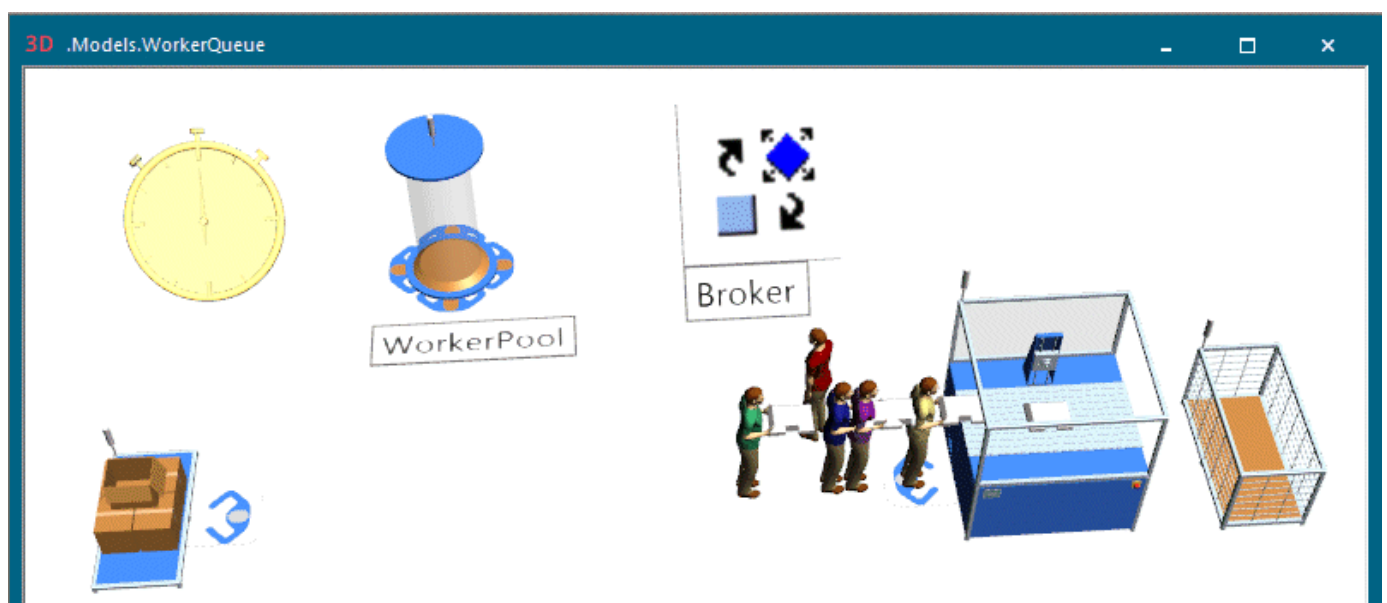


Plant Simulation automatically selected the check box **At Entrance** to attach the *Workplace* to the entrance of the *Station*.

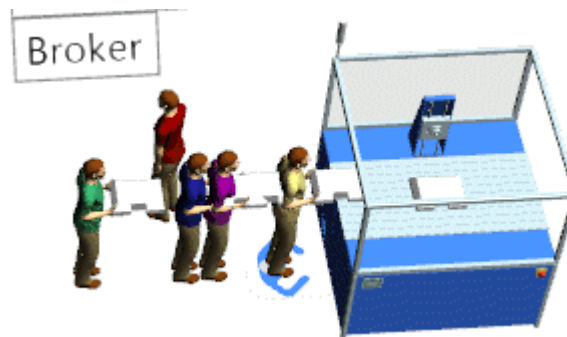


- The *Drain* removes the *Parts* from the plant. We use the *Drain* with its default settings.
- Connect the material flow objects with *Connectors*.

Run the simulation. You'll see that the *Workers* first walk to *Workplace1*, then jump into the queue in front of *Workplace1*, and stand in line until they can step onto it to deliver the *Part*.



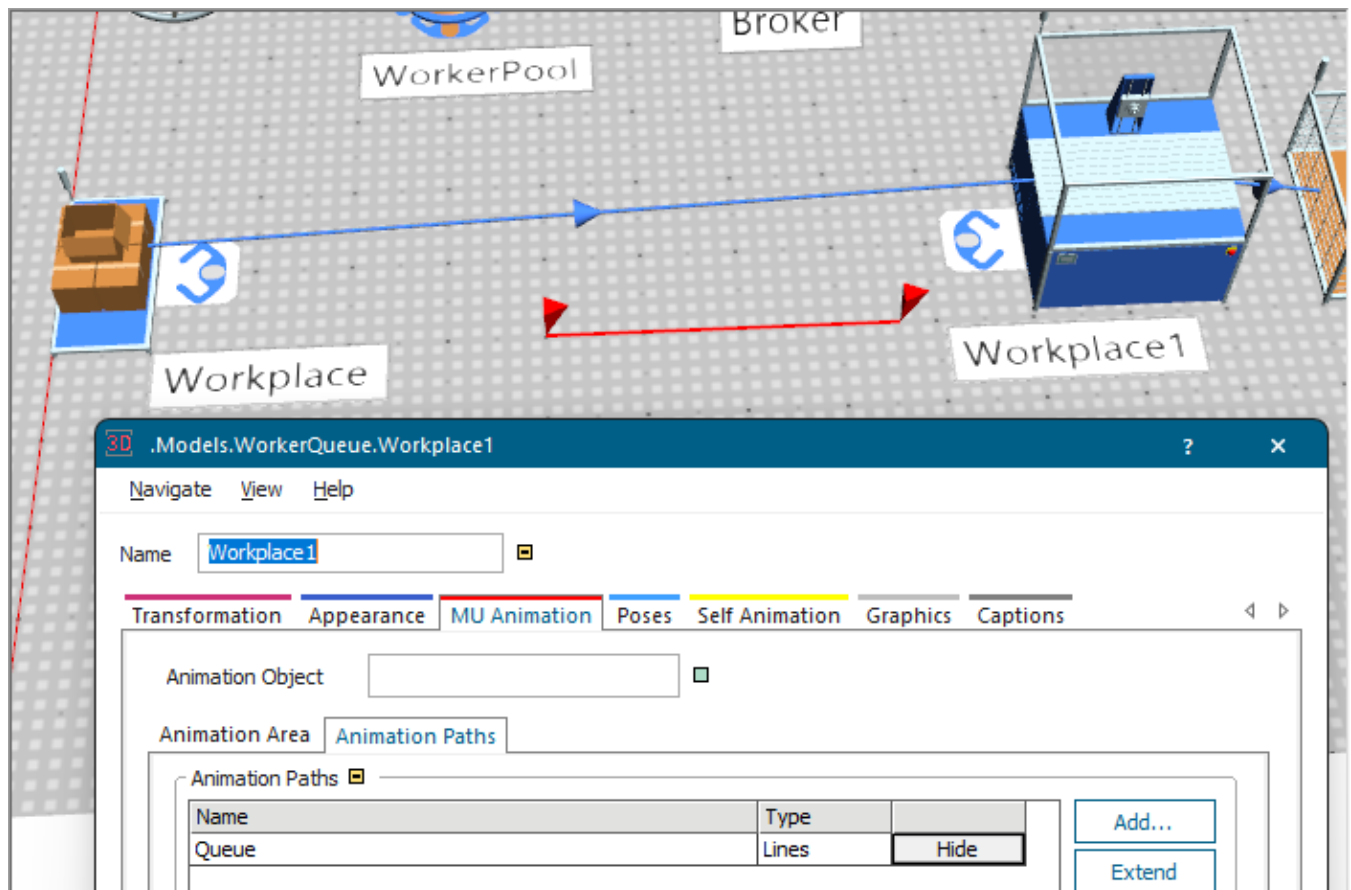
You'll also notice that the *Workers* seem to walk through each other while standing in line.



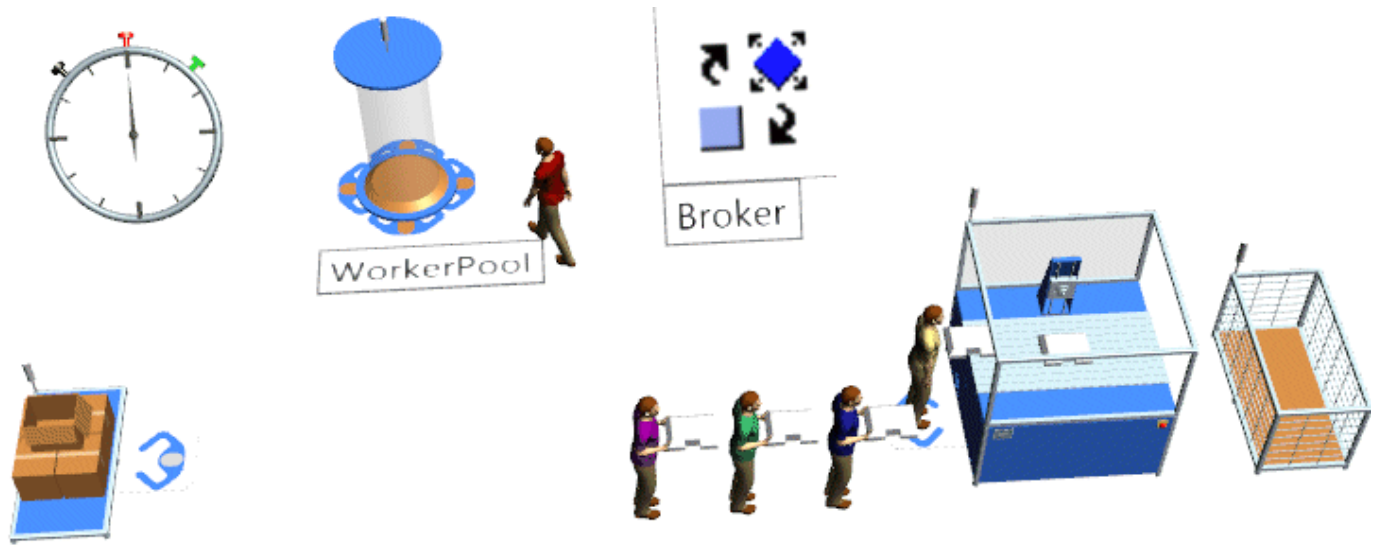
This looks unrealistic and is caused by the straight animation path that is placed directly in front of the center of the *Workplace* by default.

This is easy enough to fix though:

- Select the *Workplace* and press the **spacebar**.
- Change to the tab **MU Animation > Animation Paths** and click **Show** to show the animation path.
- Click the path markers and move them. We held down **Shift** and pressed the **Down Arrow** to move the animation line on grid unit down.



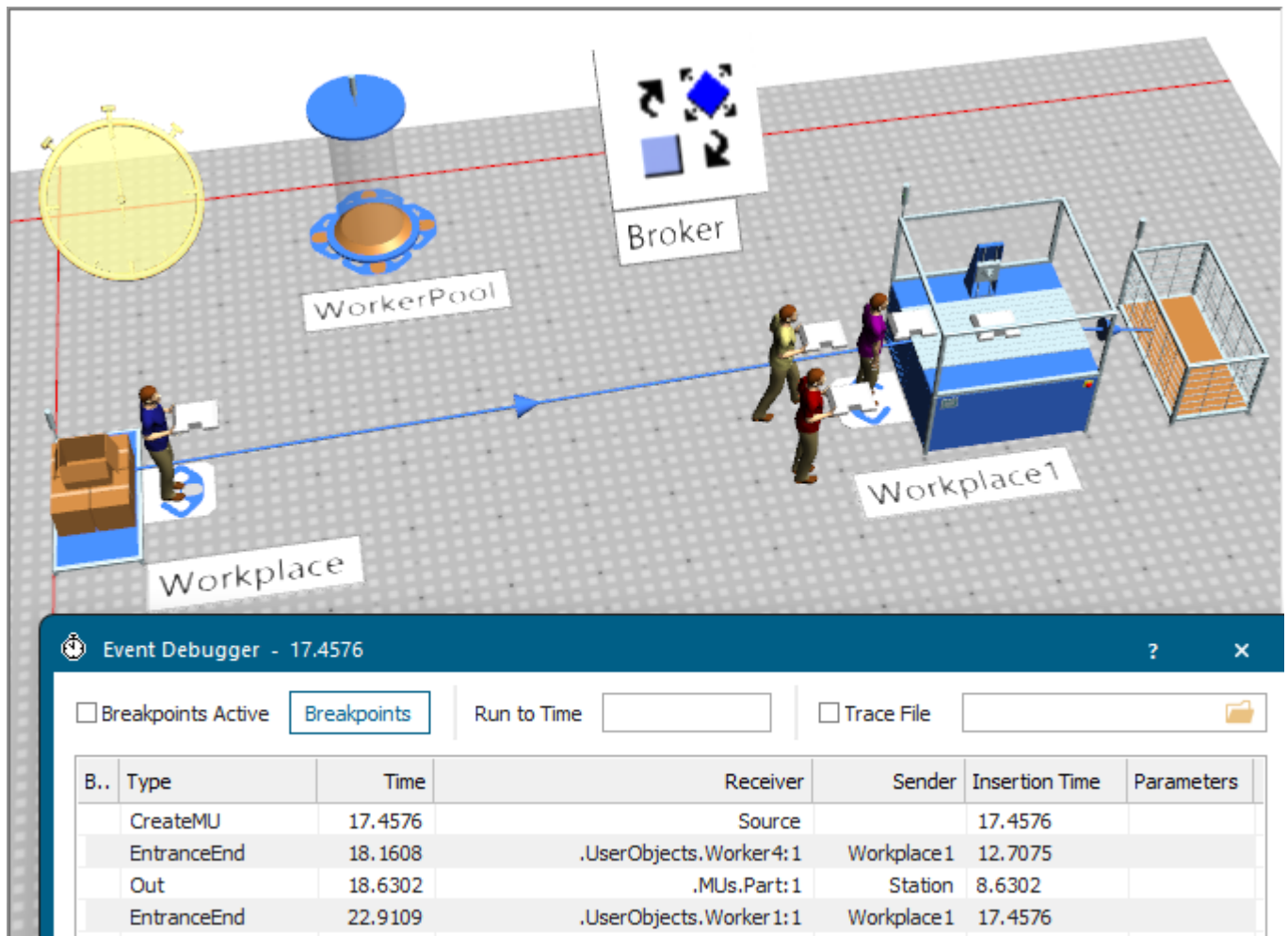
- Click **OK** in the dialog and run the simulation again.



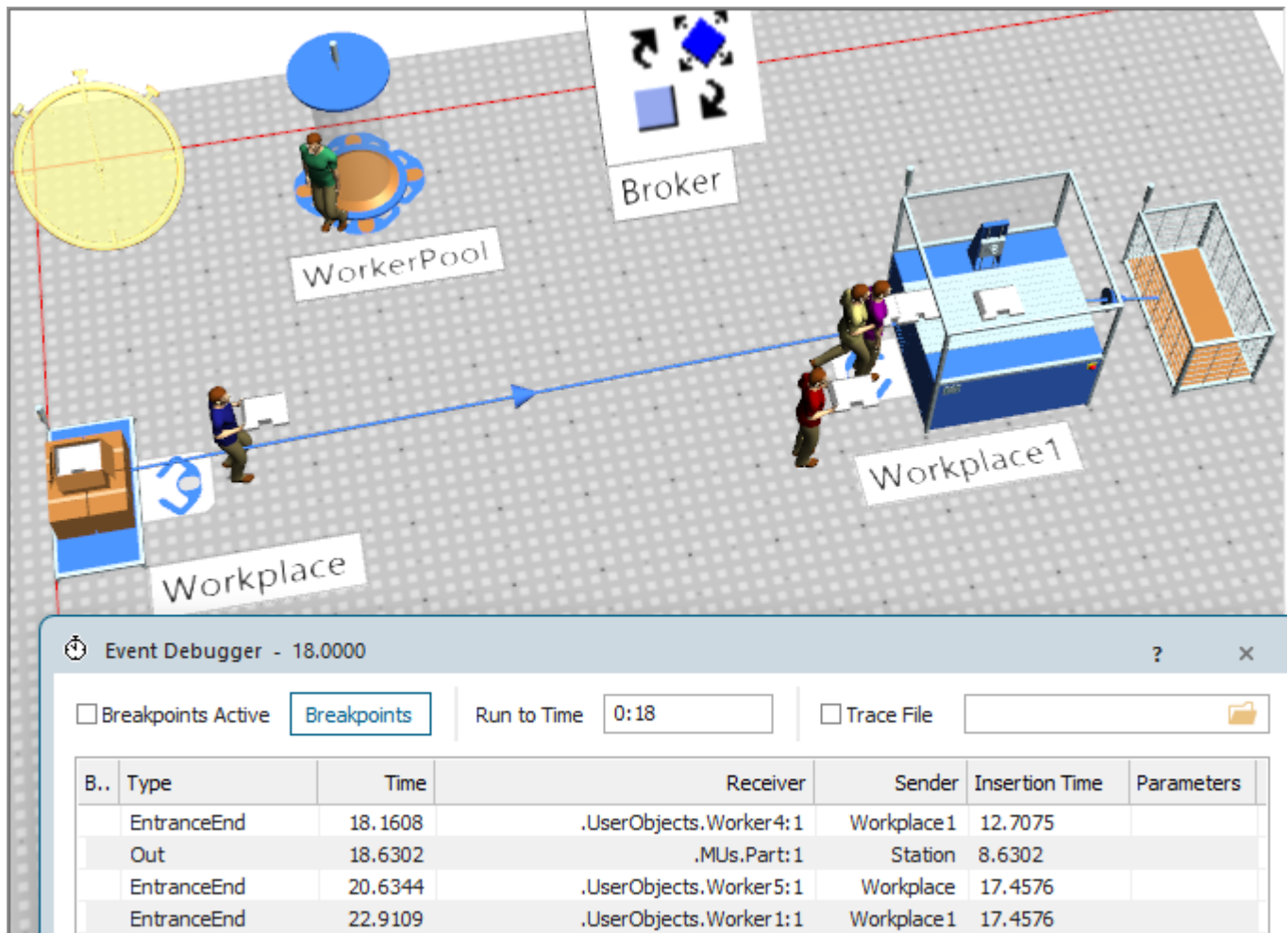
Now it looks good.

As we want to see that the *Worker* first steps onto the *Workplace* and then jumps to the end of the line, we open the *EventDebugger*. We click  to follow our *Worker* on his way from the *Workplace* attached to the *Source* to *Workplace1* attached to the *Station*. Once there he has to place the part and has to wait in line if this *Workplace* is still occupied by another *Worker*.

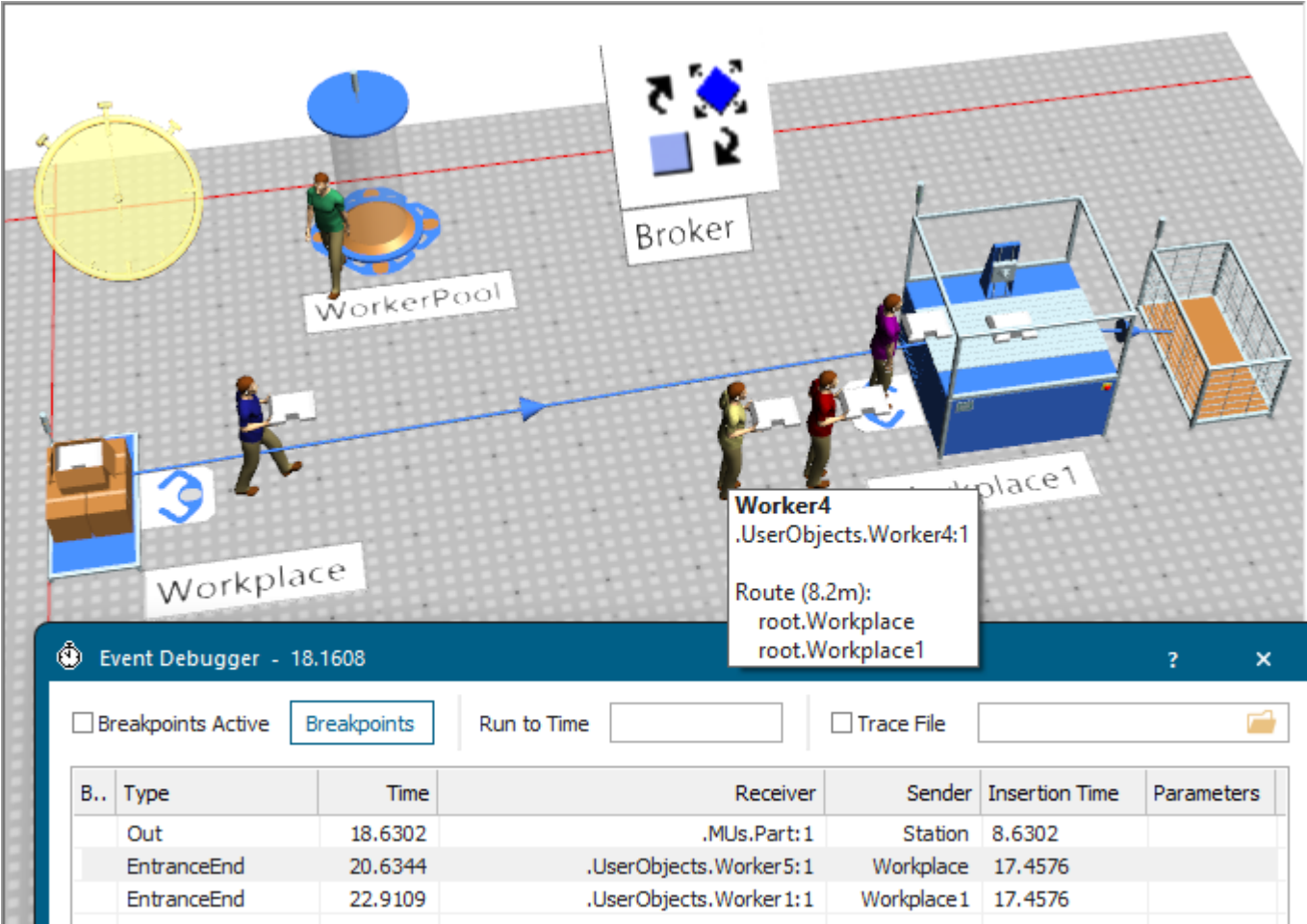
In the screenshot below *Worker2* stands on *Workplace1*, *Worker3* stands in line, and *Worker4*, who is wearing the yellow T-Shirt, is several steps away from *Workplace1*.



After clicking  several times, we see that *Worker4* steps onto *Workplace1* at the time 18.1608.



After the next step *Worker4* jumps from *Workplace1* to the end of the queue.

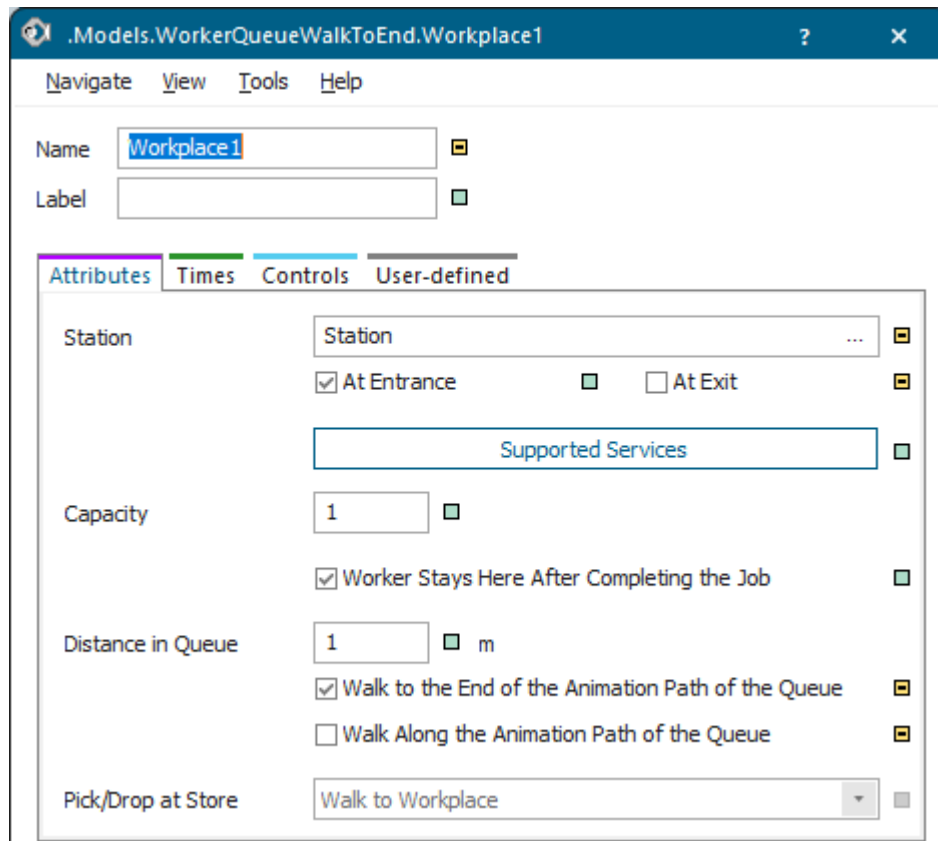


Back to [Model a Worker Who Waits in Line](#)

Model Workers Who Walk to the End of the Queue

You can model *Workers*, who walk to the end of the queue, and then move up the queue to the *Workplace* where they deliver the *Part*.

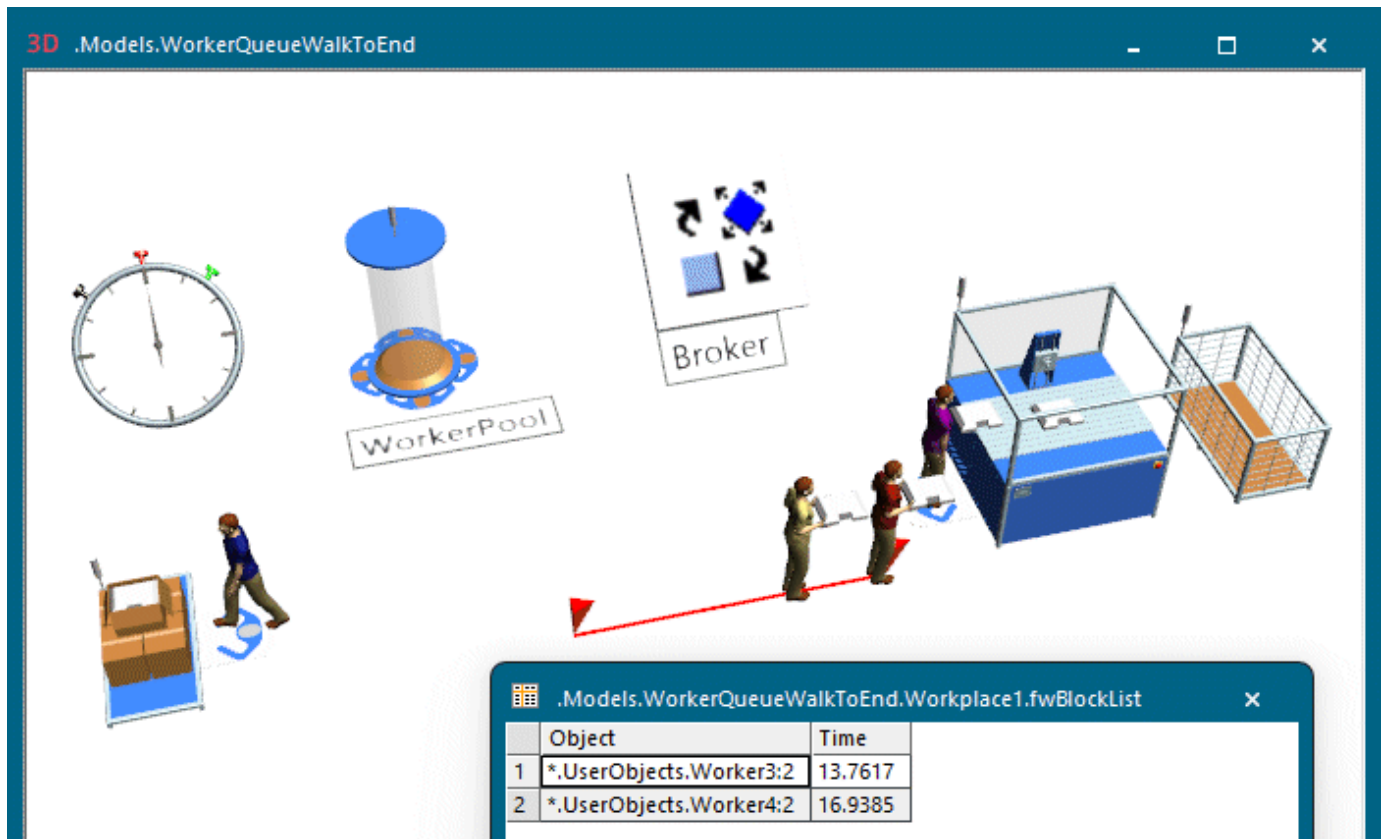
To do so, we use the existing model where we select the check box **Walk to the End of the Animation Path of the Queue** in the *Workplace* attached to the *Station*.



Run the simulation. You'll see that our *Worker4*, the one wearing the yellow T-Shirt, walks to the end of the queue in front of the *Workplace* and then to the *Workplace* once *Worker3* has delivered his part. The **Forward Blocking List** of *Workplace1* shows this.

Note

For demonstration purposes we show the **MU animation path named Queue** on which the **Workers** wait.

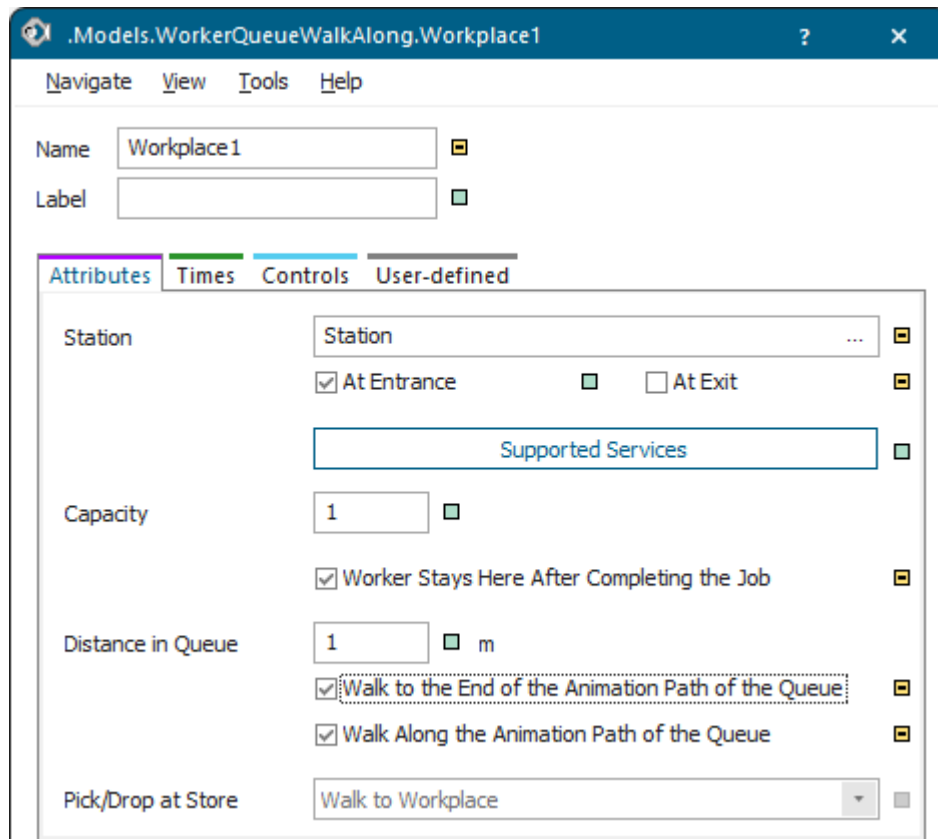


Back to [Model a Worker Who Waits in Line](#)

Model Workers Who Walk Along the Queue

You can model *Workers*, who walk to the end of the queue, and then along the queue behind the *Workers* in front of them, to the *Workplace* where they deliver the *Part*.

To do so, we use the existing model where we select the check box **Walk Along the Animation Path of the Queue** in addition to **Walk to the End of the Animation Path of the Queue** in the *Workplace* attached to the *Station*.

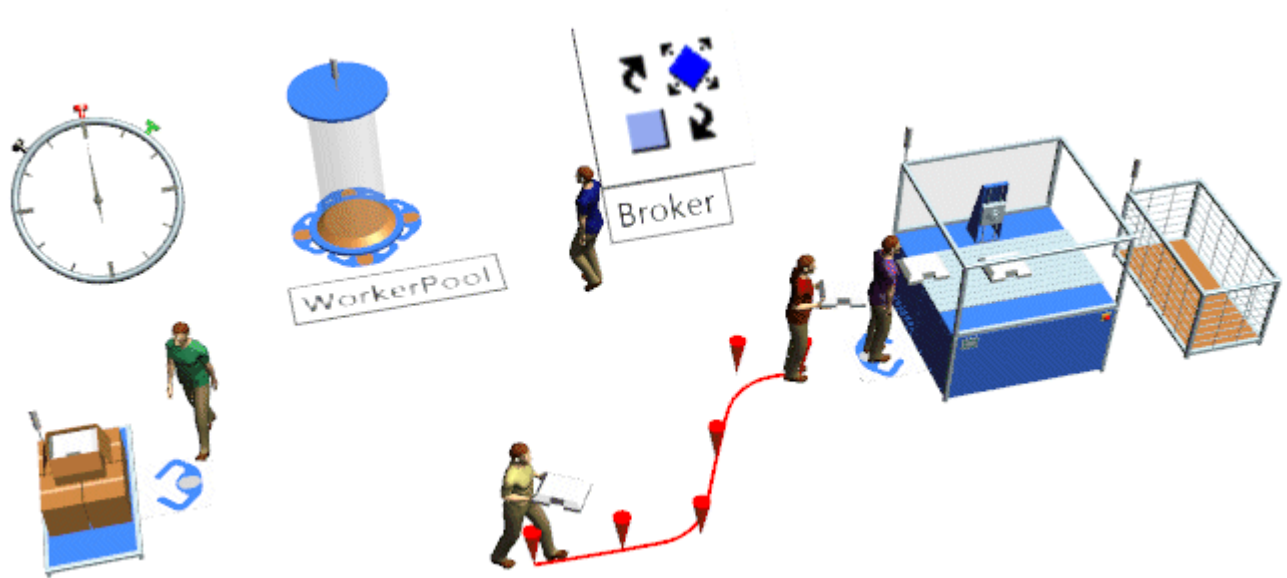


Run the simulation. You'll see that *Worker4* walks to the end of the queue in front of the *Workplace* and then along the queue, keeping his place in the queue, to the *Workplace* or to the **waiting position**.

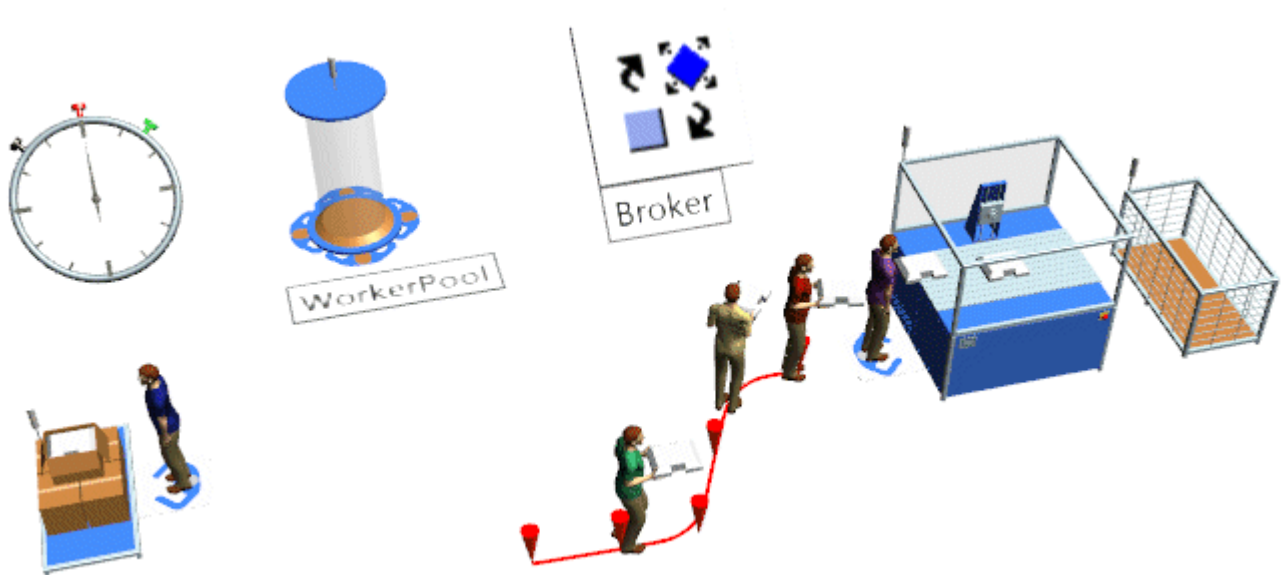
Note

For demonstration purposes we show the **MU animation path named Queue** on which the **Workers** wait. In this case we modeled it as a spline path.

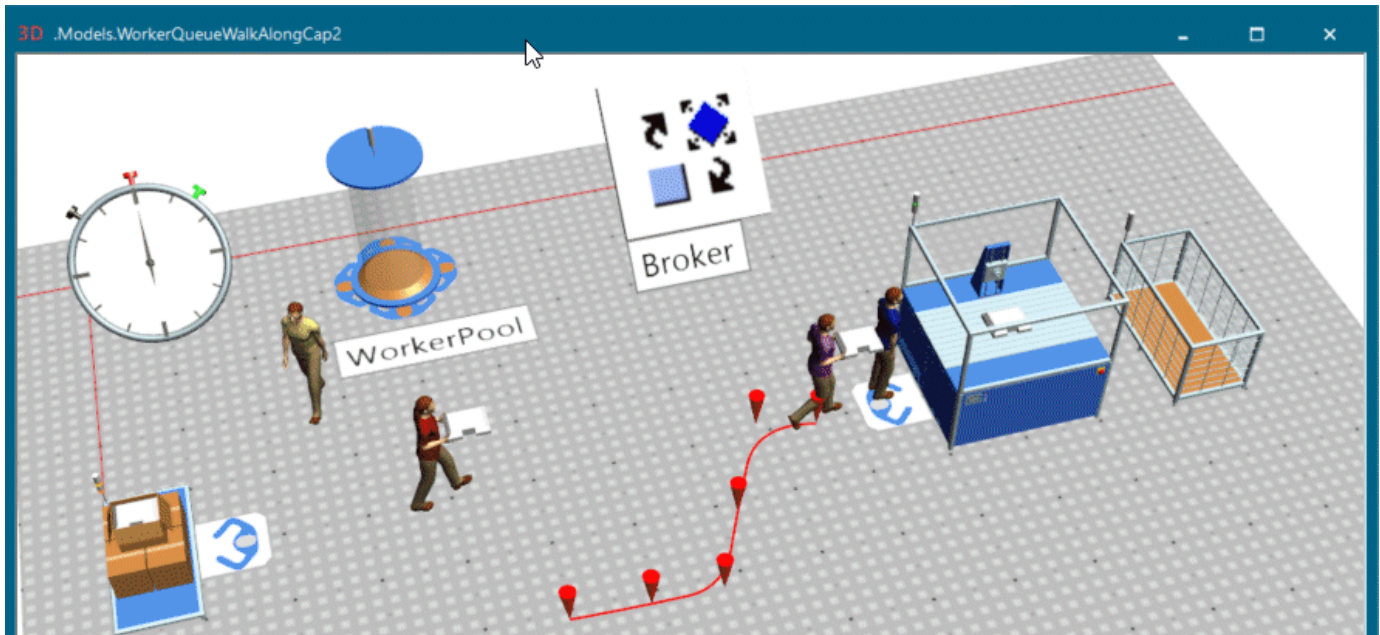
Here we see how *Worker4* steps on the visualized queue.



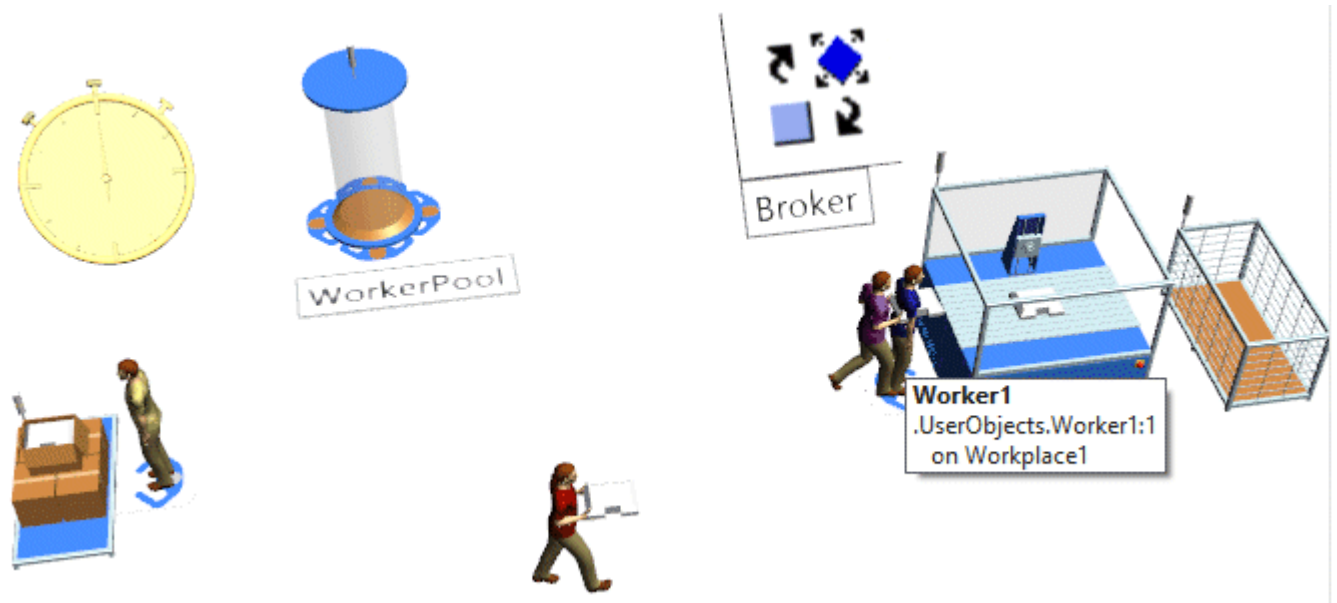
Here we see how he moves up in the queue and waits his turn to deliver his part.



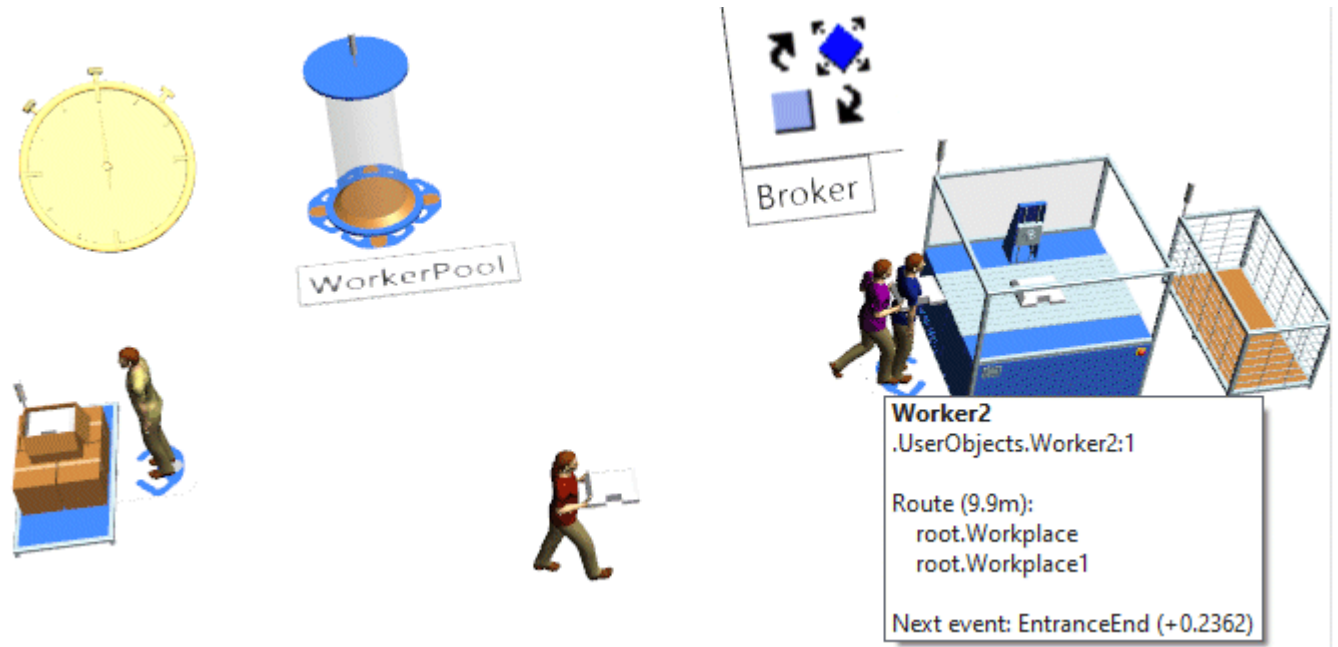
If you clear the check box **Walk to the End of the Animation Path of the Queue** of *Workplace1*, you'll notice that the *Workers* neither walk to the end of the waiting queue nor do they walk along the queue while *Workplace1* is available. Instead, they walk directly from the *Workplace* to *Workplace1*. *Workers* that do have to wait until *Workplace1* becomes available, wait in line in the queue on the defined animation line.



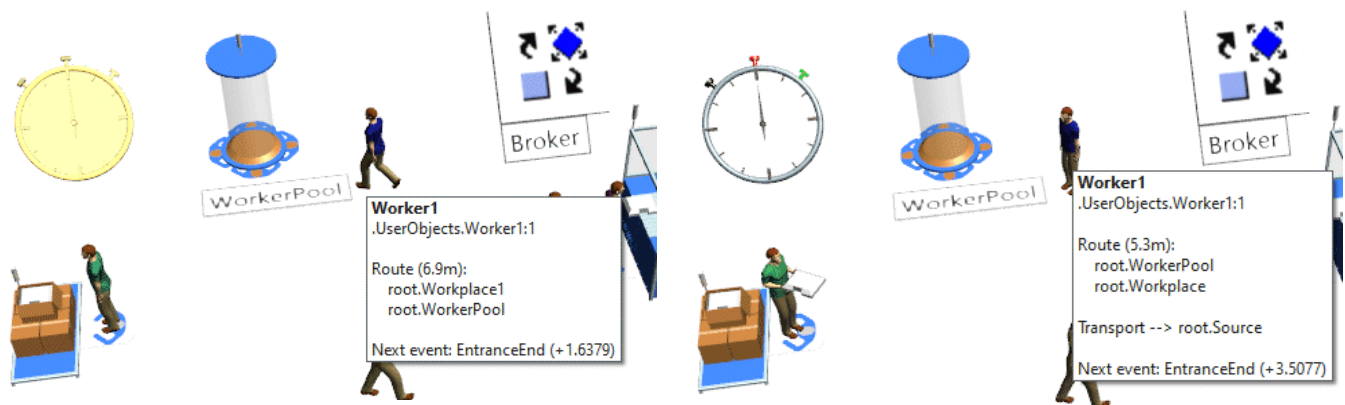
You'll also notice that the *Worker*, who stays on *Workplace1* while another *Worker* has to work on it, is shoved aside if the **Capacity** of *Workplace1* is used up. In the example *Worker1*, wearing the blue T-shirt, is shoved aside by *Worker2* wearing the maroon T-shirt. The tooltip of *Worker1* shows this.



The *Worker* who is shoved aside then exits the *Workplace* and walks to the *WorkerPool* or to the **Home Location** if you configured one.



In our case *Worker1* walks toward the *WorkerPool*, receives a new work order on his way there, does not continue to walk to the *WorkerPool* but walks to the *Workplace* again, and picks up another part, which he carries to *Workplace1*. The tooltip of *Worker1* shows this.



Back to [Model a Worker Who Waits in Line](#)

Model a Worker Who Returns to the Home Location

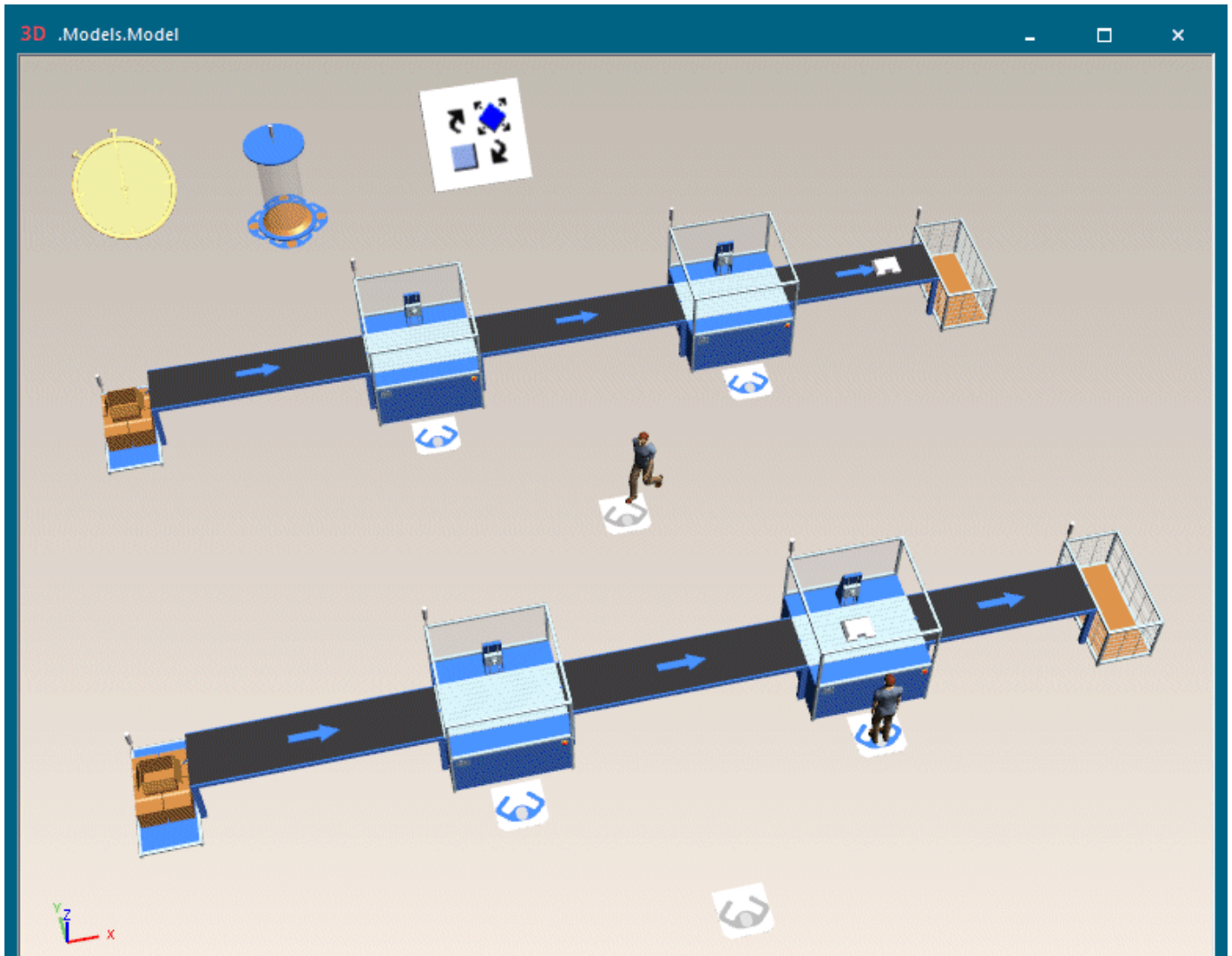
Model a Worker Who Returns to the Home Location

You can model a *Worker*, who **returns to the Home Location** after he has finished his current job within the defined **Scope**, i.e., at any of the built-in material flow objects to which the *Worker* can be brokered to do a job.

We demonstrate how to:

- [Model the Material Flow for Home Location and Scope](#)
- [Set Home Location and Scope for the Worker](#)

The finished simulation model looks like this:



See also [Worker](#) 

Back to [Model Workers and the Jobs They Do](#)

Model the Material Flow for Home Location and Scope

You can model a *Worker*, who walks from the *WorkerPool* to the *Workplace* and works at *Station1*. He then walks to his **Home Location**, namely **Home1**.

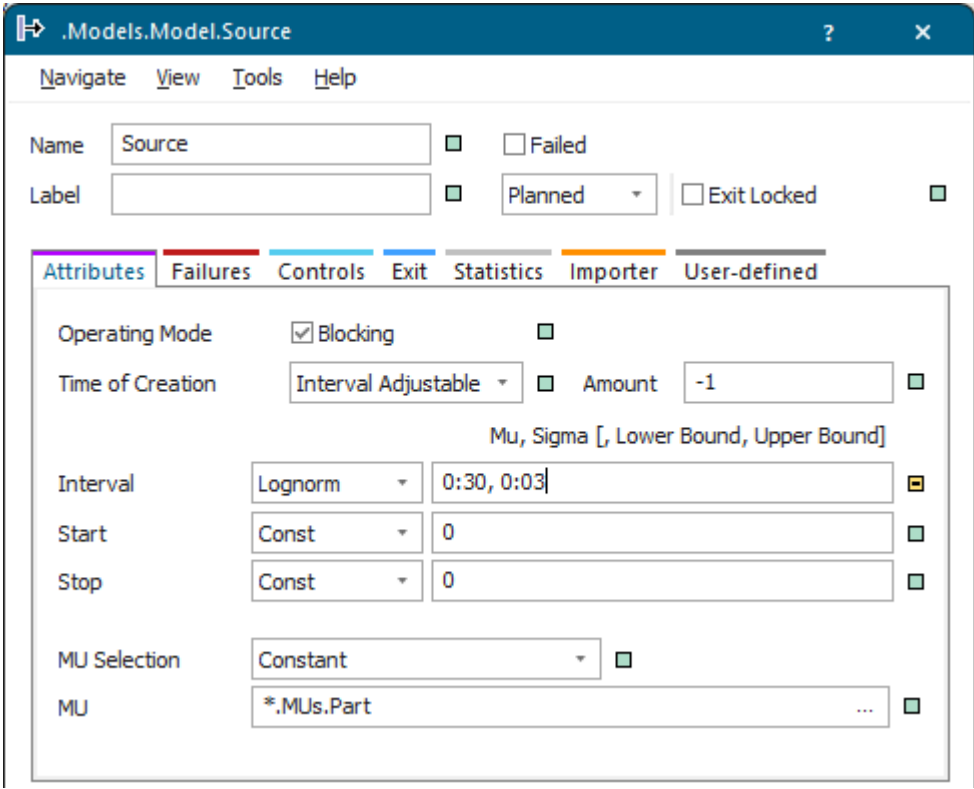
There the **Worker**  waits until the part has been transported to *Station2*. He then walks to *Workplace1* and works there.

After that he returns to the **Home Location** and waits for his next job order.

Proceed as follows to create the simulation model:

Create the processing line consisting of a *Source*, a *Conveyor*, a *Station* with an attached *Workplace*, a second *Conveyor*, a second *Station* with an attached *Workplace*, a third *Conveyor*, and a *Drain*.

- Insert the *Broker* which the *Stations* and the *Workers* require. *Plant Simulation* will then enter this *Broker* into the respective text boxes of the respective objects.
- Insert the *Source*. We used the settings below. We did not change any other settings on the remaining tabs.



.Models.Model.Source

Attributes View Tools Help

Name: Source ☐ Failed

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Failures Controls Exit Statistics Importer User-defined

Operating Mode ☒ Blocking ☐

Time of Creation: Interval Adjustable ☐ Amount: -1 ☐

Mu, Sigma [, Lower Bound, Upper Bound]

Interval: Lognorm ☐ 0:30, 0:03 ☐

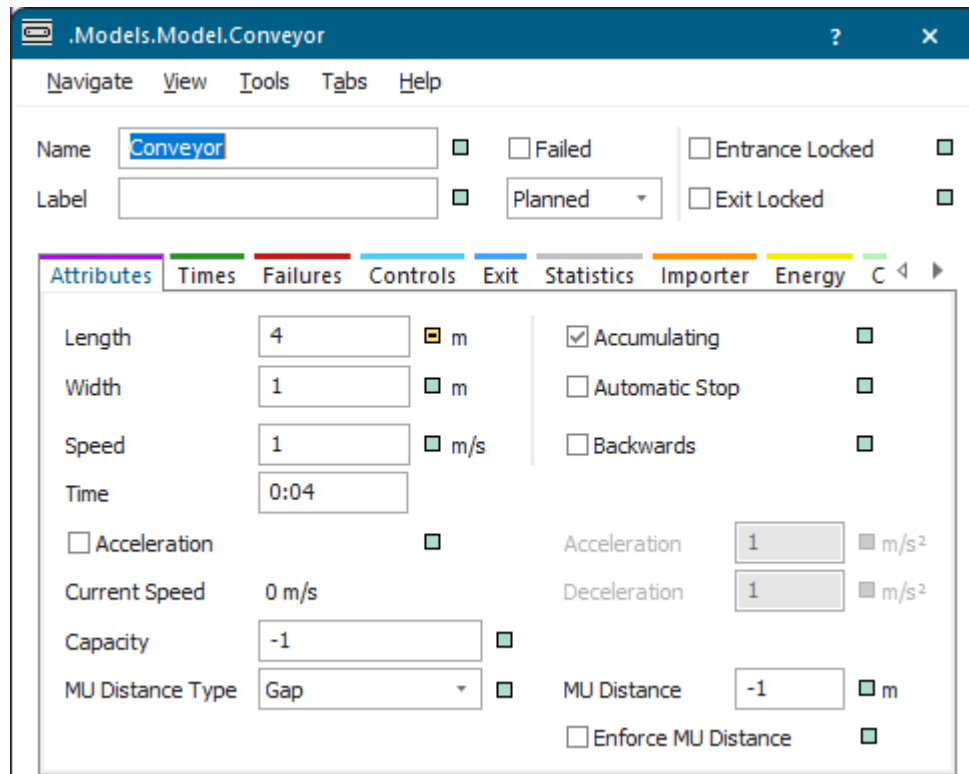
Start: Const ☐ 0 ☐

Stop: Const ☐ 0 ☐

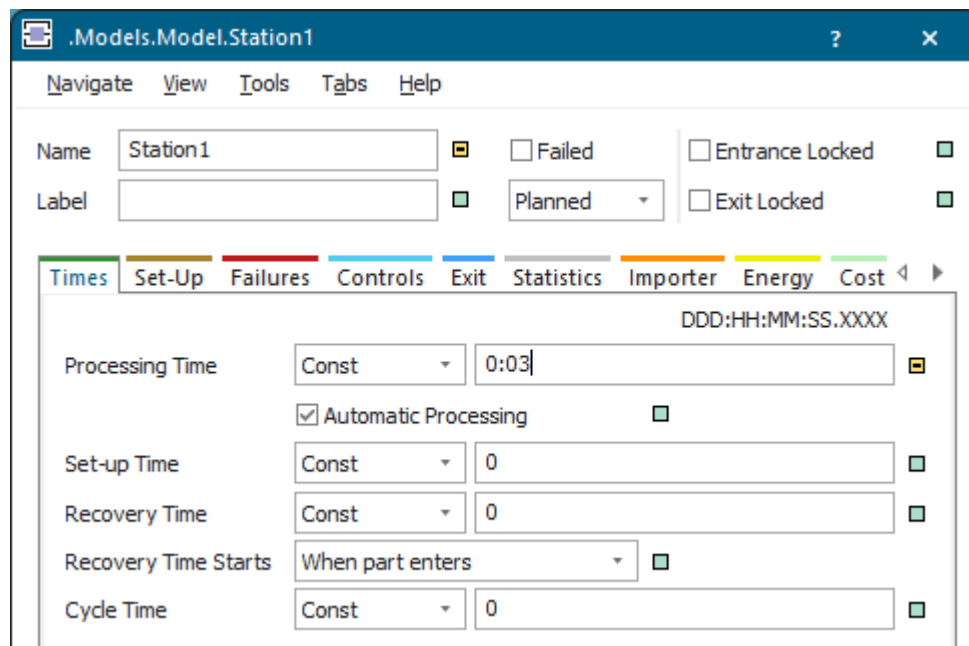
MU Selection: Constant ☐

MU: *,MUs.Part ☐

- Insert the first *Conveyor*. We used the settings below. We did not change any other settings on the remaining tabs.

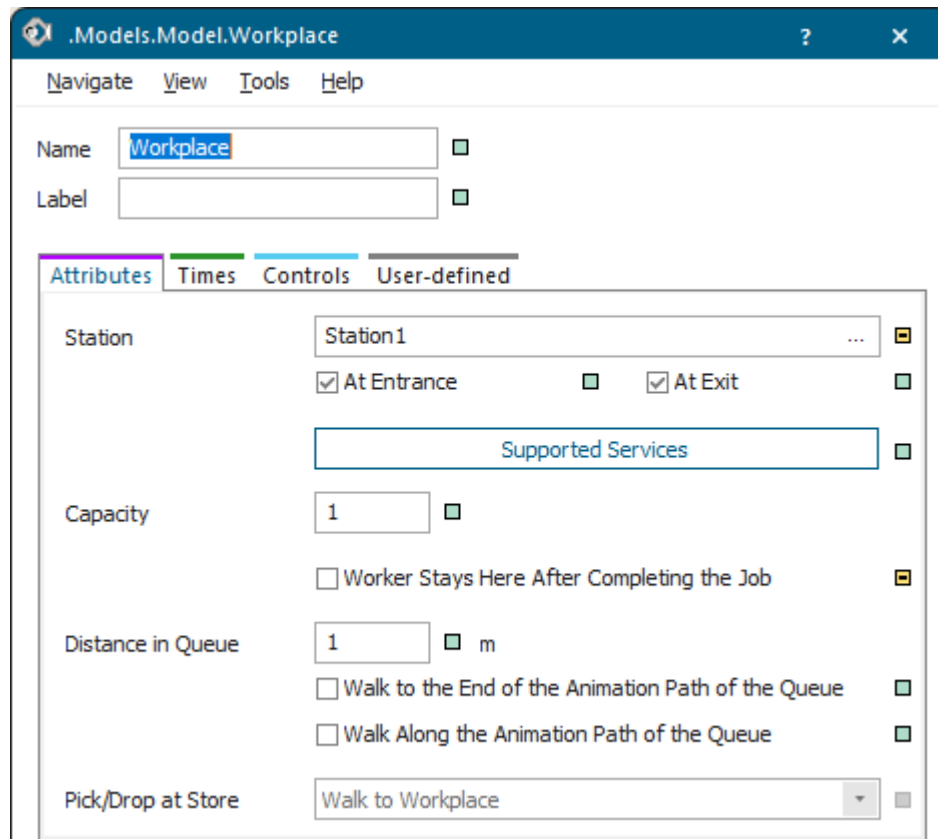


- Insert the first processing *Station*. We renamed it to *Station1*. We used the settings below. We did not change any other settings on the remaining tabs.



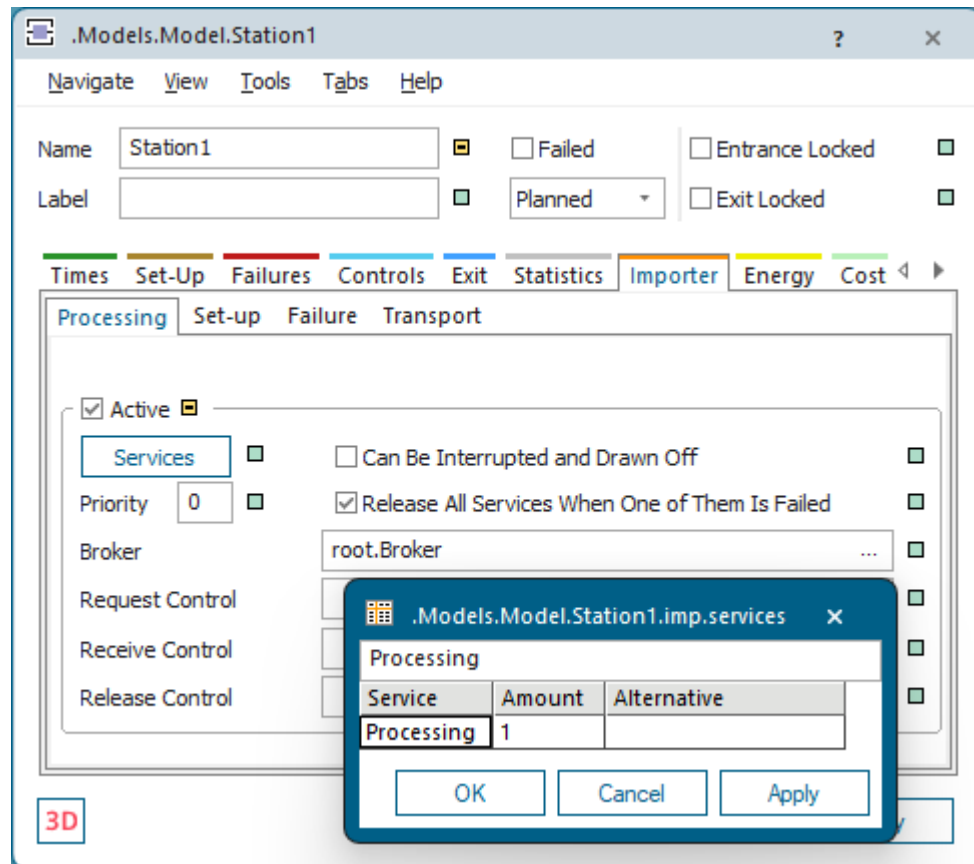
- Insert a **Workplace** below *Station1*. Drag the *Workplace* close to one of the sides of *Station1*, which you want to use as the **Station**.

Clear **Worker** stays here after completing the job.



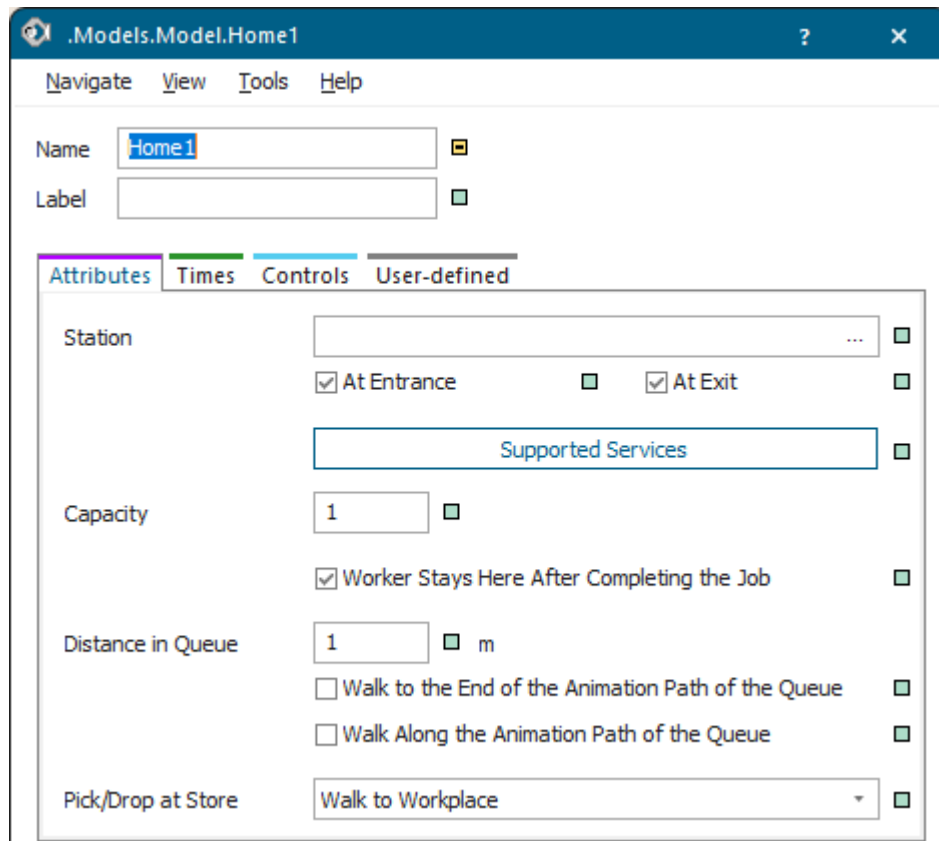
Plant Simulation then

- Opens a window asking you if you want to activate the required *importer*, a **Processing Importer** in our case. Click the button to activate it. *Plant Simulation* then activates the **Processing Importer** of *Station1* and enters **1 Processing** service into the **Services** list.



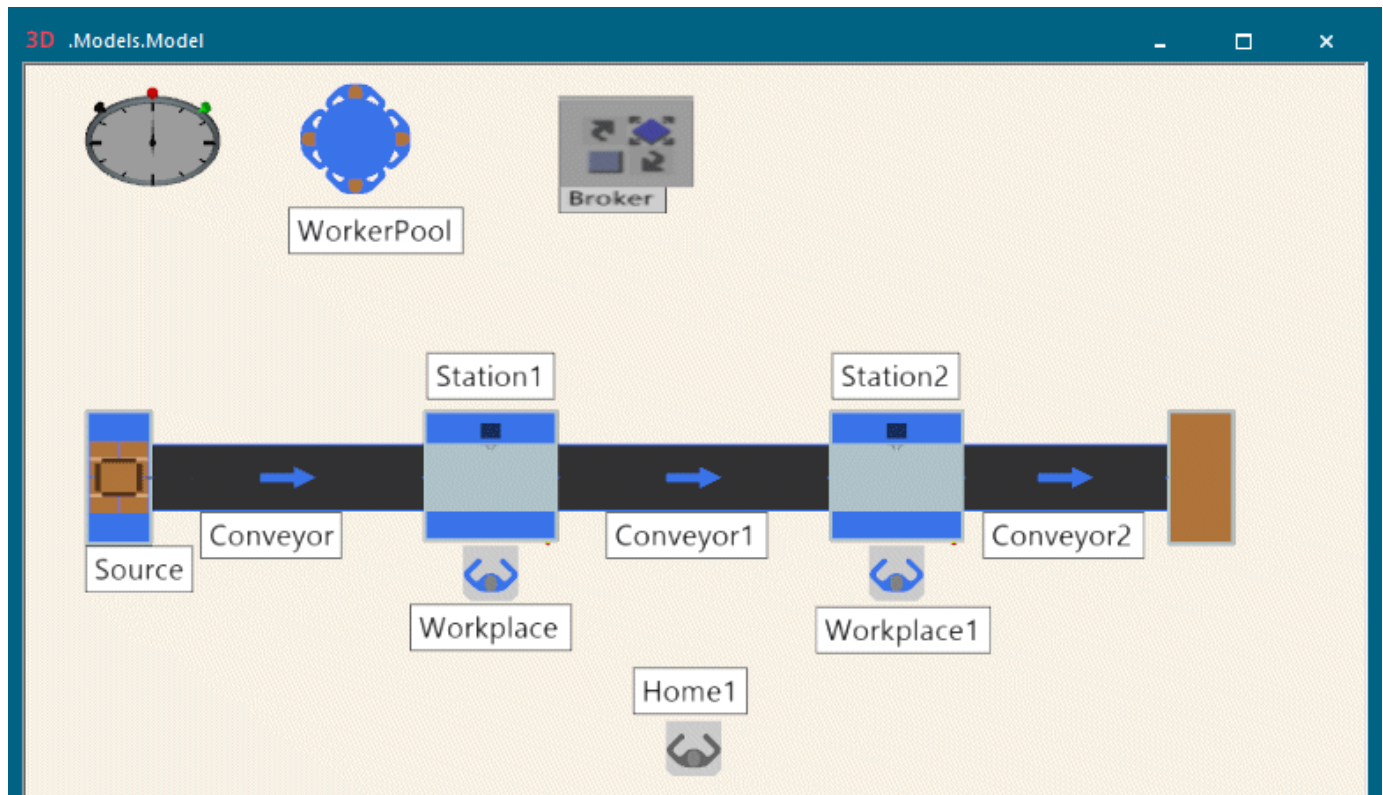
- Enters the *Station1* as the **Station** into the *Workplace*.
- Insert the *Workplace* which the *Worker* uses as his **Home Location**. This is the *Workplace* to which he always returns once he has finished his current job and waits until he is assigned a new job.

We named our *Workplace* Home1 and used it with its default settings.



- Insert the remaining material flow objects. We used the same settings as described above. We only made the last *Conveyor* 3 meters long as compared to the other two *Conveyors*, which are 4 meters long.
- Insert the **WorkerPool** which creates the required *Workers*. We are going to configure it in the next step.

Our model now looks like this:



Back to [Model a Worker Who Returns to the Home Location](#)

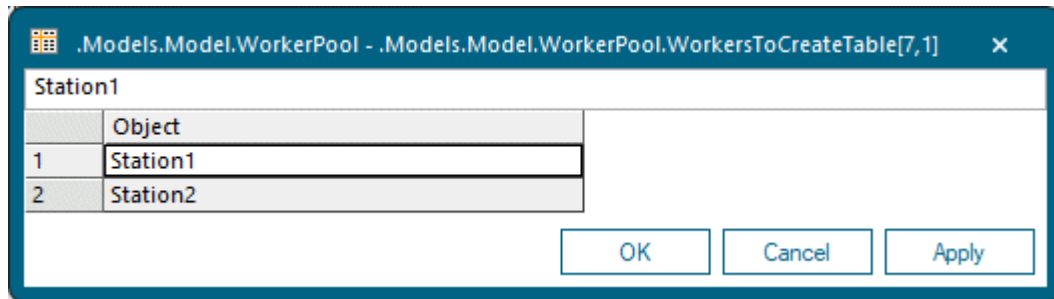
Set Home Location and Scope for the Worker

After we modeled the material flow for our *Worker* using a **Home Location** and a **Scope**, we are going to configure the *WorkerPool* which creates the *Worker*.

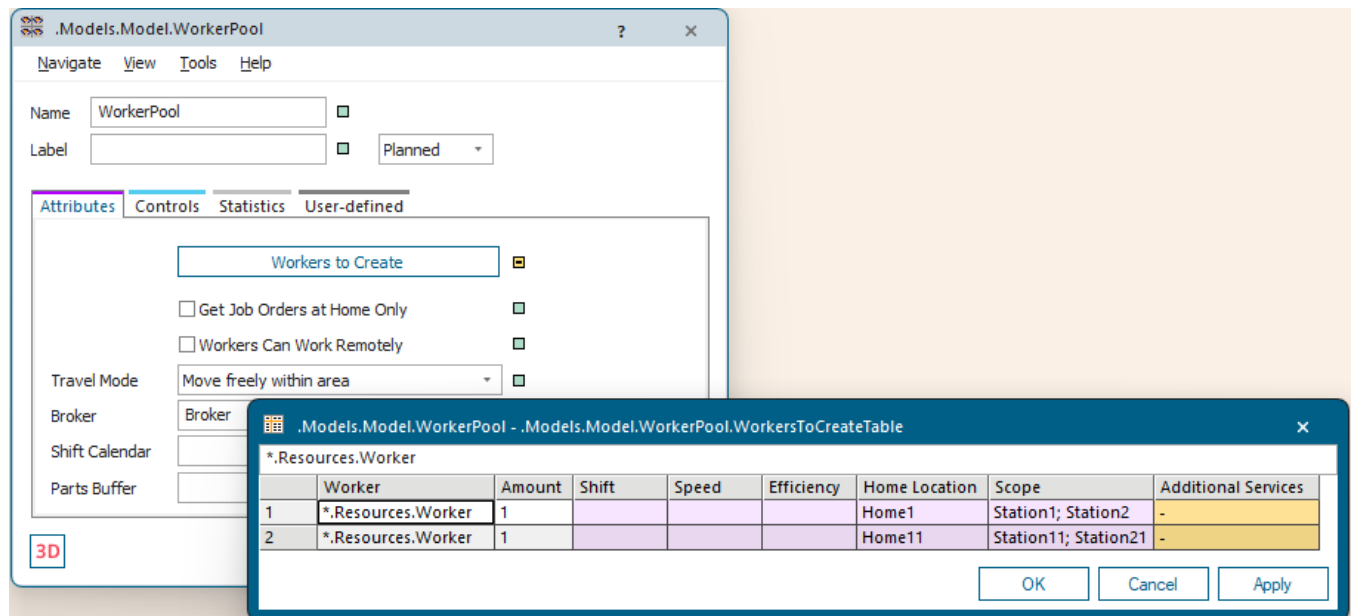
Proceed as follows:

- Open the **WorkerPool** we inserted in the previous step. Click the button **Workers to Create** and configure the creation table.
 - Drag the *Worker* from the **Tab Resources** to the cell named **Worker**.
 - Drag the *Workplace* named **Home1** to the cell named **Home Location**.
 - Double-click in the cell **Scope**. Drag the stations to which the *Worker* can be brokered to do a job into the cells of the table.

We dragged *Station1* to cell 1. Then we pressed **Enter** to add another cell and dragged *Station2* on cell 2.

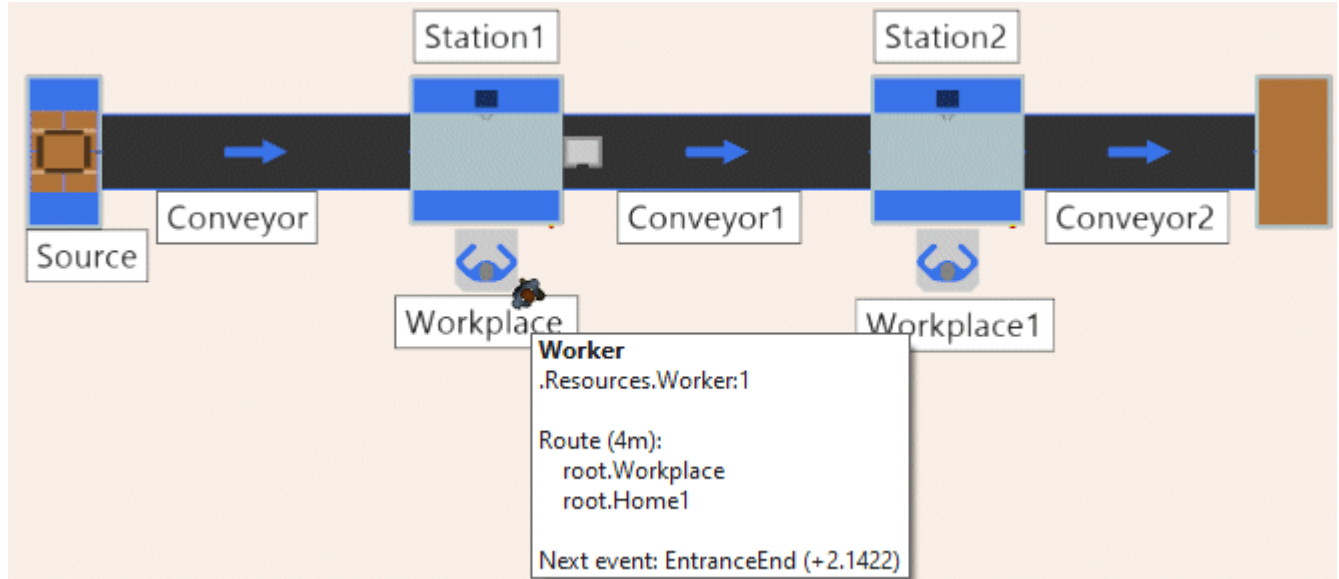


When we click **OK**, *Plant Simulation* adds the **Scope** stations to the cell **Scope** and separates them with a semicolon.

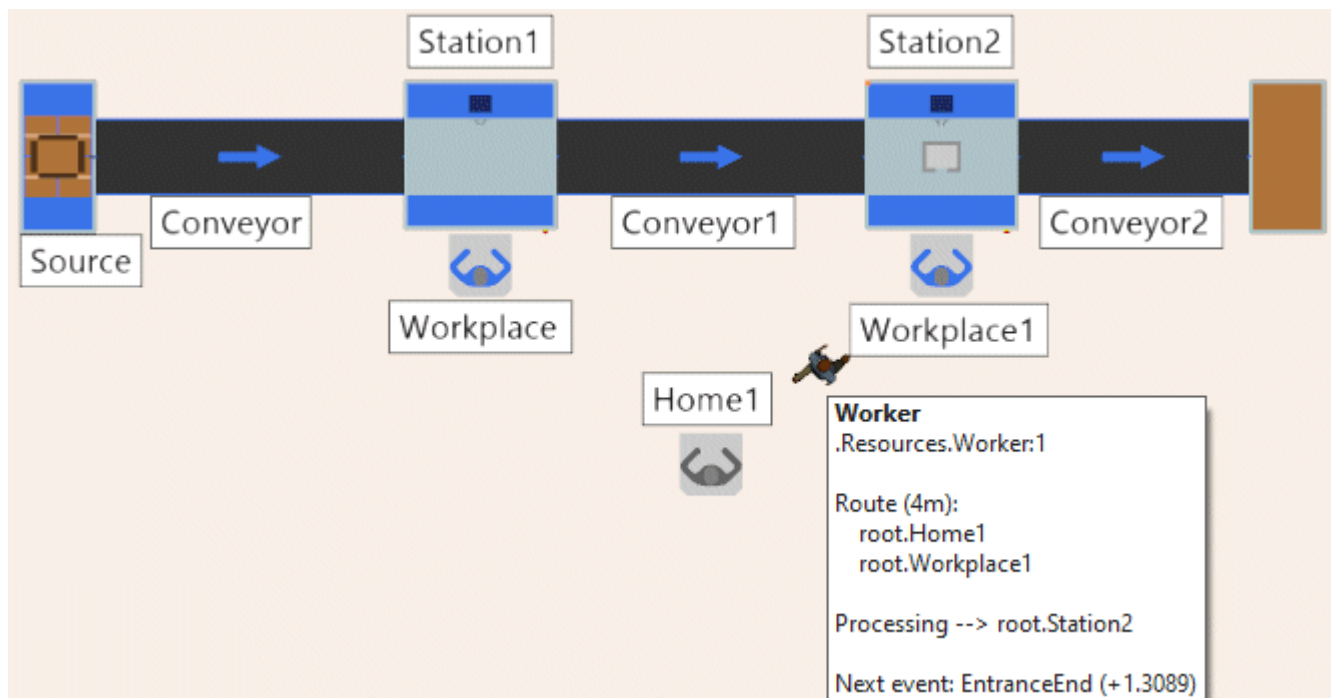


Click **OK** to close the **WorkersToCreateTable** and click **OK** in the dialog of the *WorkerPool*.

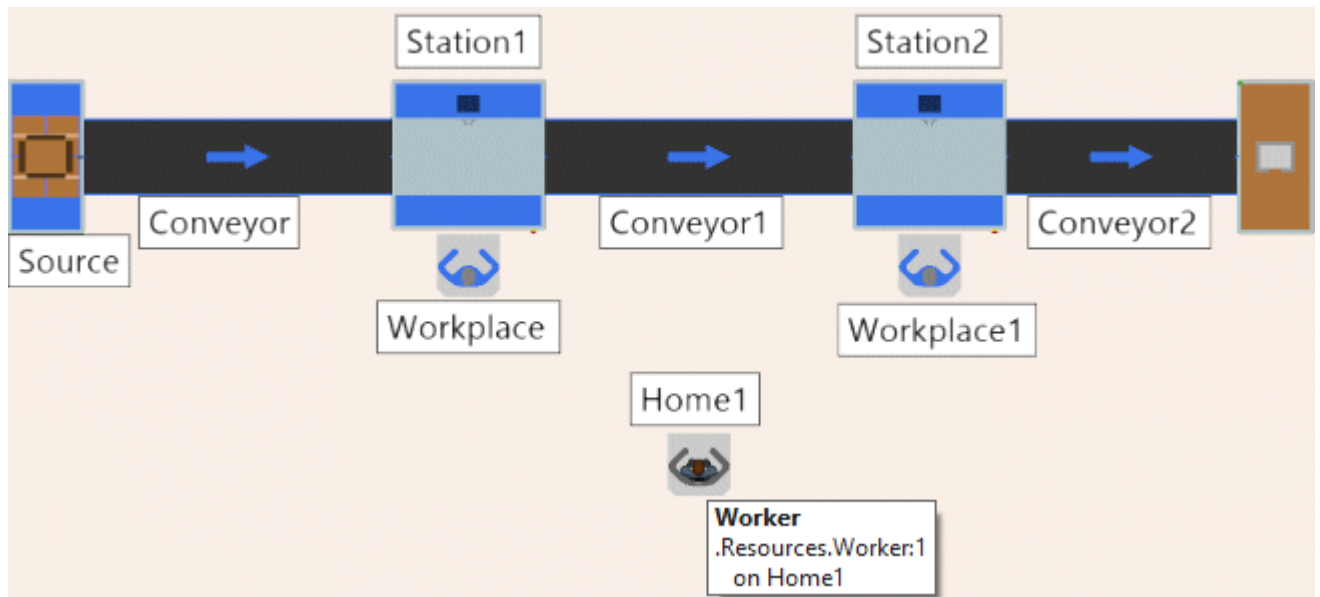
- Start the simulation run and watch the *Worker* walk from the *WorkerPool* to the *Workplace* and do his job at *Station1*. He then walks to his **Home location**, namely **Home1**.



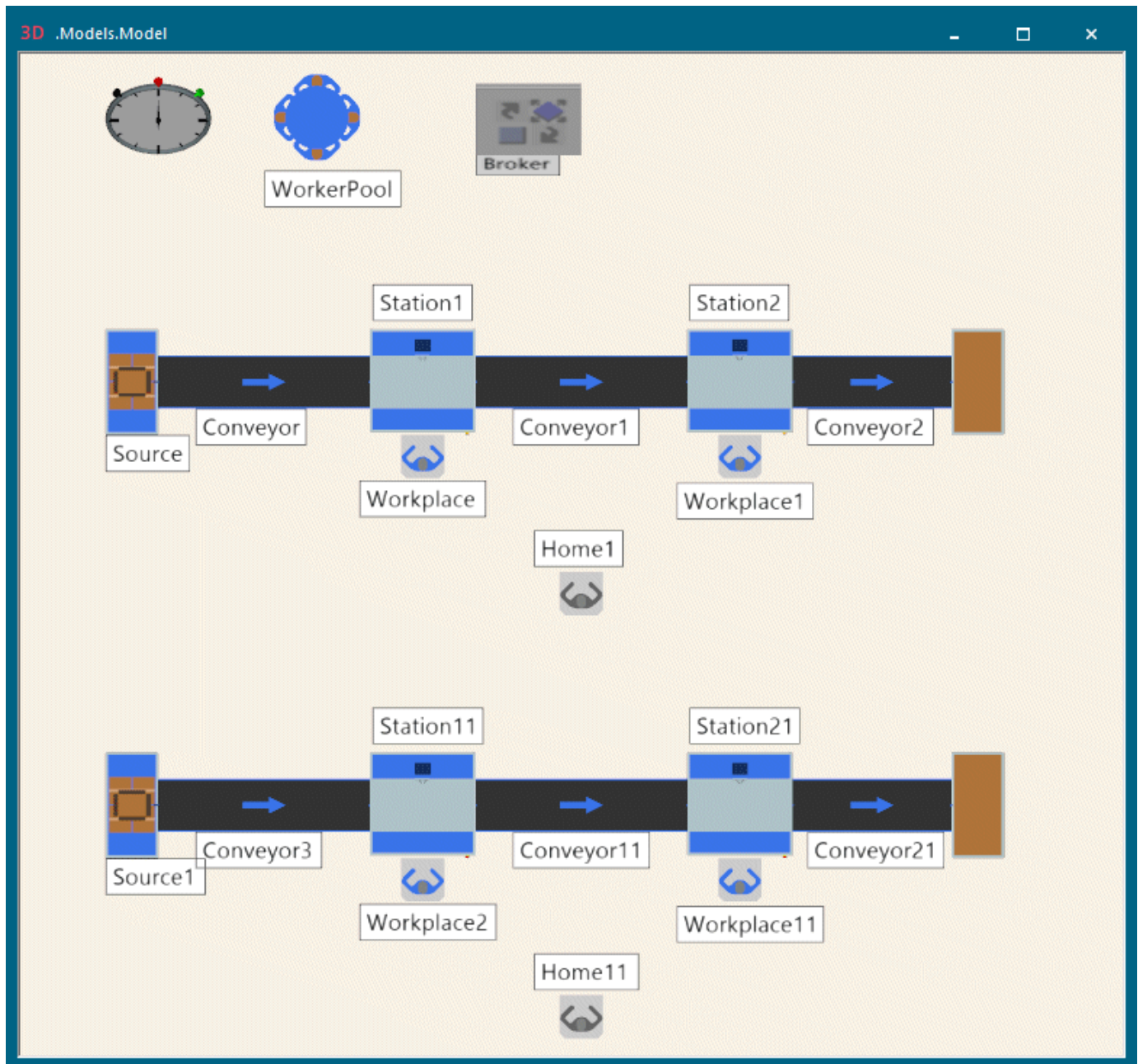
There he waits until the part has been transported to *Station2*. He then walks to *Workplace1* and does his job there.



After that he returns to the **Home location** and waits for his next job order.



We then copy our first processing line and paste it below that.



- We add the **Home location** and the **Scope** of the second processing line to the **Workers to Create** table.

.Models.Model.WorkerPool - .Models.Model.WorkerPool.WorkersToCreateTable

	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	*.Resources.Worker	1				Home1	Station1; Station2	-
2	*.Resources.Worker	1				Home11	Station11; Station21	

OK Cancel Apply

- When we run the simulation, we can watch how our second *Worker* uses *Home11* as his **Home location** and *Station11* and *Station21* as his **Scope**.

Back to [Model a Worker Who Returns to the Home Location](#)

Video on YouTube

https://youtu.be/gU1pqu7sWA8?si=dUIUQrzFfJ_XIAv7&t=38

<https://youtu.be/gU1pqu7sWA8?si=pn-PWhO--ldQE0Ai&t=294>

Model a Worker Picking Up/Placing Parts At a Store

Model a Worker Picking Parts Up at/Placing Parts in a Store

You can model a *Worker*, who **walks freely within the area** of the simulation model and who **picks up a part at a Source** and **places it in the Store**.

We demonstrate how to:

- [Model a Worker Who Walks to a Workplace Attached to a Store](#)
- [Model a Worker Who Walks to a Store Column](#)
- [Model a Worker Who Walks Along the Store to a Store Column](#)

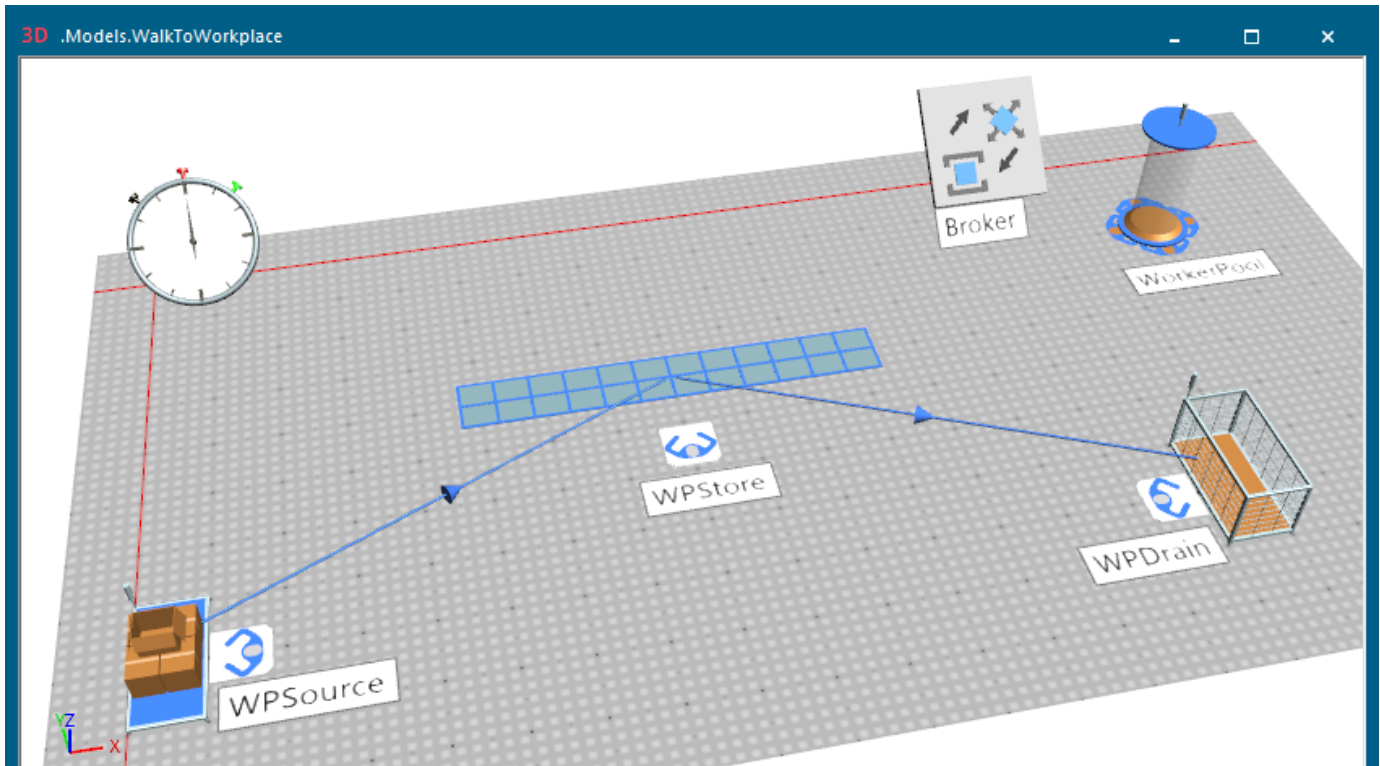
See also [Worker](#) 

Back to [Model Workers and the Jobs They Do](#)

Model a Worker Who Walks to a Workplace Attached to a Store

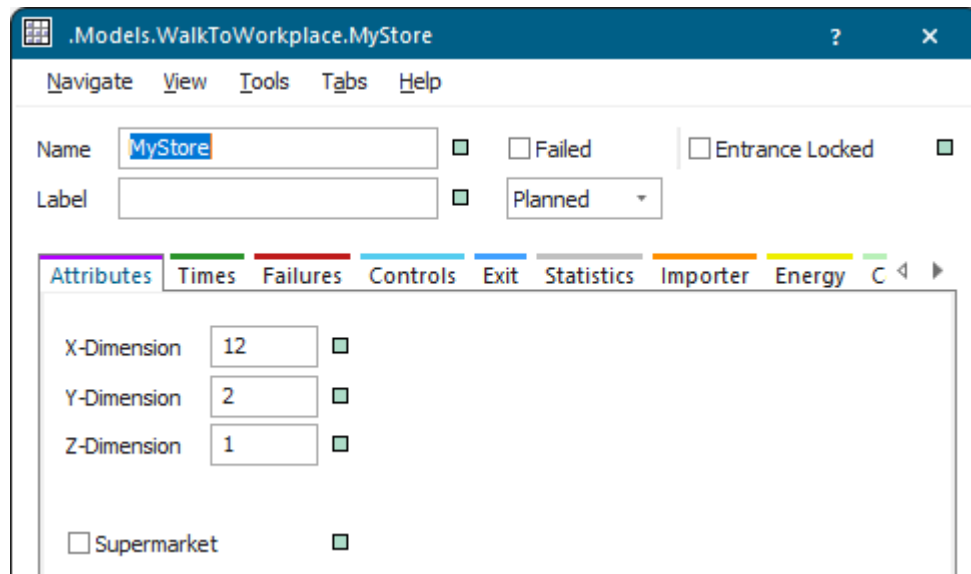
In this example we model a *Worker*, who **walks freely within the area** of the simulation model, who picks up a part at a *Source*, walks to the *Workplace* attached to the *Store*, and places the part in the *Store*.

Insert the objects into the simulation model so that they match the screenshot below:

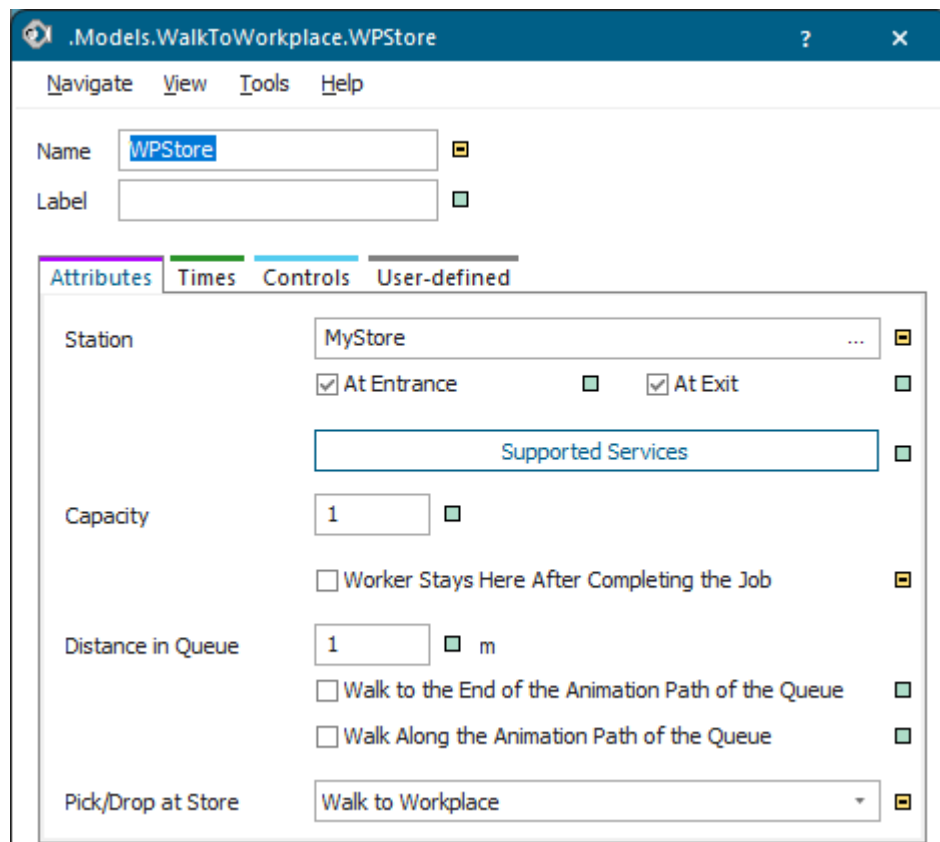


The *Source* produces the parts, which the *Worker* picks up at the attached *Workplace* and carries them to the *Workplace* attached to the *Store*.

- We use the *Source* with its default settings.
- For the *WPSource* we selected the **Transport Importer** when we inserted it. We did not change any other settings.
- For the *Store* we selected these settings:

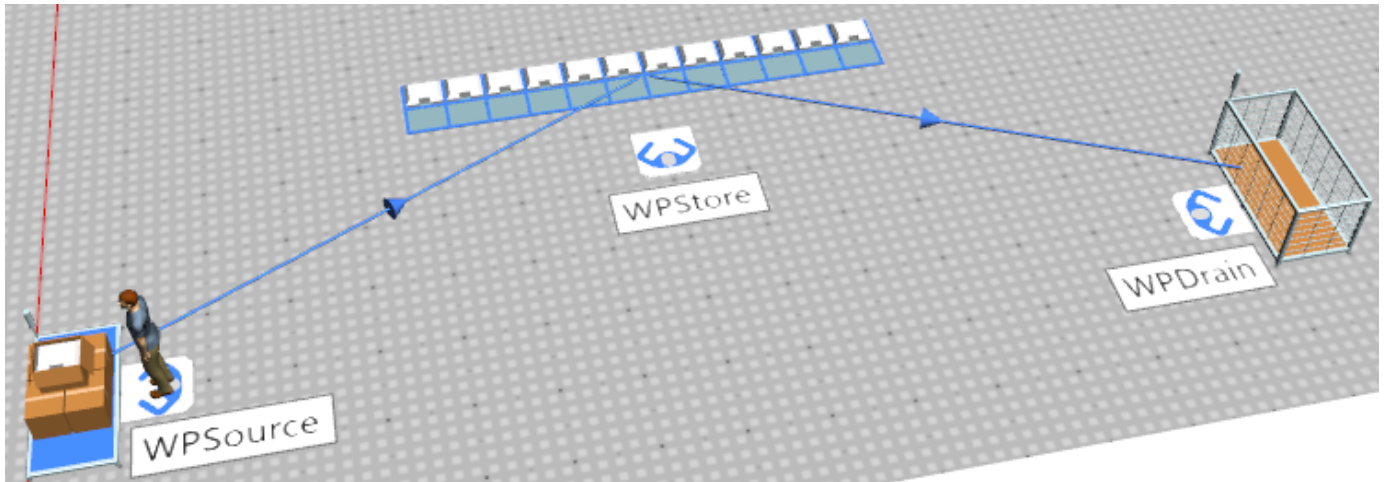


- For the *WPStore* we selected the **Transport Importer** when we inserted it. We use it with its default settings, namely **Pick/Drop at Store** > **Walk to Workplace**.



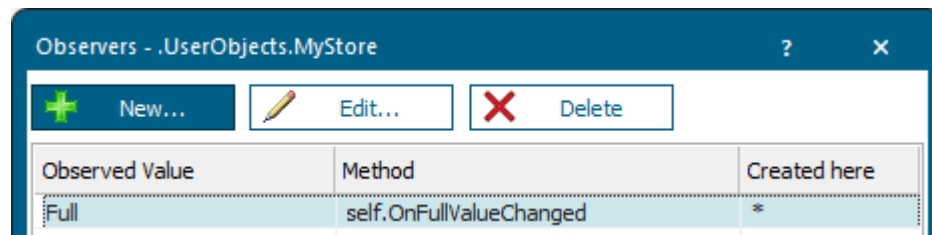
- We use the *Drain* and its attached *WPDrain* with their default settings.
- Connect the *Source*, the *Store*, and the *Drain*.

Run the simulation. You'll see that the *Worker* picks up a part from the *WPSource*, walks to the *WPStore*, steps onto it, and places the part into the leftmost storage bay in the top row. He then walks back to the *WPSource* and repeats the procedure until all storage bays in both rows are filled with a part each.



When he is finished, he returns to the *WorkerPool*.

As we want the *Worker* to pick the parts from the storage bays and carry them to the *WPDrain*, we created an **observer** for the read-only attribute *Full* of the class of our *Store*. As soon as the *Store* is full, we will lock the exit of the *Source*, make the *Worker* stay on the *WPStore*, and carry the parts to the *WPDrain*.



The observer calls the method named *OnFullValueChanged* is a *user-defined attribute* of the *Store*.

We typed in this source code:

```
-- param attribute: string, oldValue, newValue: boolean
-- param attribute: string, oldValue: boolean
param newValue: boolean

Source.ExitLocked := true

if newValue
  var p:object := ?.cont
  while p /= VOID
    p.move
    waituntil p.location /= ?
    p := ?.cont
```

```

    end
  end
  waituntil ?.Empty
  Source.ExitLocked := false

```

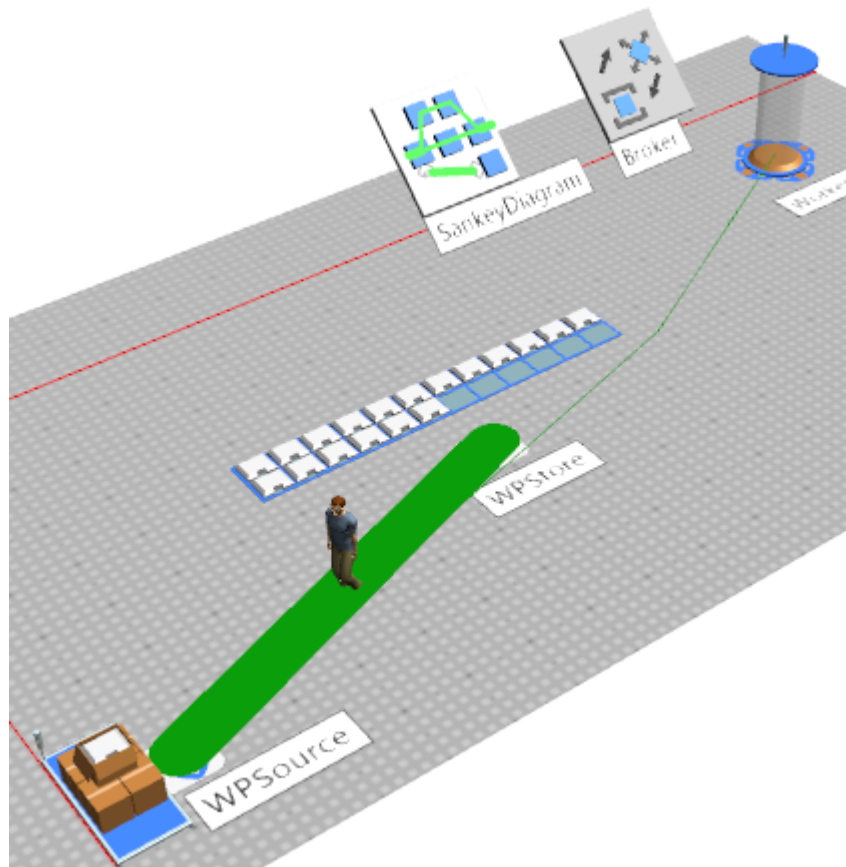
When we run the simulation, we see that the *Worker* picks the part he placed first into the leftmost storage bay in the top row of the *Store*, carries it to the *WPDrain*, and puts it onto the *Drain*. He continues doing this until all storage bays are empty and then he starts filling the *Store* again with parts, etc.



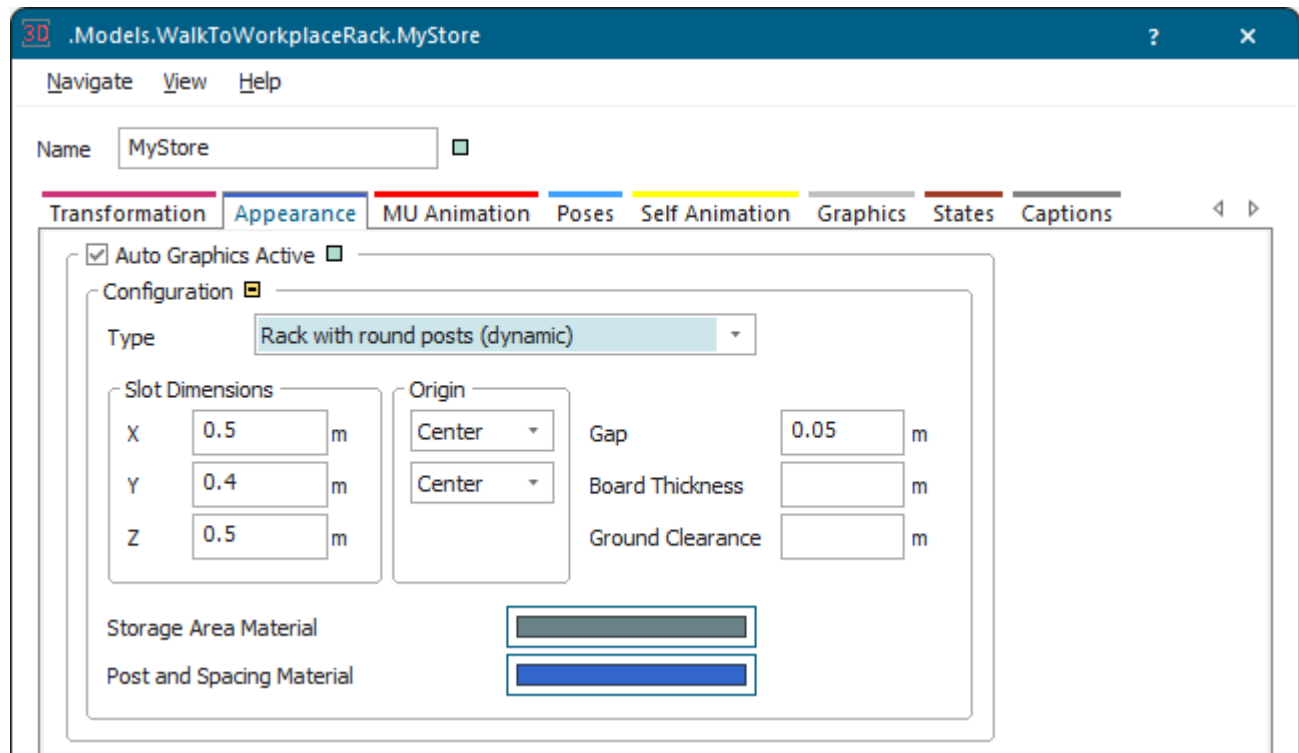
To show the route/Sankey flows of the *Worker*, we inserted a *SankeyDiagram* and specified the *Worker* on the tab **Objects**.

Attributes	Objects	User-defined
	Objects	
1	*.Resources.Worker	
2		

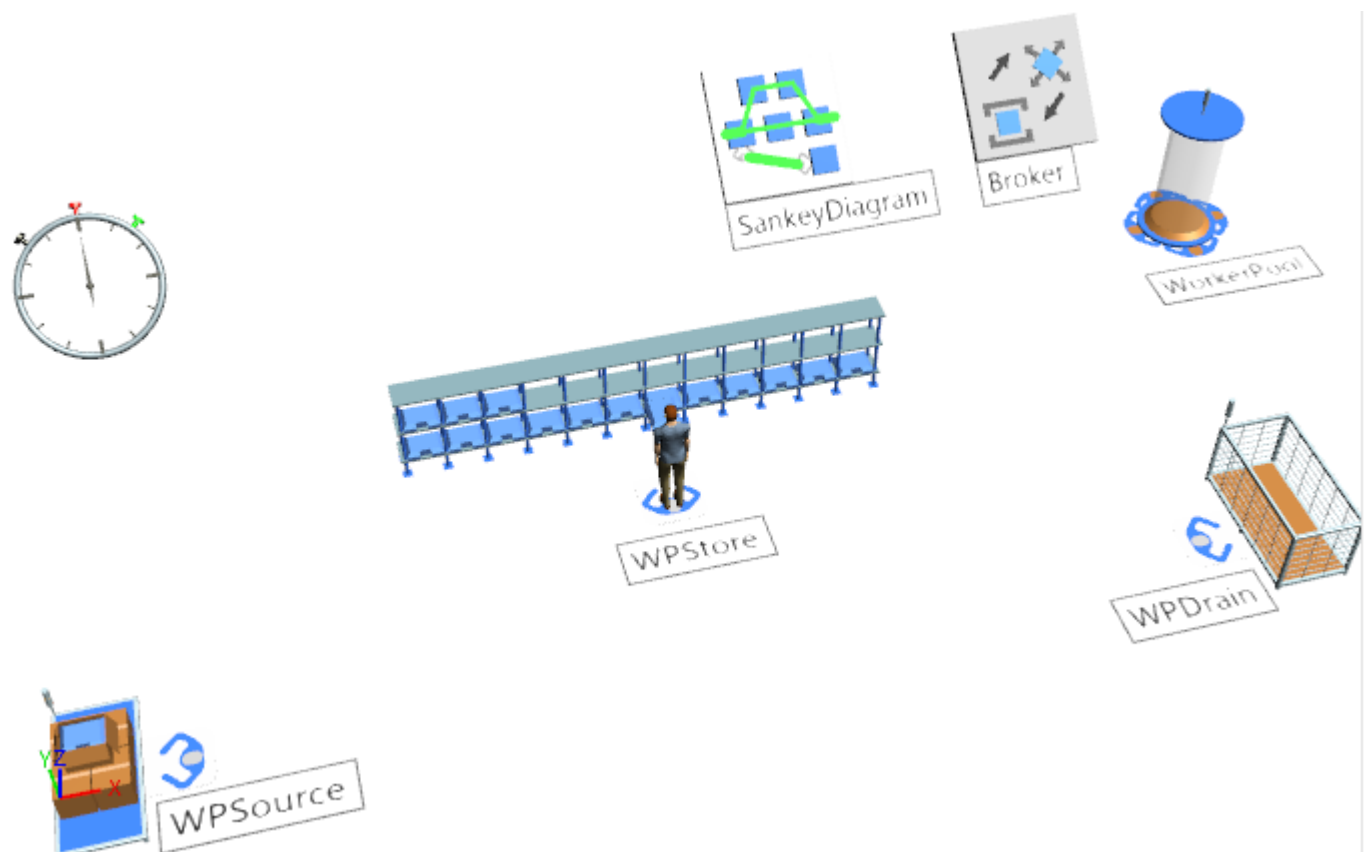
When the *Worker* walks from the *WPSource* to the *WPStore*, we see that he steps onto the *WPStore* and drops the parts, and *Plant Simulation* then automatically places them into the correct storage bay.



To make the model look more realistic, we change the **Appearance of the Store** in 3D. Instead of **Floorspace**, we selected **Rack with round posts (dynamic)** and changed the **Slot Dimension Z** to 0.5 meters.



The model then looks like this.



See also [Worker](#) 

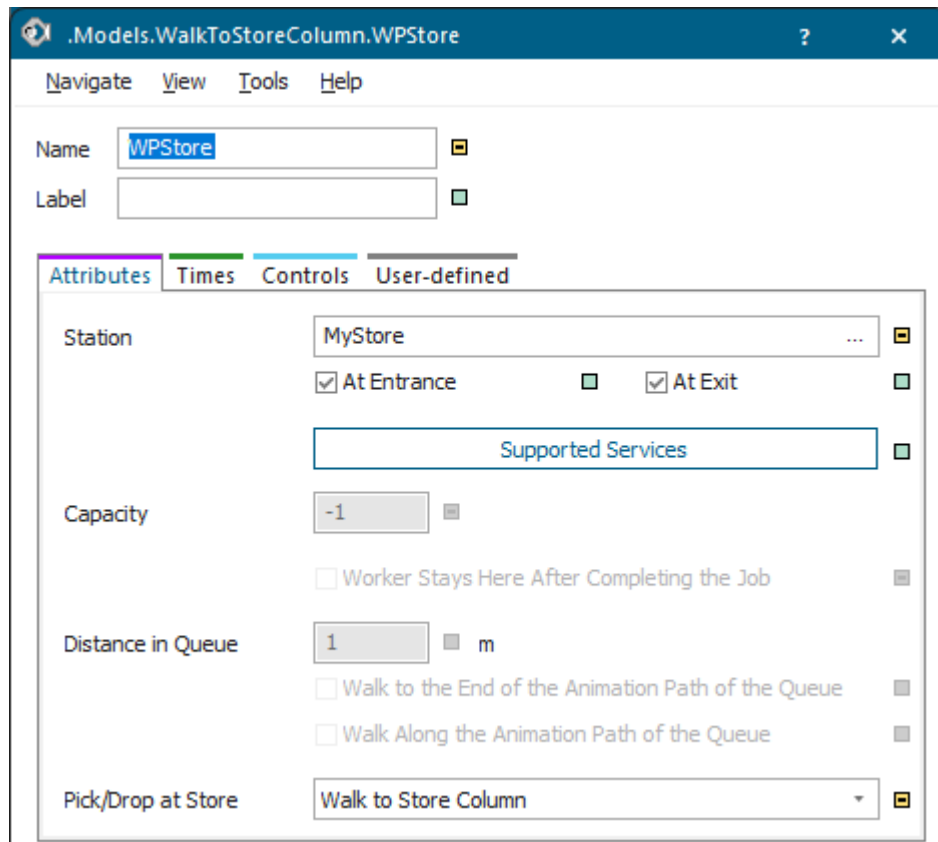
Back to [Model a Worker Picking Parts Up at/Placing Parts in a Store](#)

Model a Worker Who Walks to a Store Column

You can model a *Worker*, who **walks freely within the area** of the simulation model, picks up a part at the *Source*, walks to a **column** of the *Store*, and places the part in the *Store*.

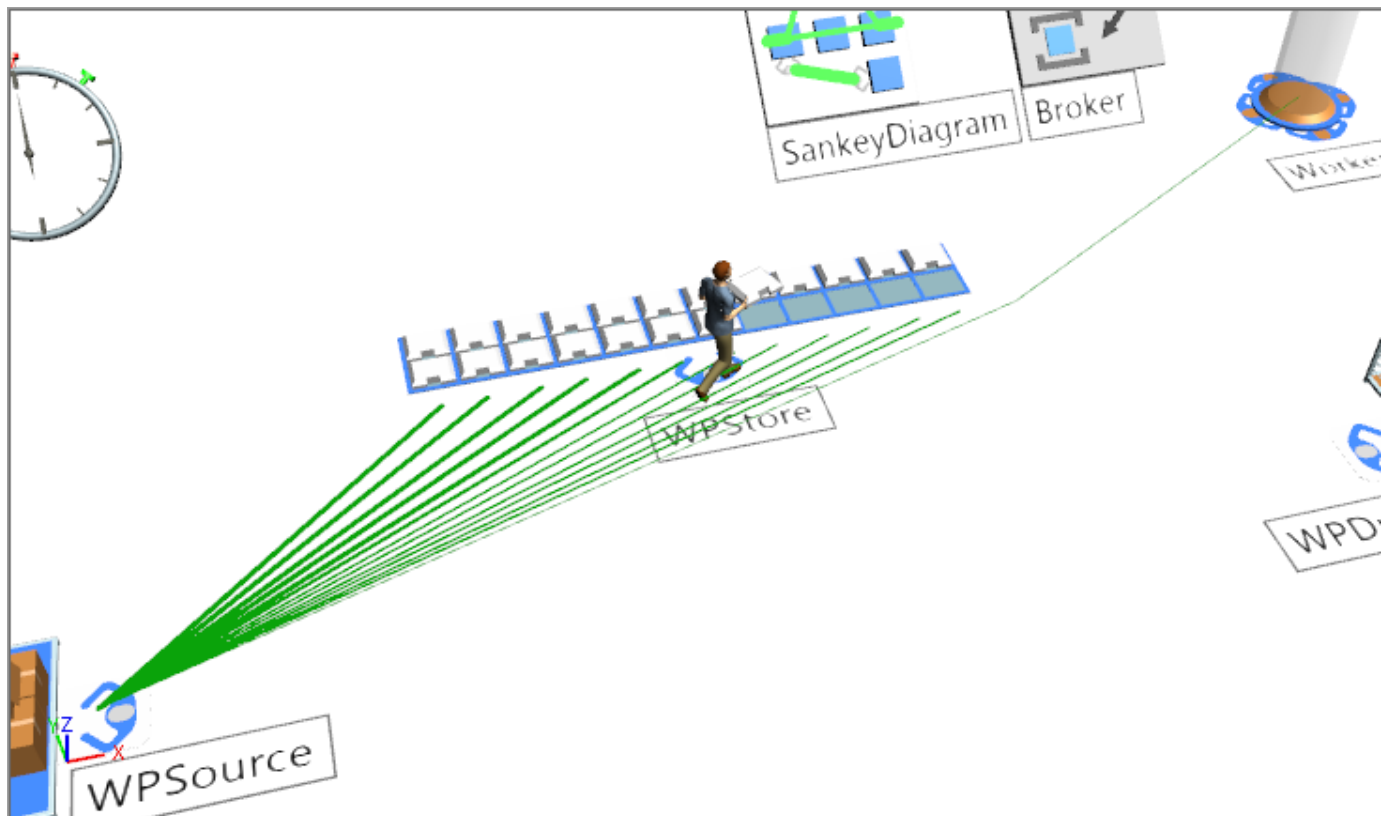
To demonstrate this, we duplicated our previous model.

We selected **Pick/Drop at Store > Walk to Store Column** for our *WPStore*. *Plant Simulation* then deactivates a number of settings that do not apply.

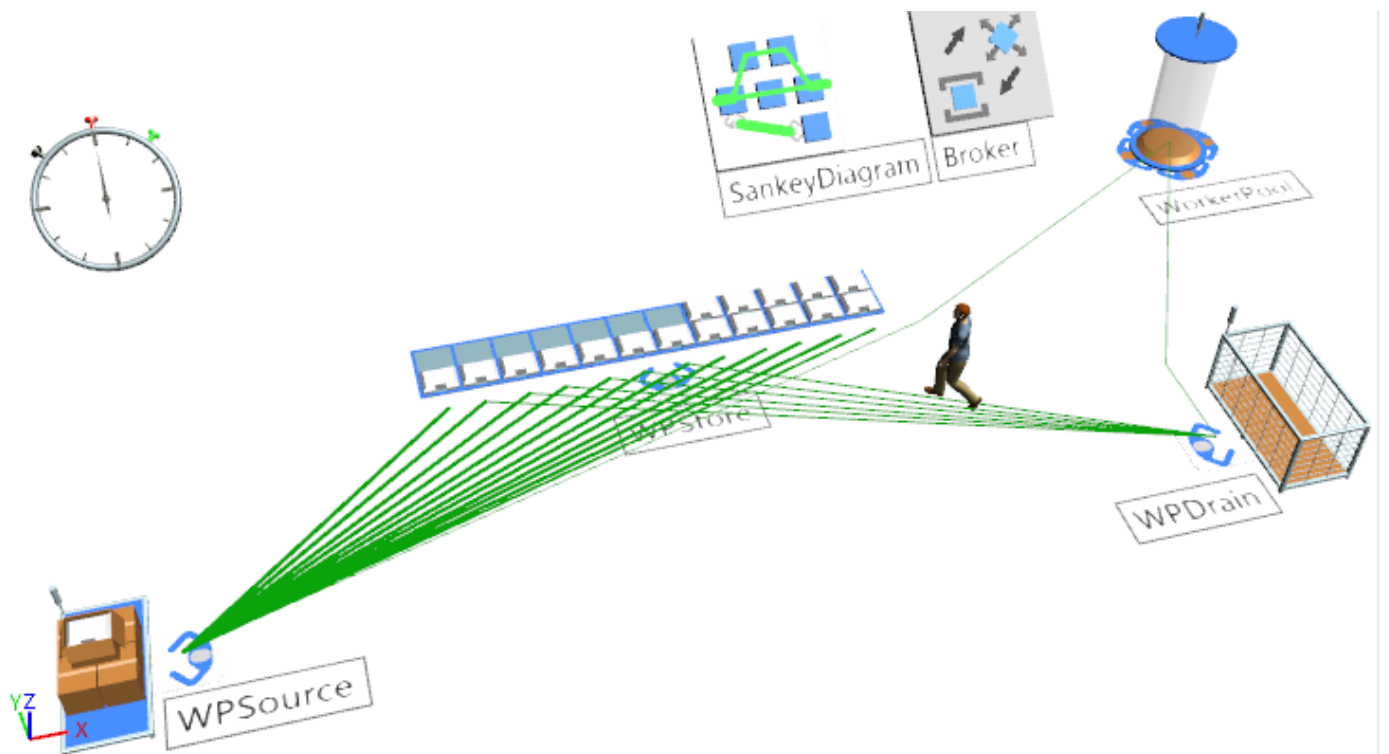


Run the simulation. You'll see that the **Worker**  picks up a part from the *WPSource*, walks to the storage bays in the respective store column instead of stepping onto the *WPStore*, and places the part into the leftmost storage bay in the top row. He then walks back to the *WPSource* and repeats the procedure until all storage bays in both storage rows are filled with a part each.

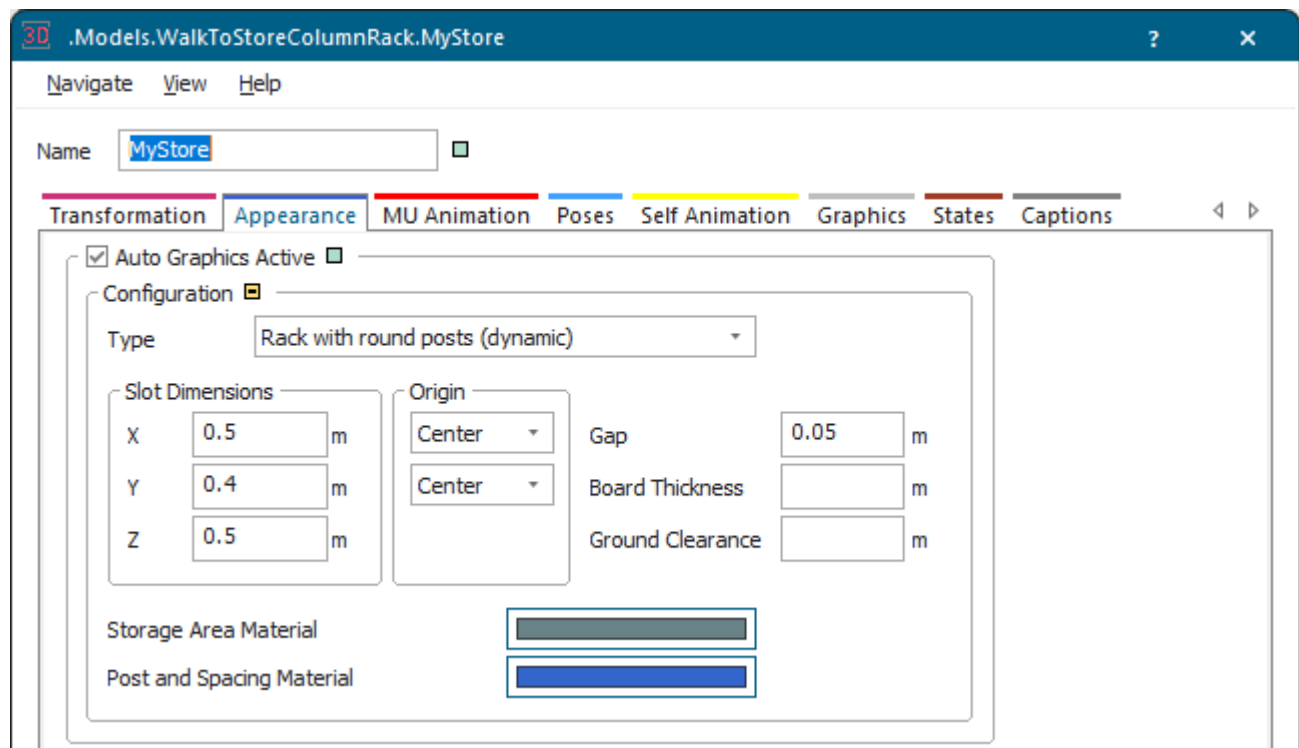
The Sankey flows get thicker as the *Worker* fills up the bottom row of storage bays.



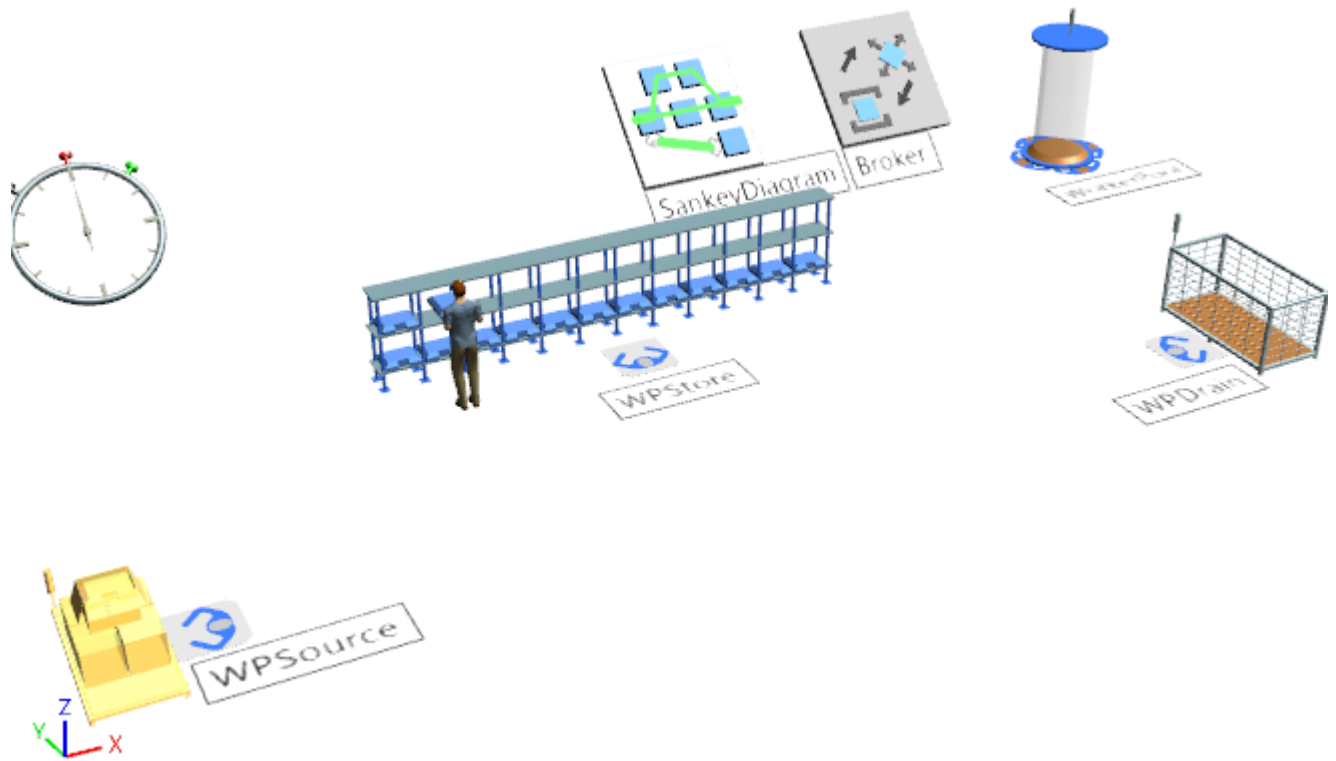
Once all storage bays are stocked, the *Worker* walks back to the *WorkerPool*. From there he walks to the *WPDrain* and then on to the *Store*, where he picks up the parts in the order in which he placed them.



To make the model look more realistic, we change the **Appearance of the Store** in 3D. Instead of **Floorspace**, we selected **Rack with round posts (dynamic)** and changed the **Slot Dimension Z** to 0.5 meters.



The model then looks like this.



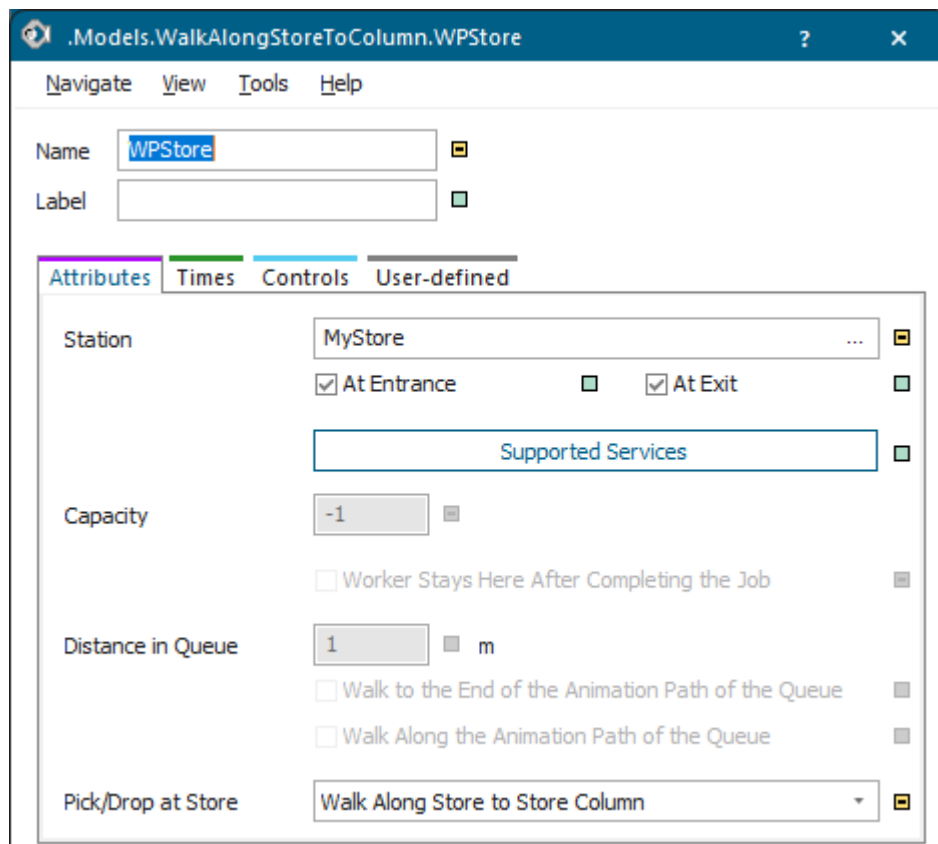
Back to [Model a Worker Picking Parts Up at/Placing Parts in a Store](#)

Model a Worker Who Walks Along the Store to a Store Column

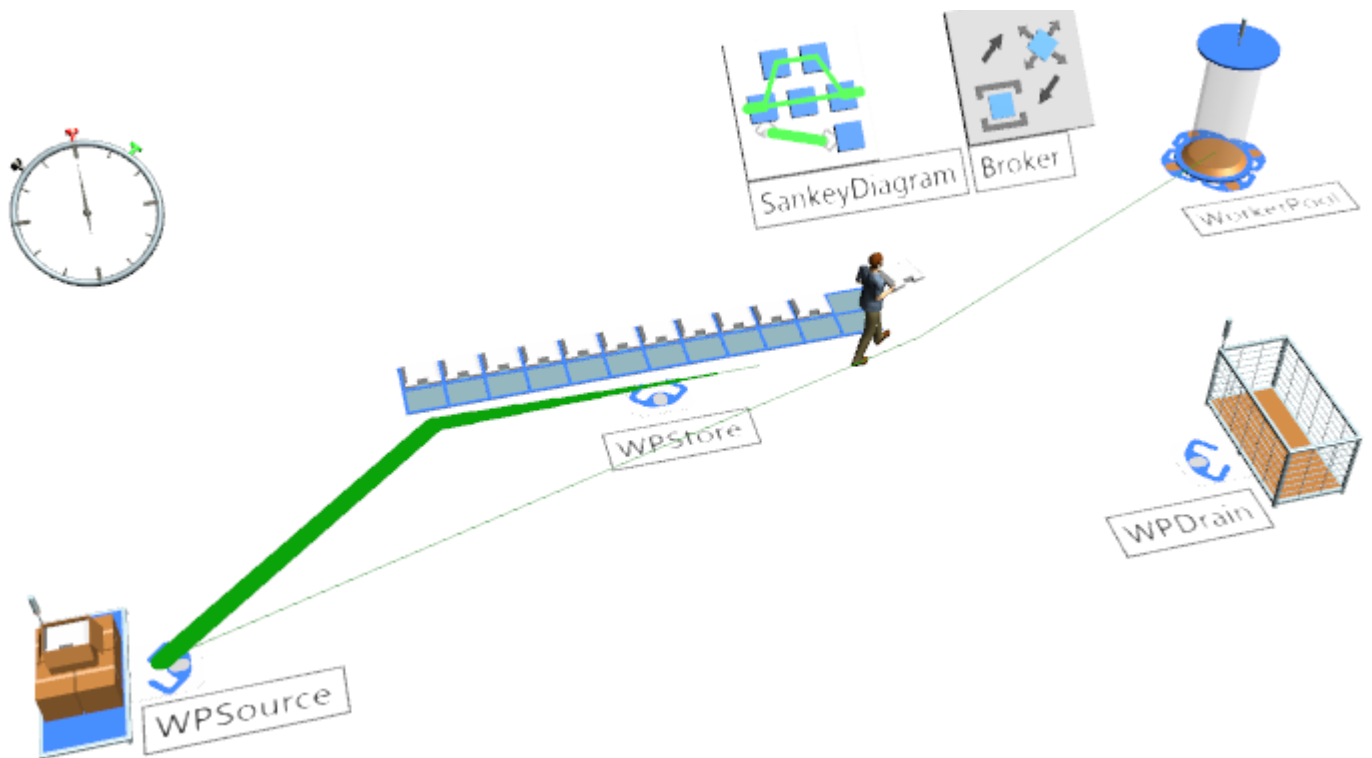
You can model a *Worker*, who **walks freely within the area** of the simulation model, picks up a part at a *Source*, **walks along the Store to a column** of the *Store*, and places the part in the *Store*.

To demonstrate this, we duplicated our previous model.

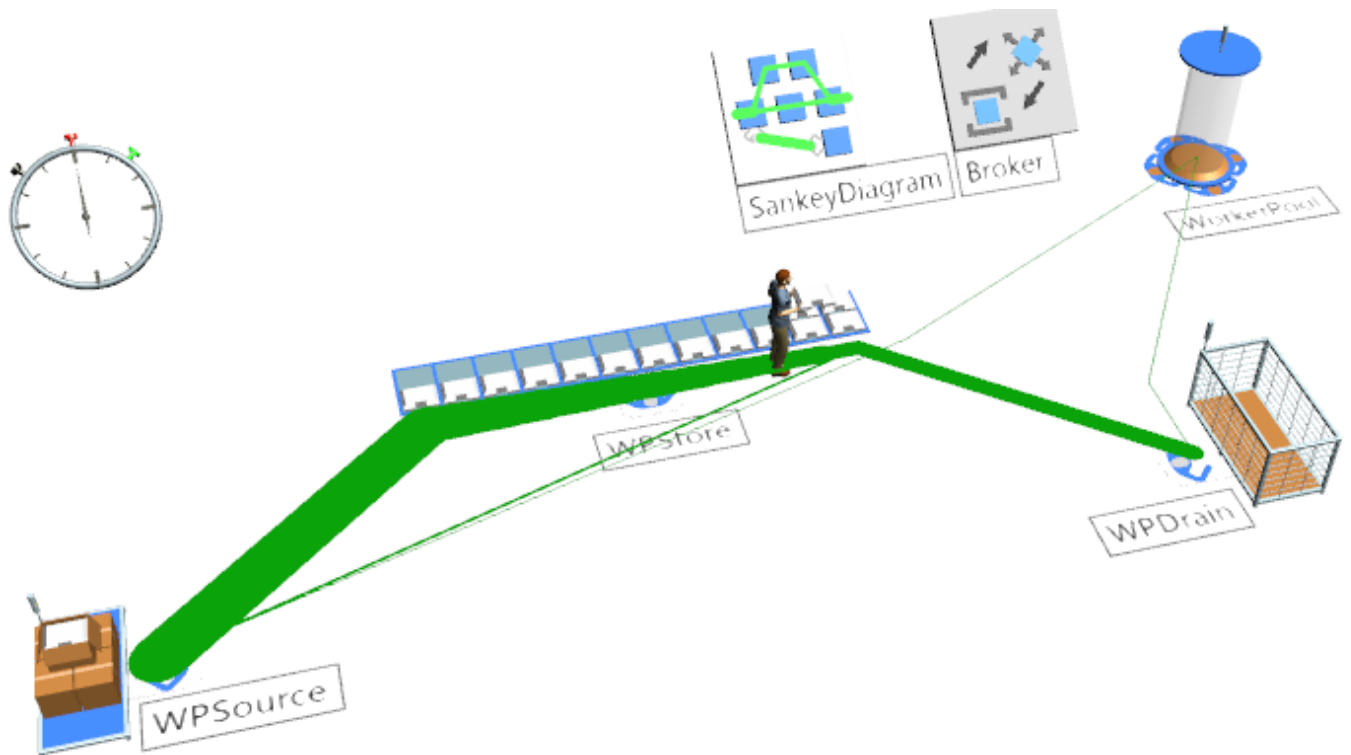
We selected **Pick/Drop at Store > Walk Along Store to Store Column** for our *WPStore*. *Plant Simulation* then deactivates a number of settings that do not apply.



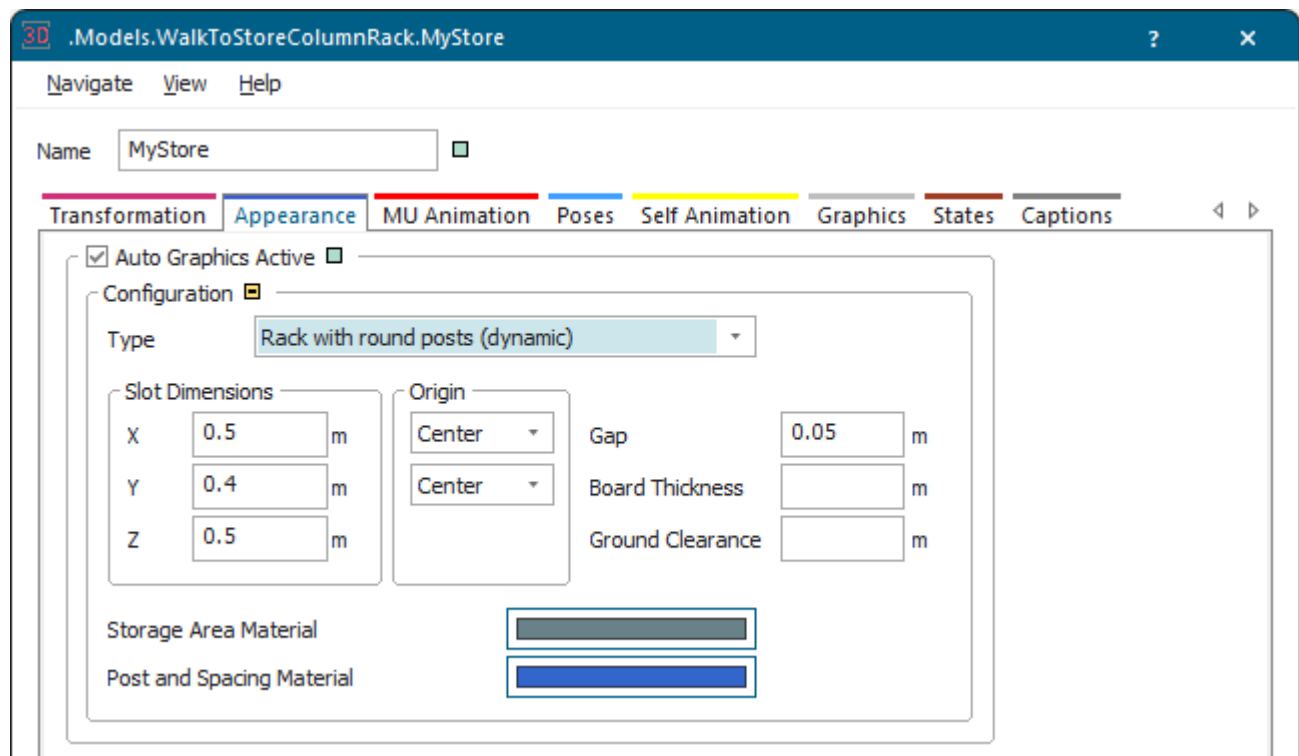
Run the simulation. You'll see that the **Worker**  picks up a part from the *WPSource*, walks to the left corner of the *Store*, turns right, and places the part in the leftmost storage bay, instead of stepping onto the *WPStore*. He then walks back to the *WPSource*, picks up the next part, walks back to the *Store* and along it to the next empty storage bay toward the right until all storage bays in both rows filled with a part each.



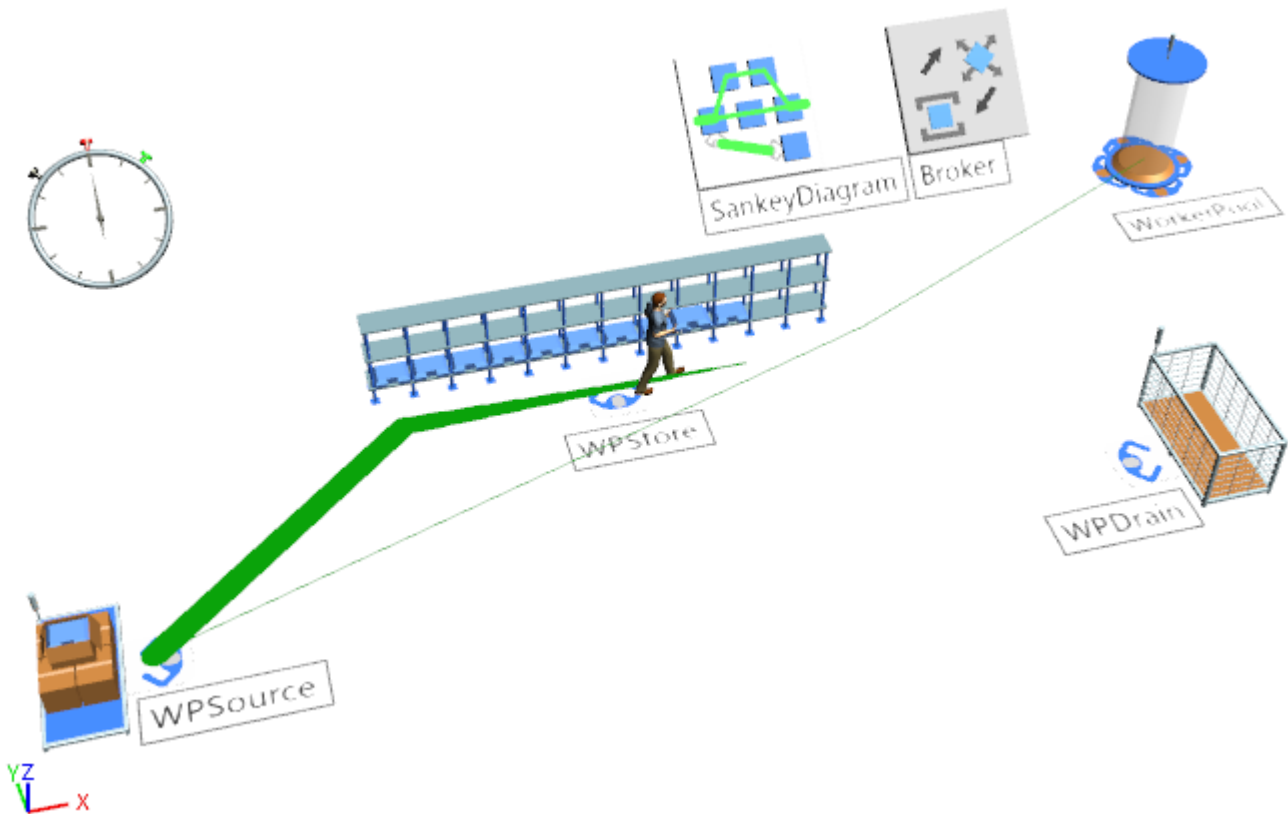
When the *Worker* carries the parts from the *Store* to the *WPDrain*, he picks up the parts in the order in which he placed them into it, walks along the *Store* to its end, and then turns right toward the *Drain*.



To make the model look more realistic, we change the **Appearance of the Store** in 3D. Instead of **Floorspace**, we selected **Rack with round posts (dynamic)** and changed the **Slot Dimension Z** to 0.5 meters.



The model then looks like this.



Back to [Model a Worker Picking Parts Up at/Placing Parts in a Store](#)

Show Worker Statistics in a Chart

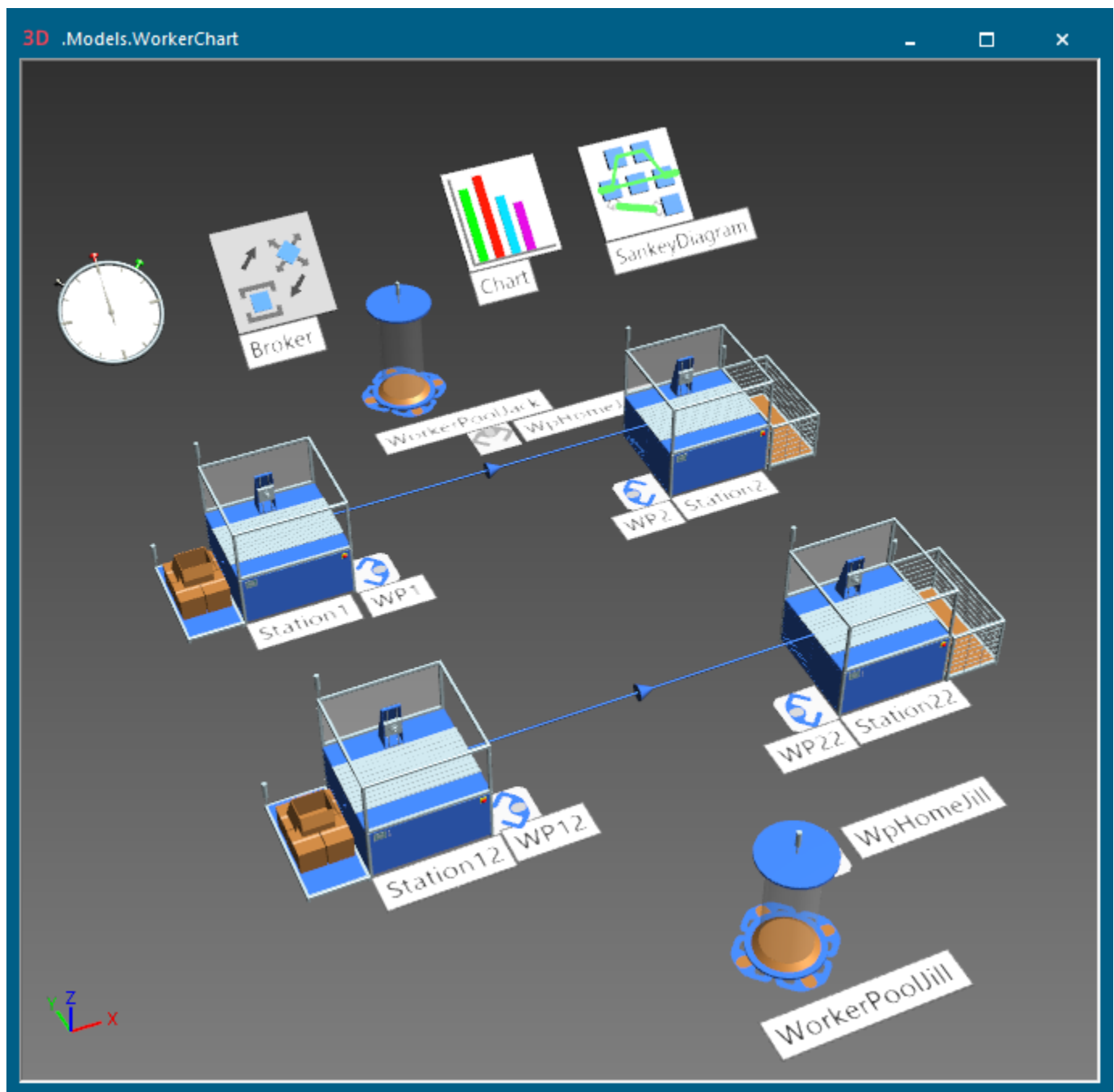
Show Worker Statistics in a Chart

In this example we show *Worker statistics* in a *Chart*.

We demonstrate how to:

- [Configure the Stations and the Workplaces](#)
- [Configure the WorkerPools for the Chart](#)
- [Configure the Chart and View Worker Statistics](#)

Our finished model looks like this:



See also

[Worker \[object\]](#)

[WorkerPool](#)

[Chart \[object\]](#)

[Back to Model Workers and the Jobs They Do](#)

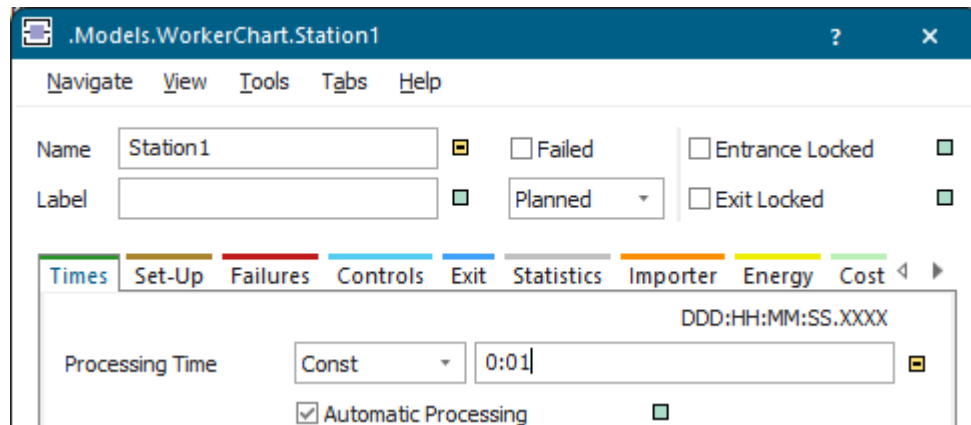
Configure the Stations and the Workplaces

To demonstrate how to show *Worker Statistics* in a *Chart*, we first configure the *Stations* that process the parts and the *Workplaces* at which the *Workers* pick up the parts and place them.

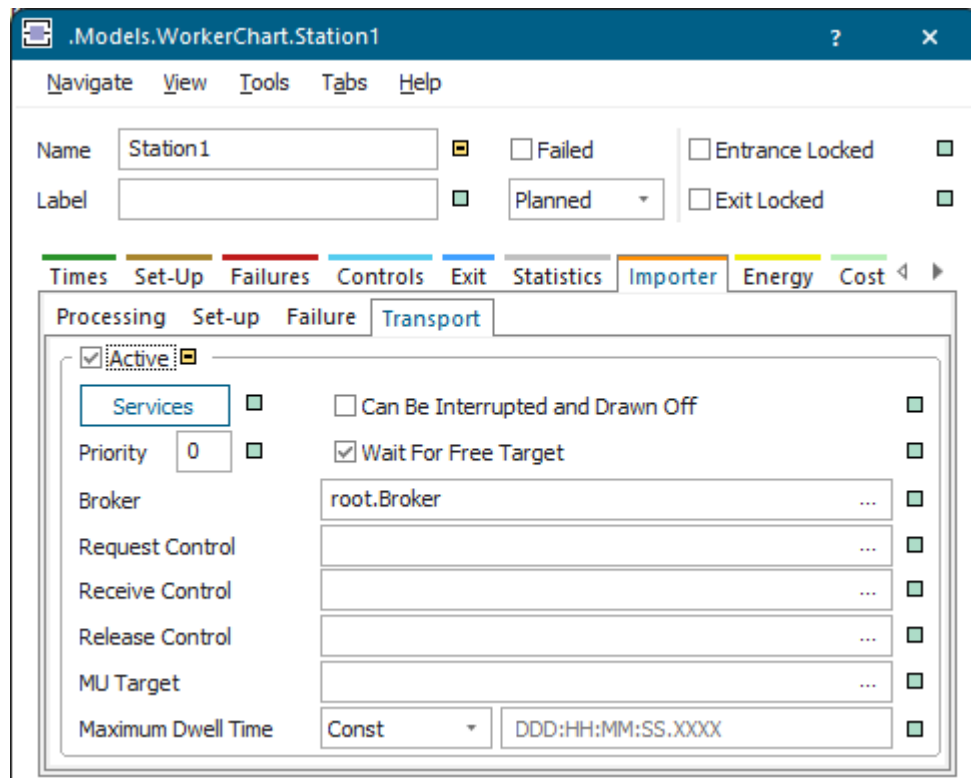
Settings for Station1 and Workplace (WP1)

For *Station1*, which processes the parts, which the *Source* provided, we selected these settings:

- We typed in a constant **Processing Time** of 1 second.



- We activated the **Transport Importer**.



We did not change any other settings.

For the *Workplace WP1* we typed in a **Loading Time** of 1.5 seconds.

.Models.WorkerChart.WP1

Navigate View Tools Help

Name WP1

Label

Attributes Times Controls User-defined

DDD:HH:MM:SS.XXXX

Loading Time	Const	0:01.5
Unloading Time	Const	0
Recovery Time	Const	0:01

We did not change any other settings.

Settings for Station2 and WP2

For the *Workplace WP2* attached to *Station2*, to which the *Worker* carries the part and places it, we typed in an **Unloading Time** of 2 minutes and a **Recovery Time** of 1 second.

.Models.WorkerChart.WP2

Navigate View Tools Help

Name WP2

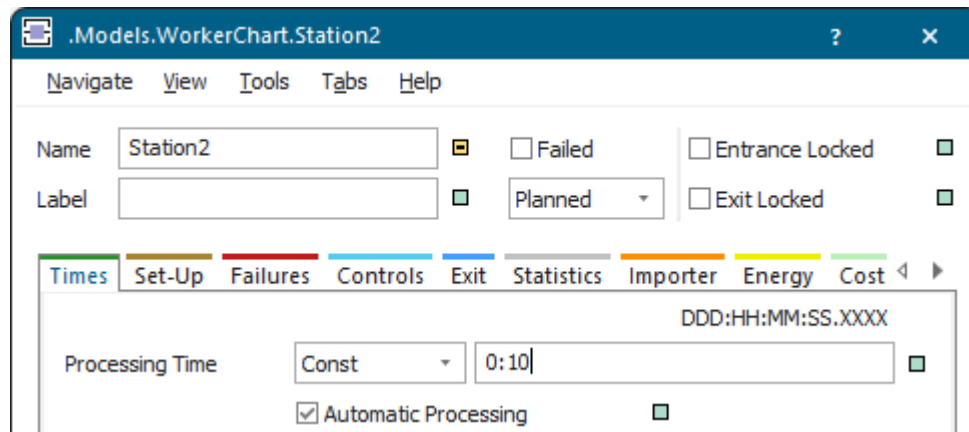
Label

Attributes Times Controls User-defined

DDD:HH:MM:SS.XXXX

Loading Time	Const	0
Unloading Time	Const	0:02
Recovery Time	Const	0:01

For *Station2*, which further processes the part, before sending it to the *Drain*, we typed in a **Processing Time** of 10 seconds.



Go to [Configure the WorkerPools for the Chart](#)

Go to [Configure the Chart and View Worker Statistics](#)

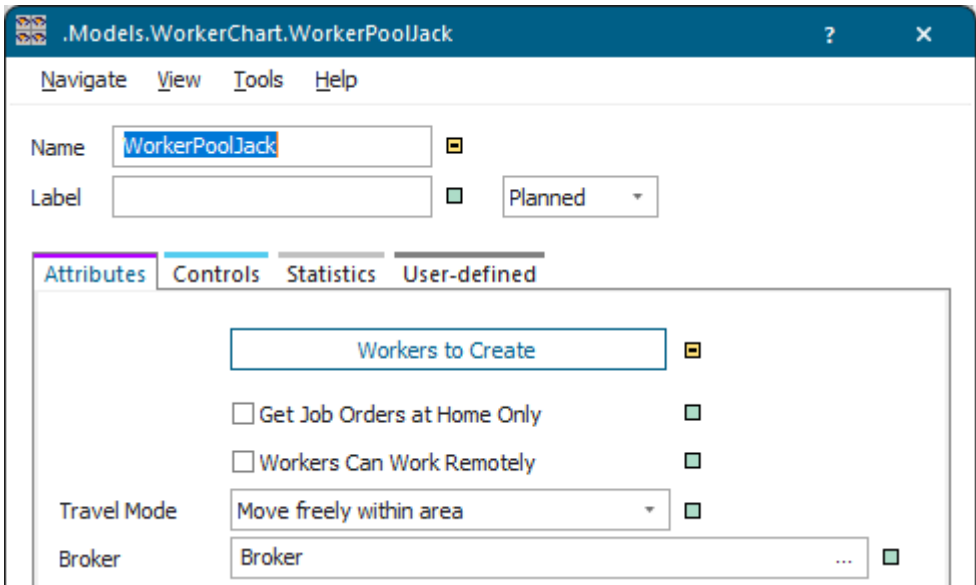
Back to [Show Worker Statistics in a Chart](#)

Configure the WorkerPools for the Chart

In this step we configure the *WorkerPools* whose *Workers* we want to show in the *Chart*.

WorkerPool and Workplace for Jack

For our *Worker* named **Jack** we clicked **Workers to Create**.



We dragged **Jack** from the folder **UserObject** to the cell **Worker** and dropped it there. We also typed 2 into the cell **Amount** and the *Workplace WpHomeJack* into the cell **Home Location**.

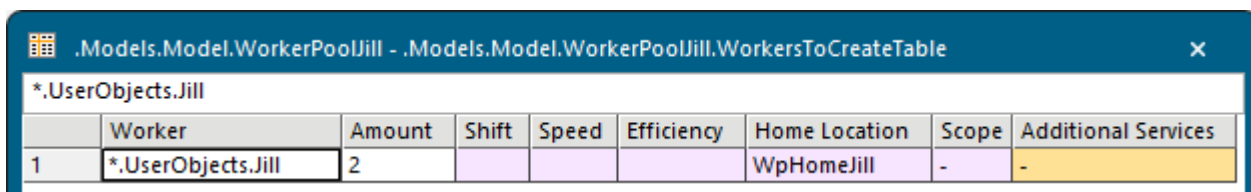
	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	*.UserObjects.Jack	2				WpHomeJack	-	-

We typed in 5 as the **Capacity** of *WpHomeJack*.

We did not change any other settings.

WorkerPool and Workplace for Jill

For our *Worker* named **Jill** we selected the same settings for her *WorkerPool* and *Workplace* as for **Jack**.



The screenshot shows a software window with a title bar that reads ".Models.Model.WorkerPoolJill - .Models.Model.WorkerPoolJill.WorkersToCreateTable". Below the title bar is a search bar containing the text "*UserObjects.Jill". Below the search bar is a table with the following columns: Worker, Amount, Shift, Speed, Efficiency, Home Location, Scope, and Additional Services. The table contains one data row with the following values: Worker: "*UserObjects.Jill", Amount: 2, Shift: (empty), Speed: (empty), Efficiency: (empty), Home Location: WpHomeJill, Scope: -, and Additional Services: -.

	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	*UserObjects.Jill	2				WpHomeJill	-	-

Back to [Configure the Stations and the Workplaces](#)

Go to [Configure the Chart and View Worker Statistics](#)

Back to [Show Worker Statistics in a Chart](#)

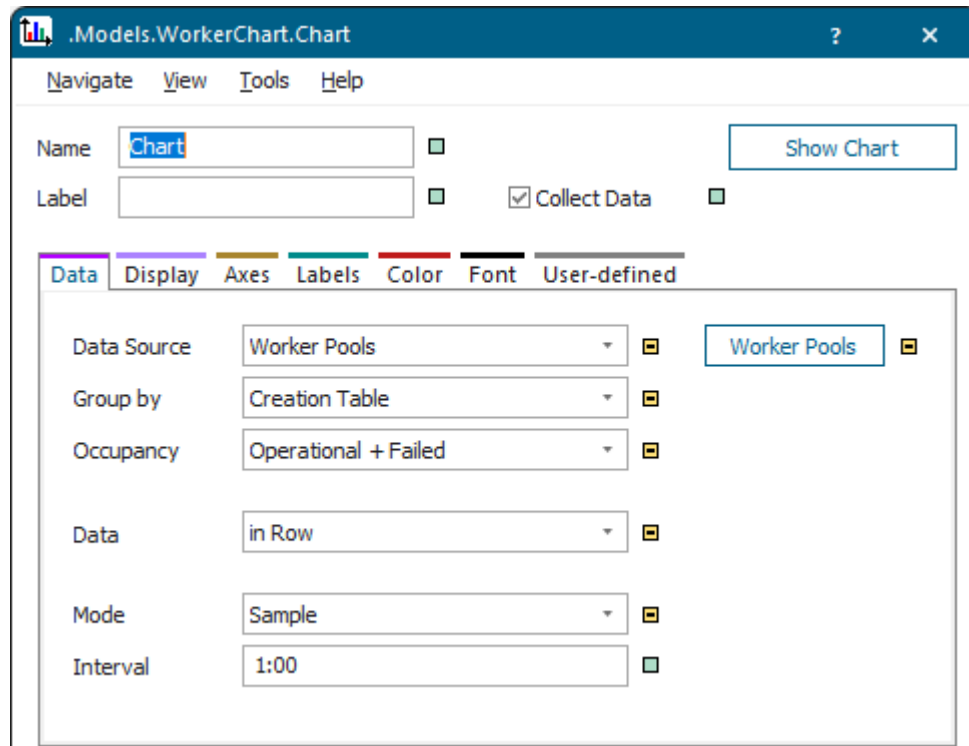
Configure the Chart and View Worker Statistics

In this step we configure the *Chart*, run the simulation, and view *Worker statistics*.

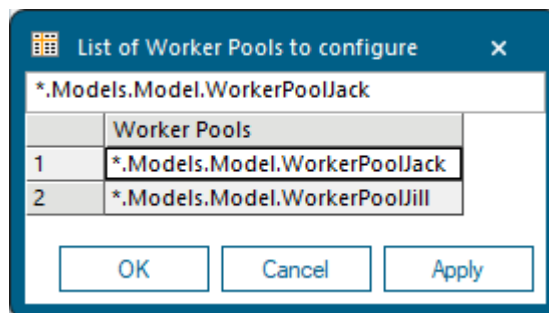
Configure the Chart

For the *Chart* we selected these settings:

- We selected **Worker Pools** as the **Data Source**.



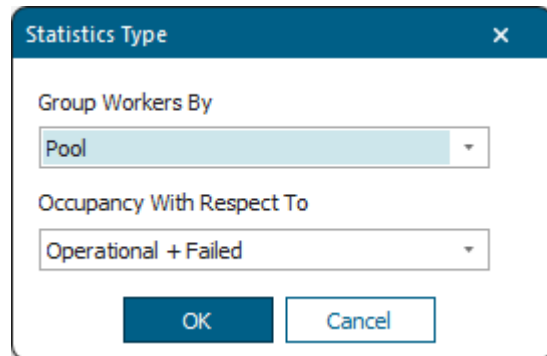
We clicked **Worker Pools** and dragged *WorkerPoolJack* and *WorkerPoolJill* into the cells of the table **List of Worker Pools to configure** to configure.



- We **group** the *Workers* according to the **Creation Table**.
- We selected **Operational + failed** as the **Occupancy**.
- We selected **Sample Mode** and accepted the default **Interval** of **1** minute.

Note

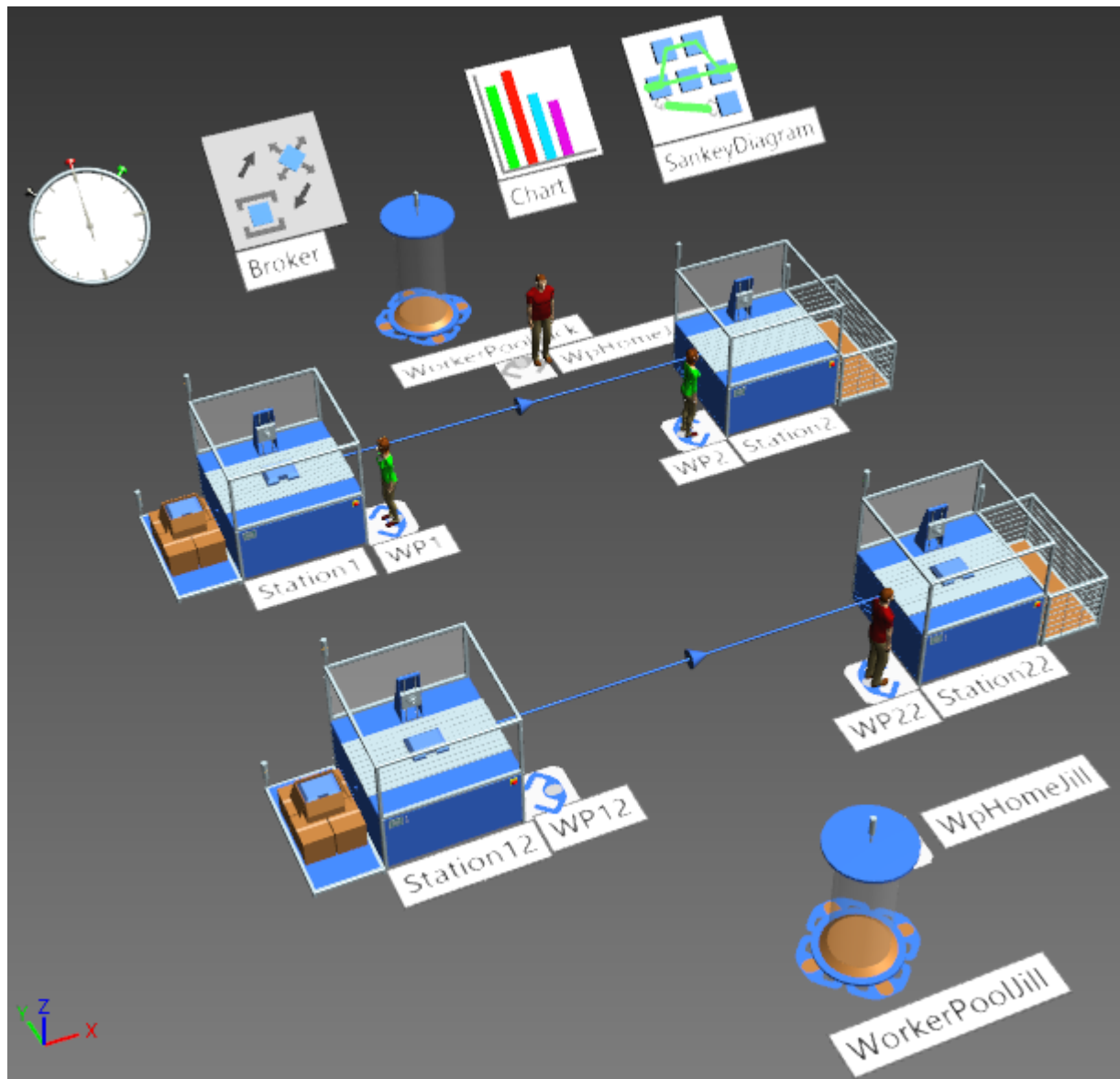
To initially add the *WorkerPools* to the *Chart*, you can select them, drag them over the icon of the *Chart* and drop them there.



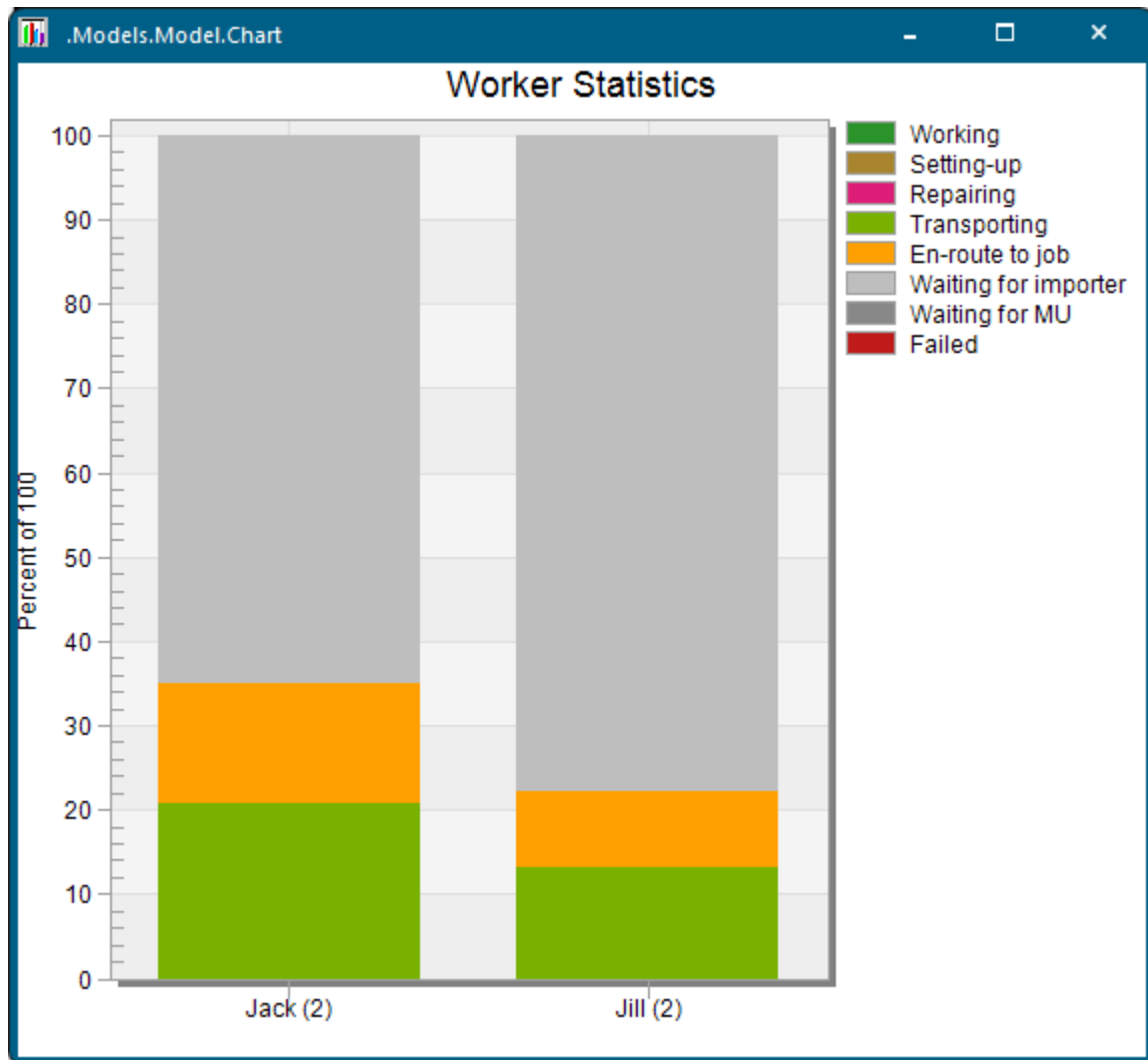
Then continue configuring the *Chart* on the tab **Data**.

Run the Simulation and View Statistics

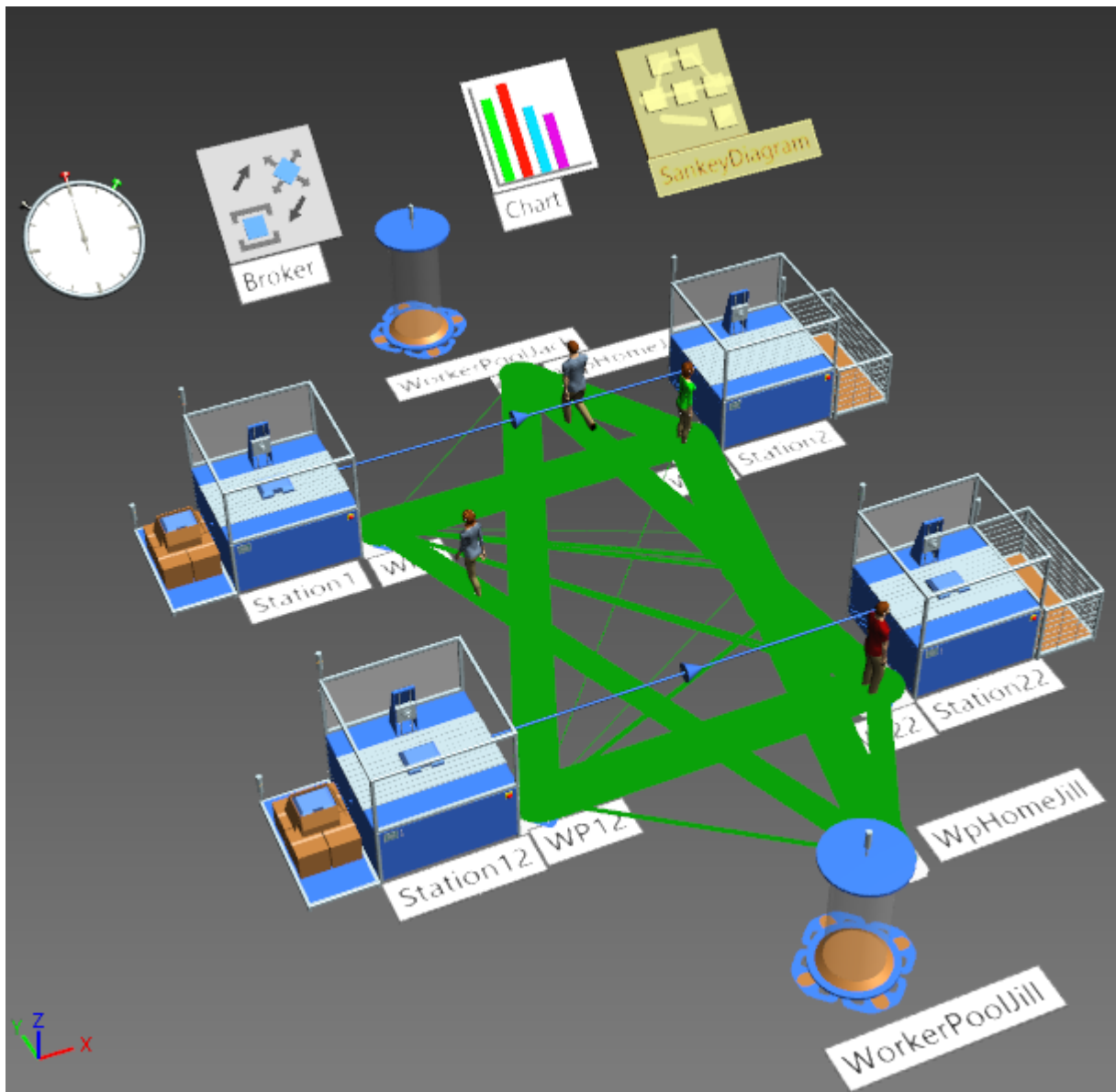
Run the simulation and watch how the *Workers* named **Jack** and **Jill** carry the finished parts between the *Stations*.



Right-click the *Chart* and select **Show** to show **Worker Statistics** in the diagram.



The *SankeyDiagram* shows the *Sankey* flows of the *Workers*.



[Back to Configure the Stations and the Workplaces](#)

[Back to Configure the WorkerPools for the Chart](#)

[Back to Show Worker Statistics in a Chart](#)

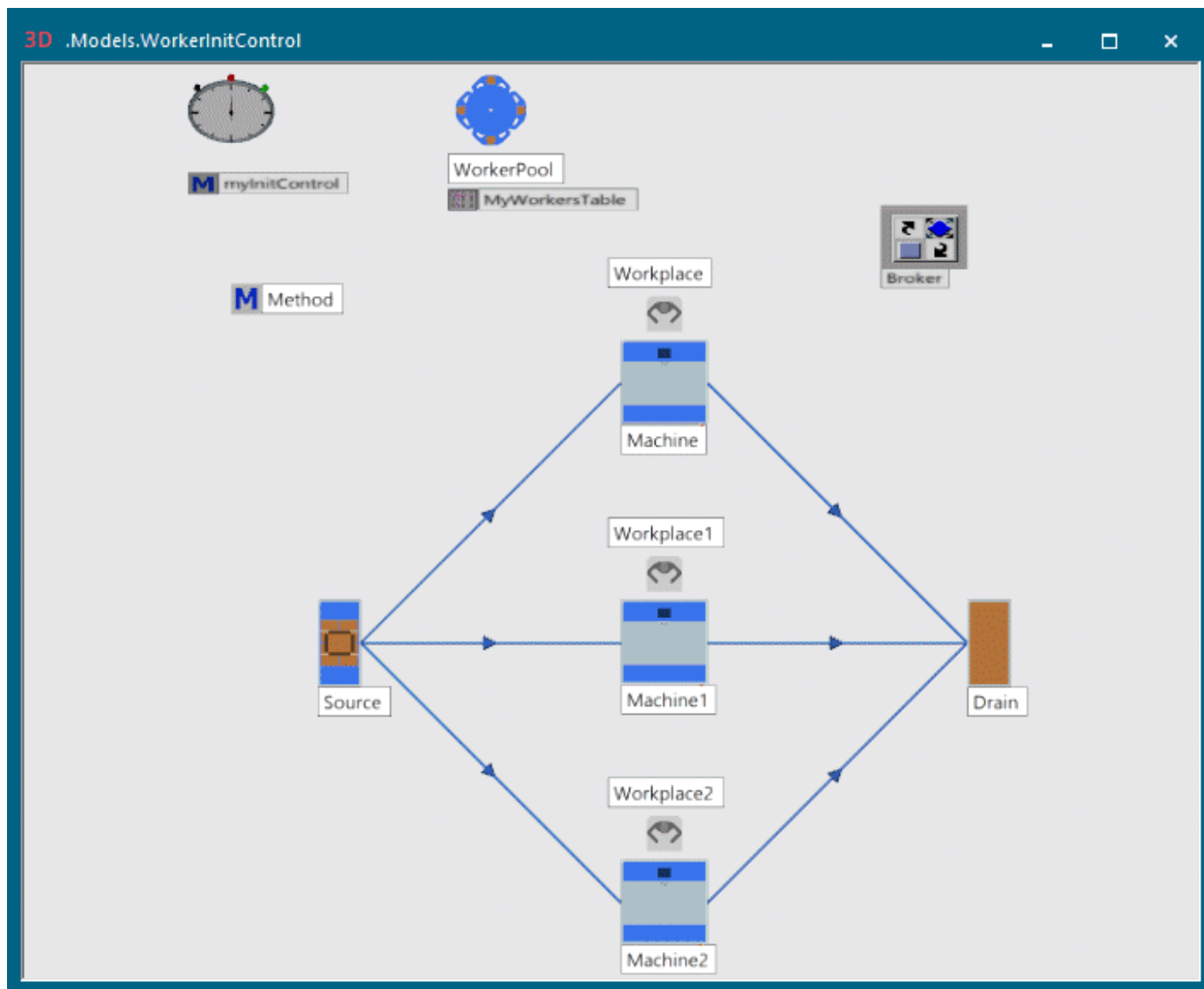
Set How Many Workers Are Created During Initialization

You can set how many *Workers* are created during initialization.

At times, you may want to change the **Workers to Create** of the **WorkerPool** for the next simulation run. You can accomplish this with the **Init Control** of the *EventController*. *Plant Simulation* calls the **init control**

once at the beginning of the simulation run during the *init* phase **before** the **objects** in the simulation model **are initialized** and before any **init methods** are executed.

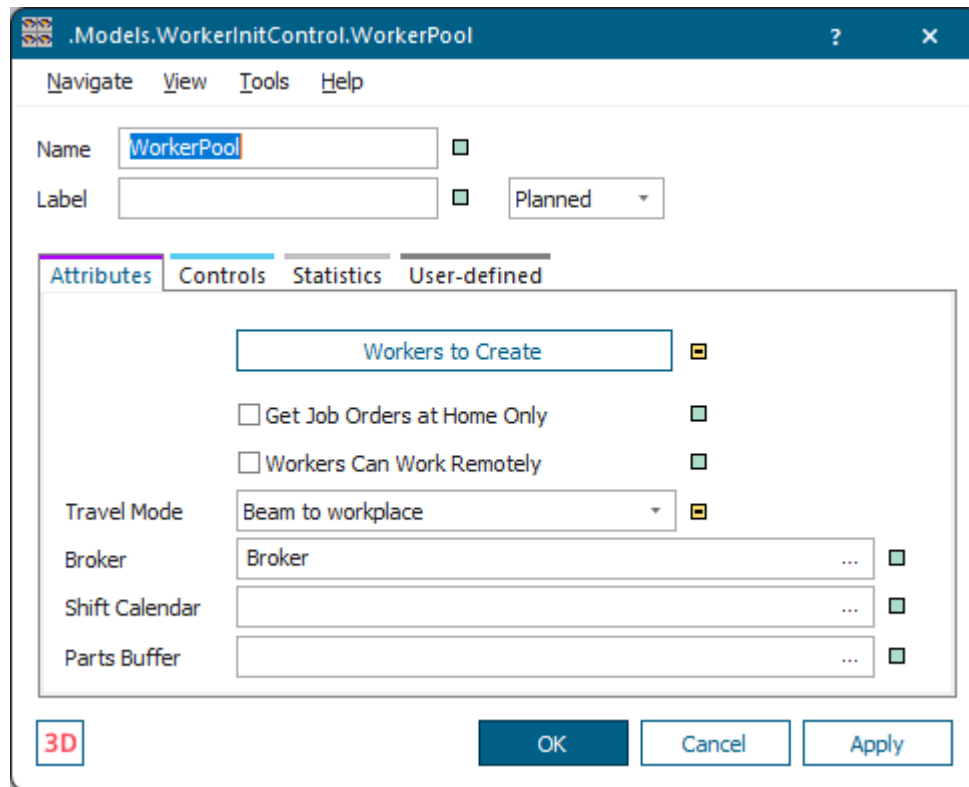
The finished simulation model to demonstrate this looks like this:



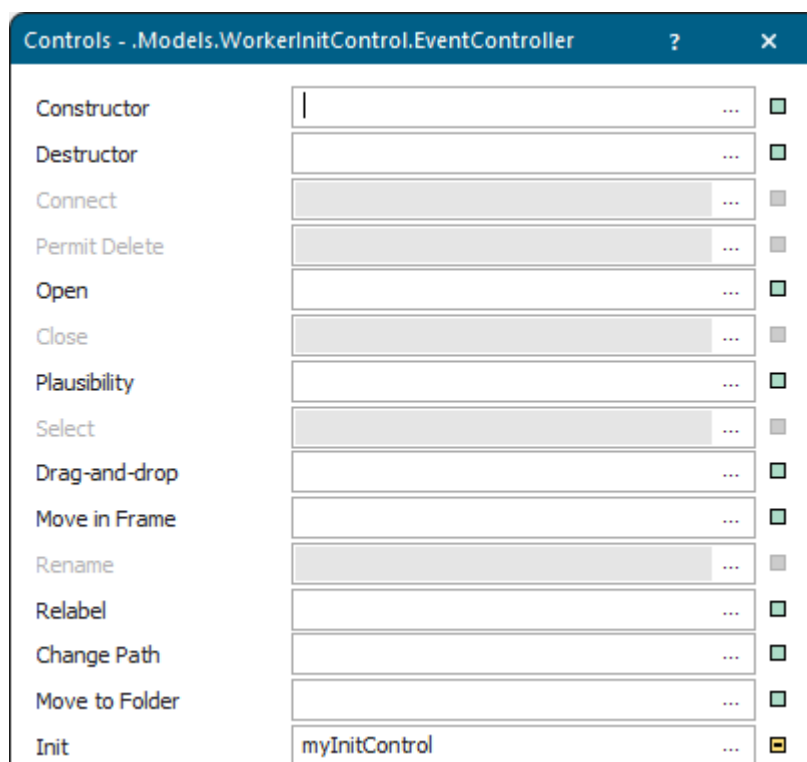
To create the model, proceed like this:

- Insert the required material flow and resource objects.

As we do not use *Footpaths* on which the *Workers* walk to the *Workplaces*, select **Beam to workplace**. This way they are teleported from the *WorkerPool* directly to the *Workplaces* without using up any time at all.



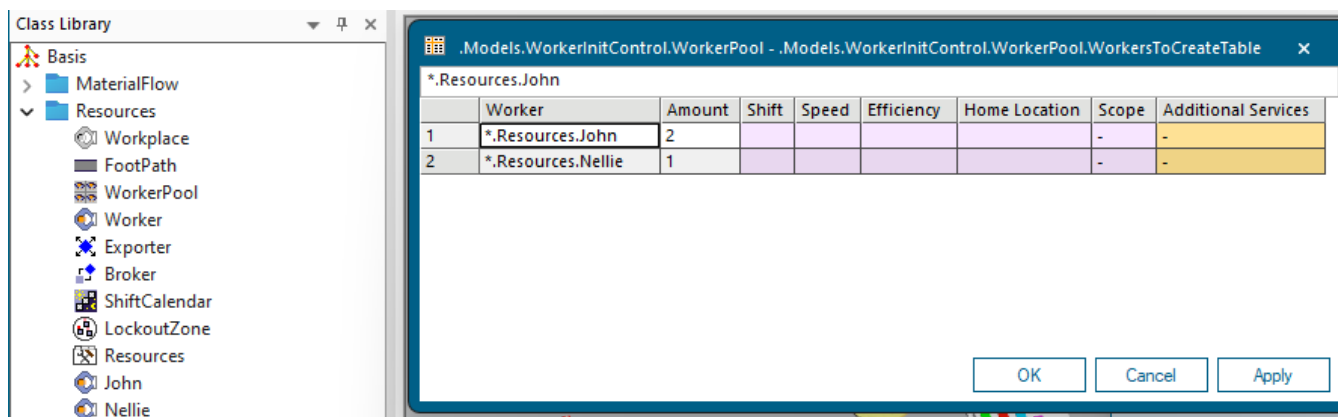
- Insert a *Method* into which you enter the source code of the **init control**. We named our method `myInitControl`.
- Double-click the *EventController*, select **Tools > Edit Controls**, click , and select the method which you just inserted as the **init control**.



- Insert a *DataTable* into which you enter the information about the *Workers* to be created. We named our *DataTable* *MyWorkersTable*. Format the table as shown in the screenshot below.

	object 1	integer 2	string 3	real 4	real 5	string 6	string 7	string 8
string	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1								
2								
3								

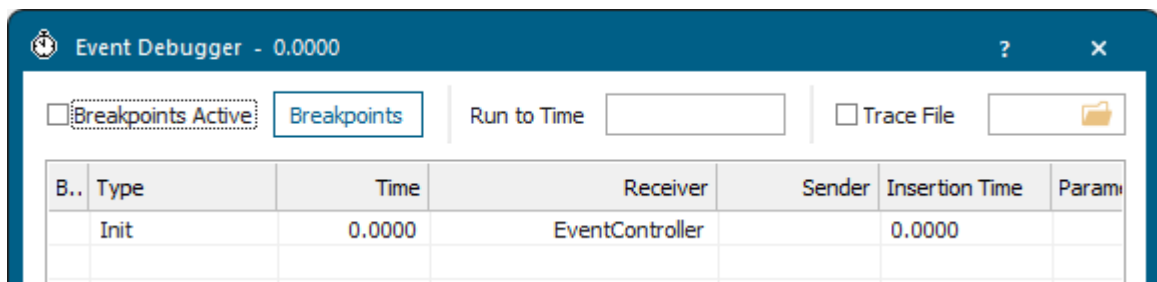
- Drag the *Worker*, which you want to create, from the *Class Library* to the first cell of the column **Worker**, and drop it there. We want to create the *Workers* **John** and **Nellie**.



- Program how many *Workers* you want to create. Double-click the **init control**, *myInitControl* in our case, and enter the source code. Our instruction `[2,1] := 2` enters **2** into the first cell the column **Amount** of the table *MyWorkersTable*. This creates two instances of our *Worker* named **John**. Our second instruction `[2,2] := 1` enters **1** into the second cell of the column **Amount**. This creates one instance of our *Worker* named **Nellie**. Finally, assign the table *MyWorkersTable* to the **Workers to Create** table of the *WorkerPool* with the method `setWorkersToCreateTable`.

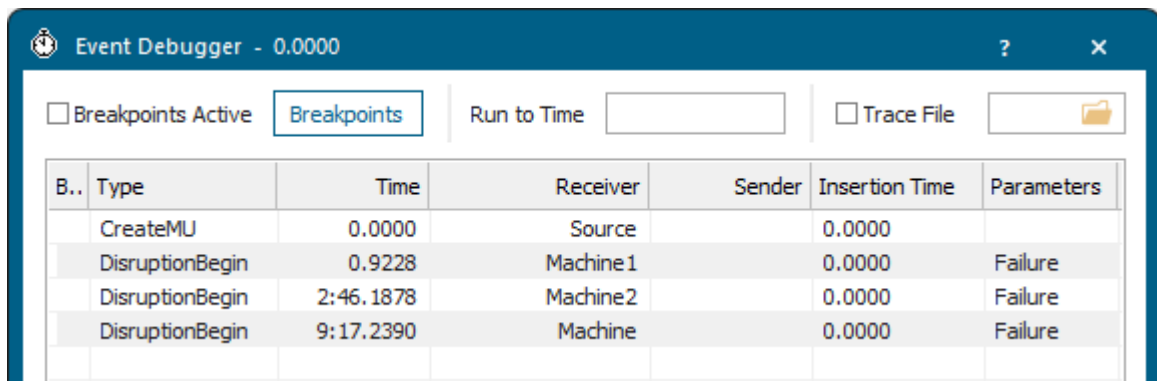
```
MyWorkersTable[2,1] := 2 // enters the desired amount of the worker John
MyWorkersTable[2,2] := 1 // enters the desired amount of the worker Nellie
WorkerPool.setWorkersToCreateTable(MyWorkersTable)
```

- Click  in the **EventController** to open the **List of scheduled events**. You'll see the first event to be processed is the **Init** event.

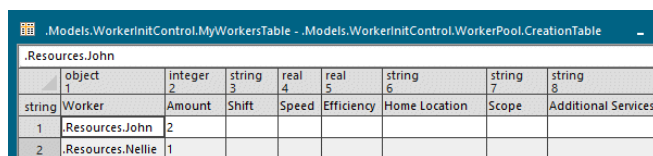


B..	Type	Time	Receiver	Sender	Insertion Time	Param
	Init	0.0000	EventController		0.0000	

When you click , *Plant Simulation* processes the event, enters the numbers of the *Workers* it created into the column **Amount** of the **Workers to Create** table and places the graphics of the *Workers* onto the *WorkerPool*.



B..	Type	Time	Receiver	Sender	Insertion Time	Parameters
	CreateMU	0.0000	Source		0.0000	
	DisruptionBegin	0.9228	Machine1		0.0000	Failure
	DisruptionBegin	2:46.1878	Machine2		0.0000	Failure
	DisruptionBegin	9:17.2390	Machine		0.0000	Failure



	object	integer	string	real	real	string	string	string
	1	2	3	4	5	6	7	8
string	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	.Resources.John	2						
2	.Resources.Nellie	1						



[Back to Model Workers and the Jobs They Do](#)

Model Workers with Importer, Broker and Exporter

Model Workers with Importer, Broker, and Exporter

You can model *Workers* with *Importer*, *Broker*, and *Exporter*.

Instead of using the *Worker*, you can also work with the **Broker–Importer–Exporter** mechanism to simulate the jobs, which a single person or a group of people do.

Using this mechanism encompasses these basic steps:

- Telling the station which services are required for doing a job. You accomplish this by typing in the names of the **services** into the services list on the sub-tabs of the tab **Importer**.
- Telling the station which *Broker* assigns these services to it. You accomplish this by specifying the name of this *Broker* in the text box **Broker** on the sub-tabs of the tab **Importer**.
- Telling an *Exporter* which services it is to supply for the stations in your simulation model. You accomplish this by typing in the names of the **services** into the **services** list on the tab **Attributes**.
- Telling the *Exporter* which *Broker* manages the services it provides. You accomplish this by specifying the name of this *Broker* in the text box **Broker** on the tab **Attributes**.

As a rule, you will use *Importer*, *Broker* and *Exporter* to model *Workers*, and the tools they need to do their jobs, if producing a part requires shared resources.

We demonstrate how to:

- [Model Processing Jobs](#)
- [Model Processing and Set-up Jobs](#)

[Back to Model Workers and the Jobs They Do](#)

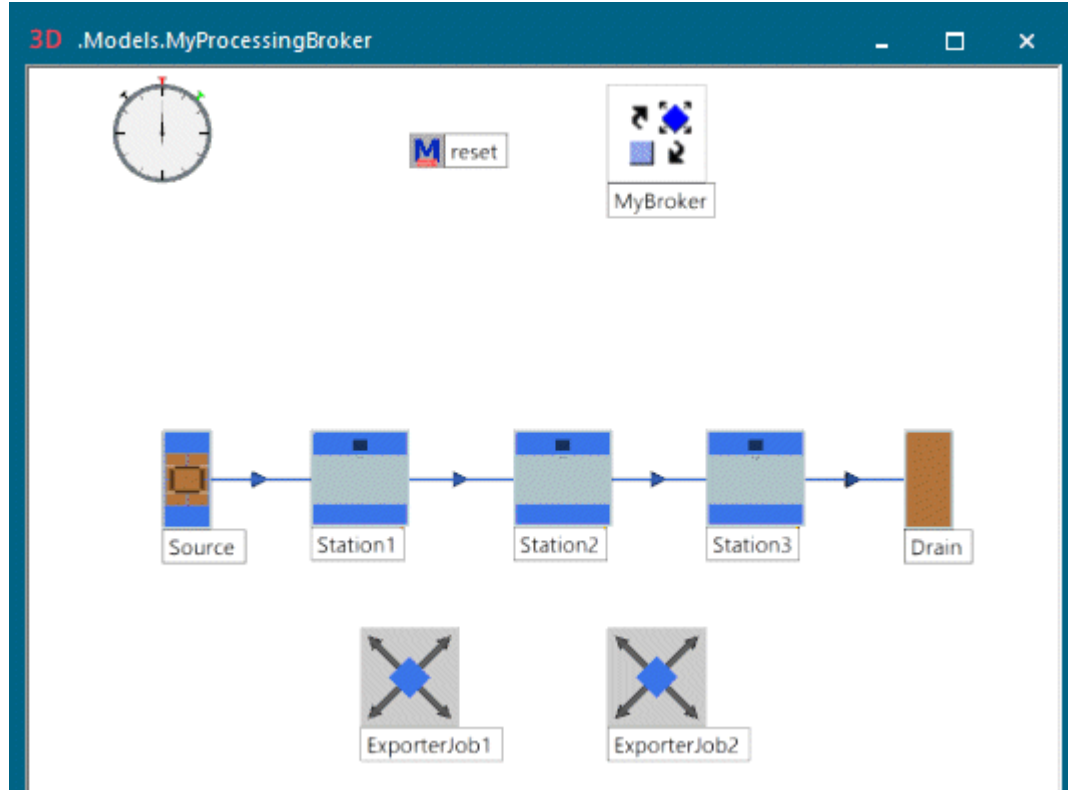
Model Processing Jobs

You can model jobs, which process parts, with *Importers* and *Exporters*.

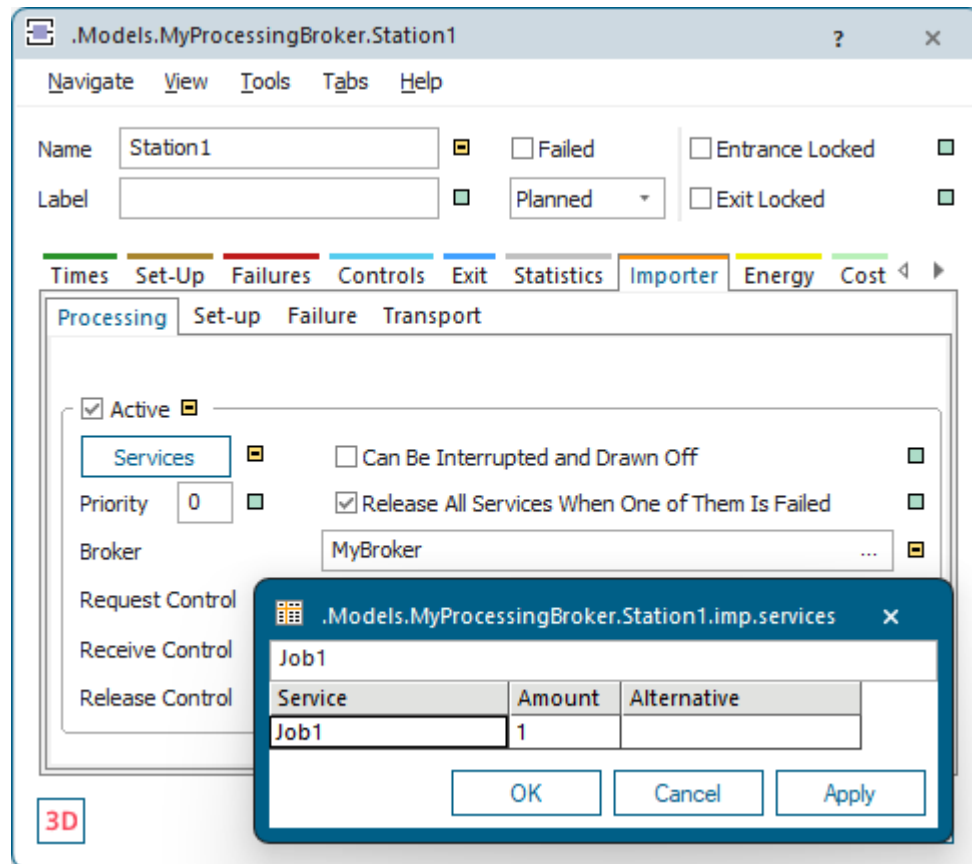
The **Exporter**  provides the services, while the **Broker**  assigns the *Exporter* to the station, which we model with the object **Station** . The stations *Station1* and *Station3* both require the *Exporter* named *ExporterJob1*, which provides the service Job1. The station *Station2* requires the service Job2, which the *ExporterJob2* provides.

To model this, we proceed as follows:

- Insert a *Source*, three *Stations*, and a *Drain*. Connect them. Then, insert two *Exporters* and a *Broker*. Finally, insert a *Method*, name it `reset`, and enter `deleteMovables` as source code.



- Open *Station1* and click the tab **Importer**. Click the **Sub-tab Processing**.
 - Select the check box **Active**.
 - Make sure that the check box **Common Resources** on the **Sub-tab Set-up** is selected. Click the **sub-tab Processing** again. Deactivate inheritance for the resources: Click the check box to the right of **Services**, so that it looks like this .



Click **Services** and enter Job1 into the column **Service** of the service list.

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

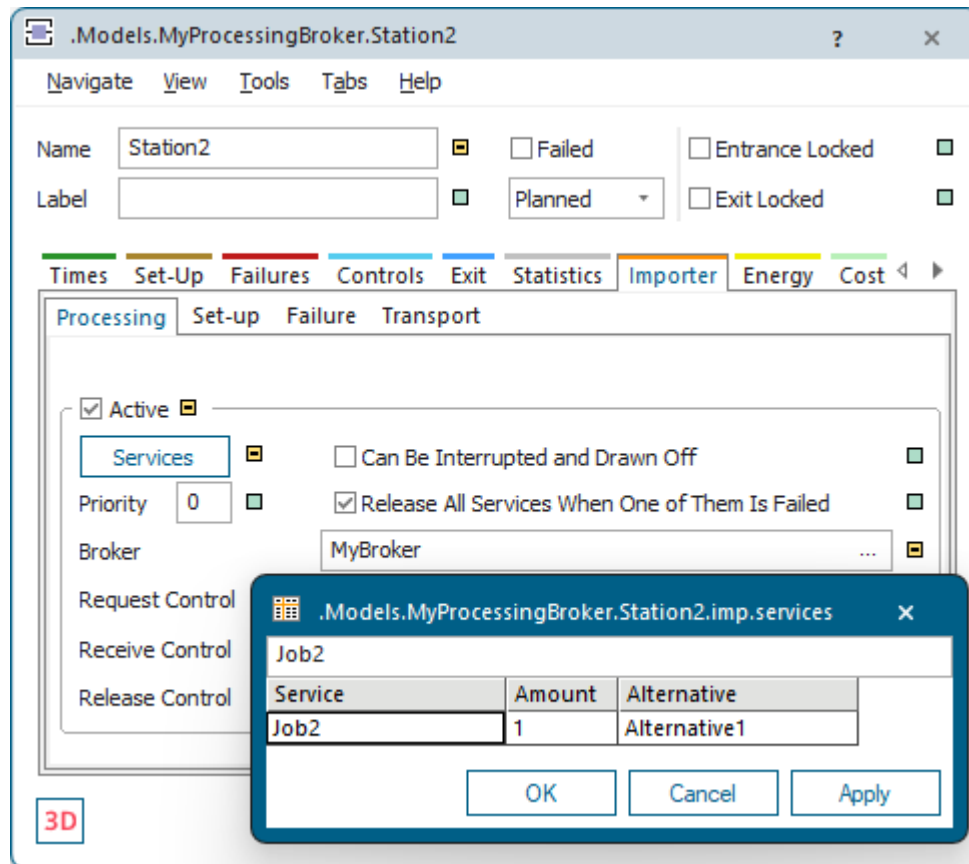
In *SimTalk* you can compare strings in a case-insensitive manner with the `~=` operator, compare [Relational Operators](#).

Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Open *Station2* and click the tab **Importer**. Click the **Sub-tab Processing**.
 - Select the check box **Active**.
 - Make sure that the check box **Common Resources** on the **sub-tab Set-up** is selected. Click the **sub-tab Processing** again. Deactivate inheritance for the resources: Click the check box to the right of **Services**, so that it looks like this .

Click **Services** and enter Job2 into the column **Service** of the service list.

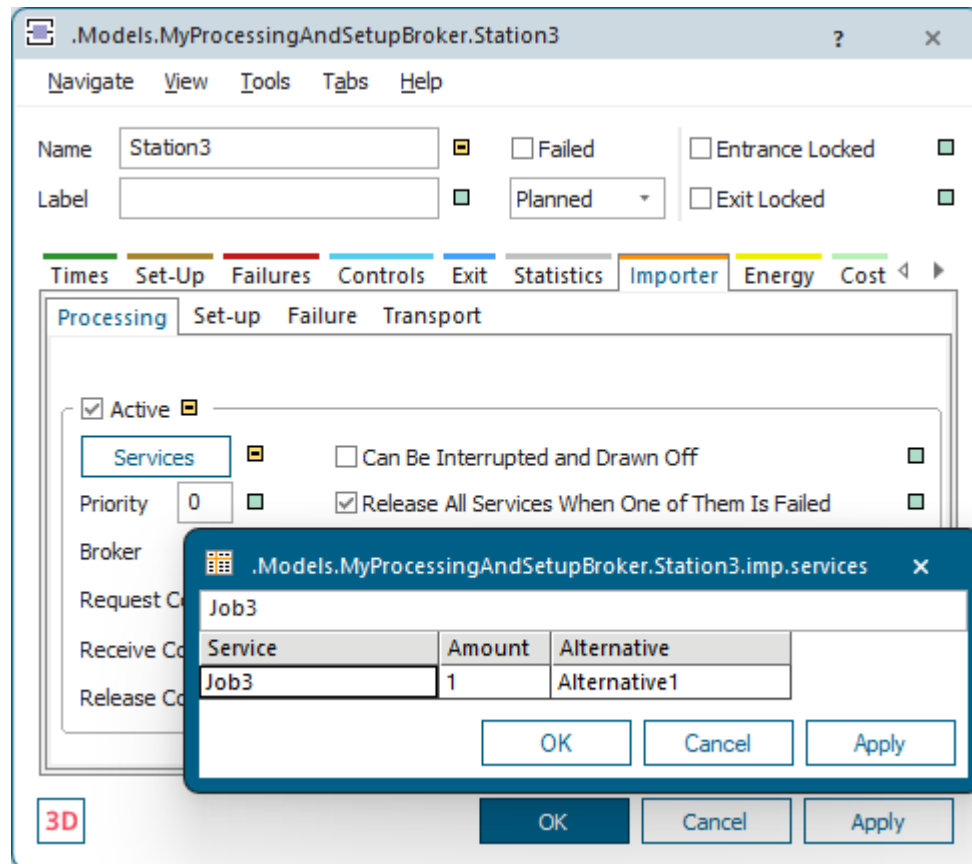
Click  and select the *Broker*, which procures the services, *MyBroker* in our example.



- Open *Station3* and click the tab **Importer**. Click the **Sub-tab Processing**.
 - Select the check box **Active**.
 - Make sure that the check box **Common Resources** on the **sub-tab Set-up** is selected. Click the **sub-tab Processing** again. Deactivate inheritance for the resources: Click the check box to the right of **Services**, so that it looks like this ☐.

Click **Services** and enter Job1 into the column **Service** of the service list.

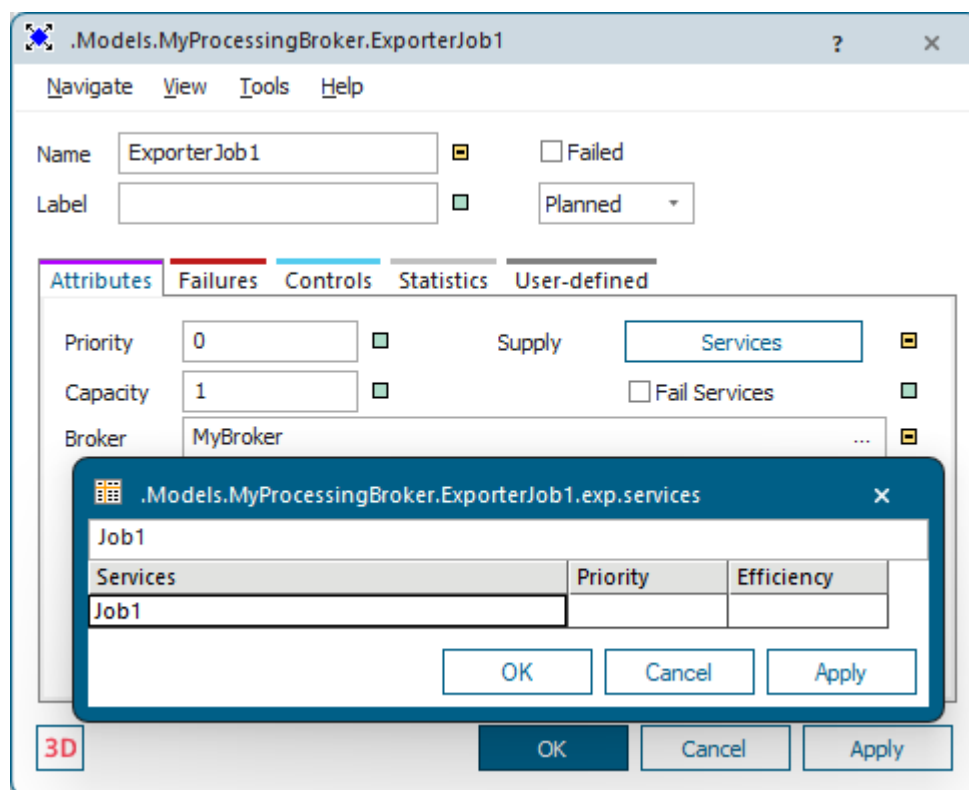
Click and select the *Broker*, which procures the services, *MyBroker* in our example.



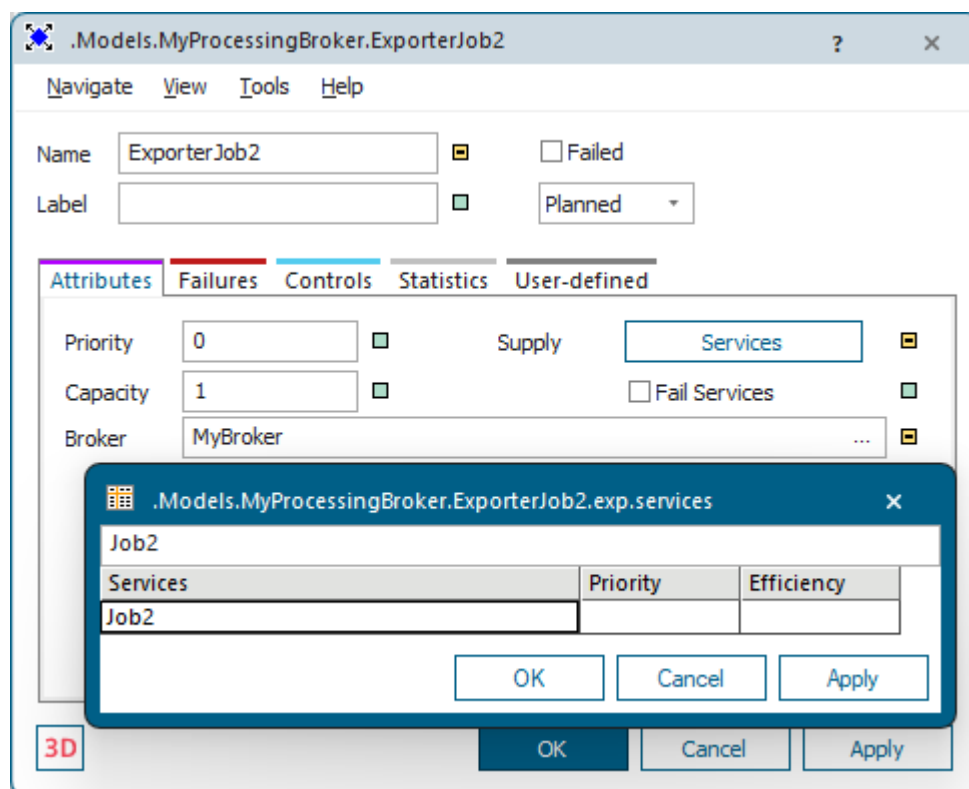
- Next, we have to tell the *Exporters* which services they have to provide and which *Broker* procures the services.

Open *ExporterJob1*.

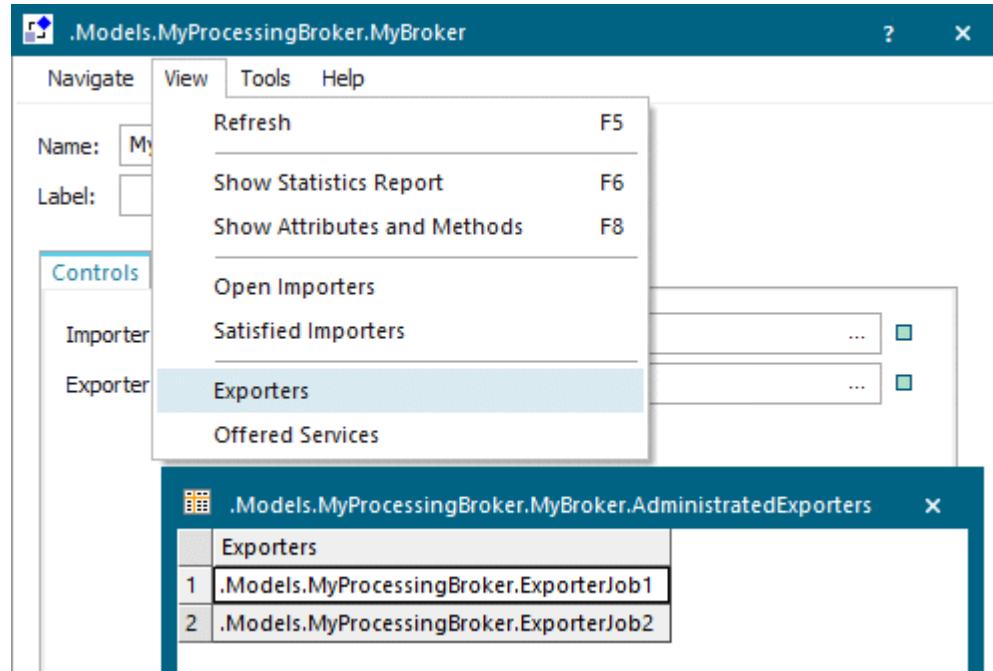
- Turn inheritance for the **Services** off: Click the check box to the right of **Services** so that it looks like this ☐. Click **Services** and enter Job1 into the column **Services** of the service list.
- Click and select the *Broker*, which procures the services, *MyBroker* in our example.



Repeat this for *ExporterJob2*. Enter Job2 into the column **Services** of the service list.



Plant Simulation automatically enters the *Exporters* and the services into the *Broker* which we specified. To view them, open *MyBroker* and select **View > Exporters** and **View > Offered Services**.



- Finally, open the *EventController* and click **Start/Stop Simulation**. As the *Exporters* can only work at one station at a time, and because the stations *Station1* and *Station3* share the *ExporterJob1*, short standstills occur. *Station1* has to wait until *Station3* releases *ExporterJob1*, and thus is blocked. *Station3* in turn has to wait until *Station1* releases *ExporterJob1*, and thus is **Waiting**.

You can also click the tab **Statistics** and view the most important data, which the object collected.

Back to [Model Workers with Importer, Broker, and Exporter](#)

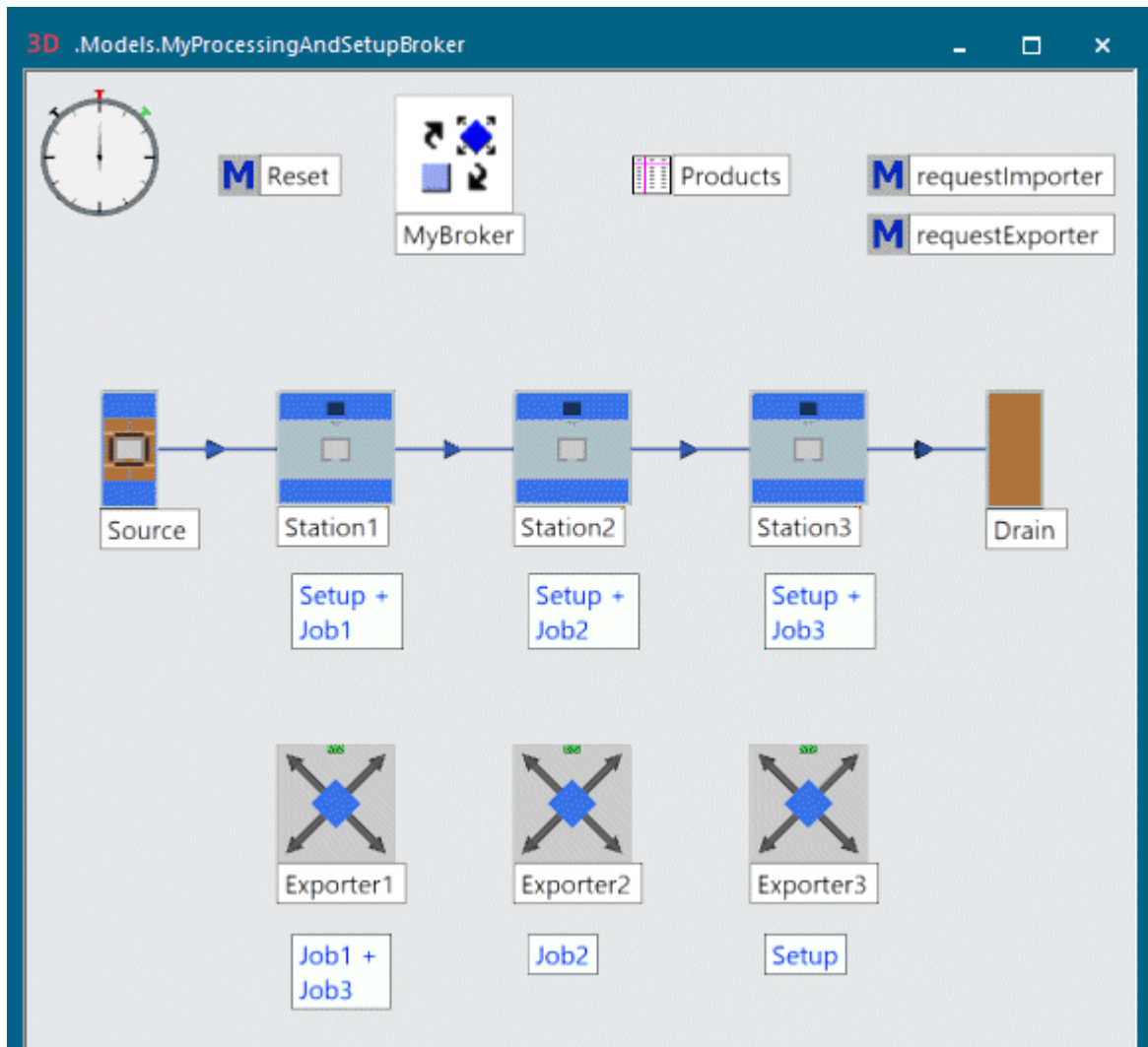
Model Processing and Set-up Jobs

You can model jobs, which process parts and which set the station up to process these different types of parts.

The **Exporter** provides the services, while the **Broker** assigns the *Exporter* to the station, which we here model with the object **Station**. All stations need the service named *Set up*, which *Exporter3* provides, for setting the station up for a new part type. *Station1* and *Station3* both require the *Exporter* named *Exporter1*, which provides the services *Job1* and *Job2*. The station *Station2* requires the service *Job2*, which *Exporter2* provides.

To model this, we proceed like this:

- Insert a *Source*, three *Stations*, and a *Drain*. Connect them. Then, insert three *Exporters*, a *Broker*, and a table, which contains the products. Finally, insert a *Method*, name it *reset*, and enter `deleteMovables` as source code.



- First we have to tell the *Source* which type of part and how many of them we want it to create. Open the *Source* and select **MU selection > Sequence Cyclical**. Click  next to **Table** and select the name of the table, which contains the products. This is the table, which we inserted above.

Name: ☒ ☐ Failed
 Label: ☒ Planned ☐ Exit Locked ☒

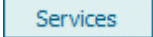
Attributes Failures Controls Exit Statistics Importer User-defined

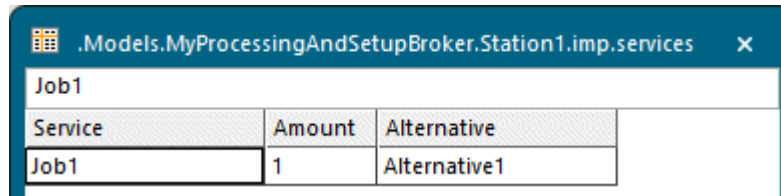
Operating Mode ☒ Blocking ☐
 Time of Creation ☒ Amount ☒
 DDD:HH:MM:SS.XXXX
 Interval ☒
 Start ☒
 Stop ☒
 MU Selection ☒ ☐ Generate as Batch ☒
 Table ☒

Double-click the table, **Products** in our example, and enter the name of the *MU* class into the column **MU**. Enter the amount of parts to be created into the column **Number**, and the name of the part into the column **Name**.

	object 1	integer 2	string 3	table 4
string	MU	Number	Name	Attributes
1	.MUs.Part	5	Part1	
2	.MUs.Part	8	Part2	
3				
4				
5				

- Open *Station1* and click the tab **Importer**.
 - Select the check box **Active** on the **Sub-tab Processing**.

Click the check box to the right of **Services**, so that it looks like this . Click **Services** and enter **Job1** into the column **Service**.



Service	Amount	Alternative
Job1	1	Alternative1

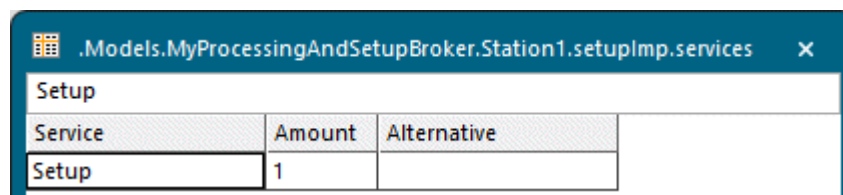
Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Click the **Sub-tab Set-up**.

Select the check box **Active**.

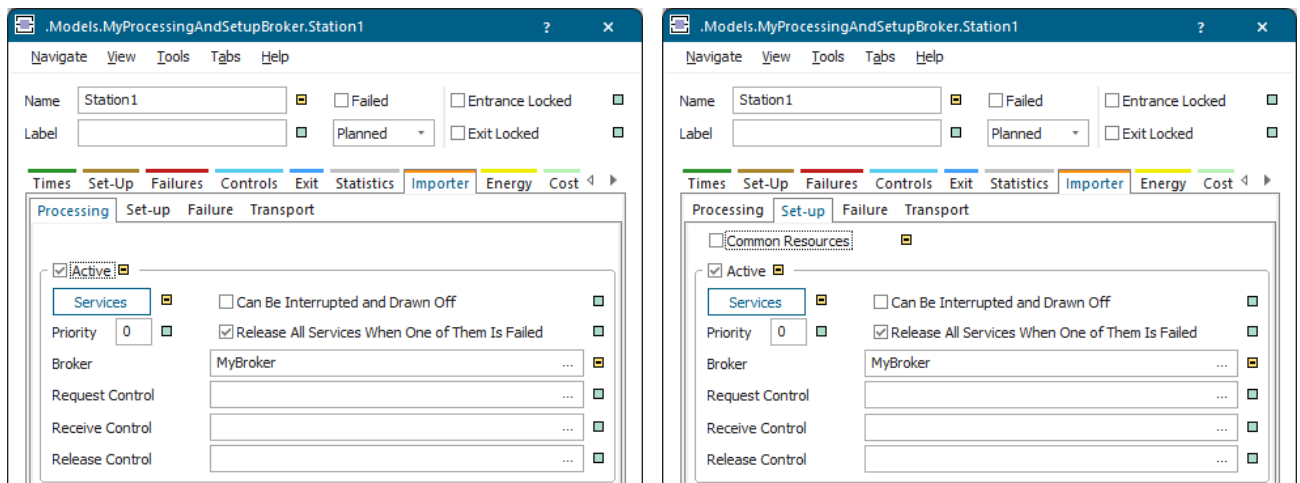
Clear the check box **Common Resources**.

Click **Services** and enter Setup into the column Service.



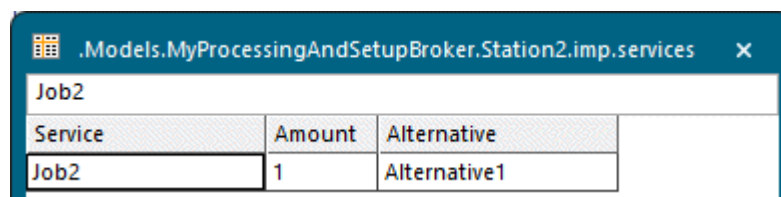
Service	Amount	Alternative
Setup	1	

Click  and select the *Broker*, which procures the services, *MyBroker* in our example.



- Open *Station2* and click the tab **Importer**.
 - Select the check box **Active** on the **Sub-tab Processing**.

Click the check box to the right of **Services**, so that it looks like this . Click **Services** and enter Job2 into the column **Service**.



Service	Amount	Alternative
Job2	1	Alternative1

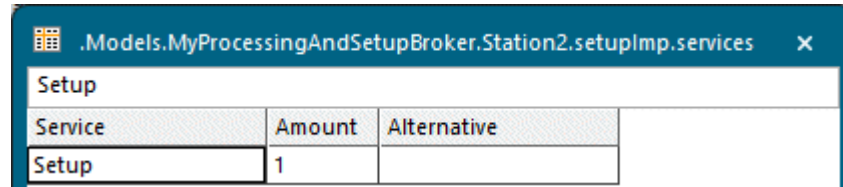
Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Click the **Sub-tab Set-up**.

Select the check box **Active**.

Clear the check box **Common Resources**.

Click **Services** and enter Setup into the column **Service**.

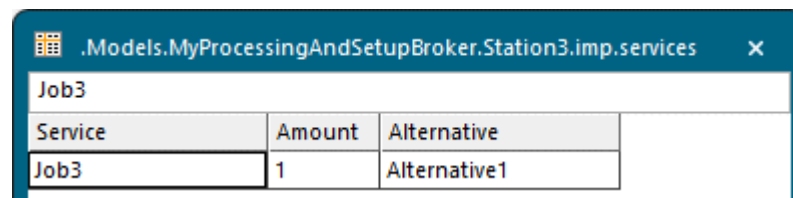


Service	Amount	Alternative
Setup	1	

Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Open *Station3* and click the tab **Importer**.
 - Select the check box **Active** on the **Sub-tab Processing**.

Click the check box to the right of **Services**, so that it looks like this . Click **Services** and enter Job3 into the column **Service**.

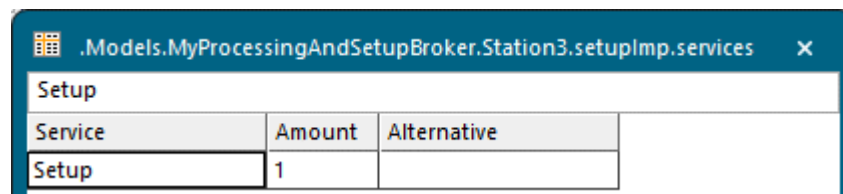


Service	Amount	Alternative
Job3	1	Alternative1

Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Click the **Sub-tab Set-up**.
- Clear the check box **Common Resources**.

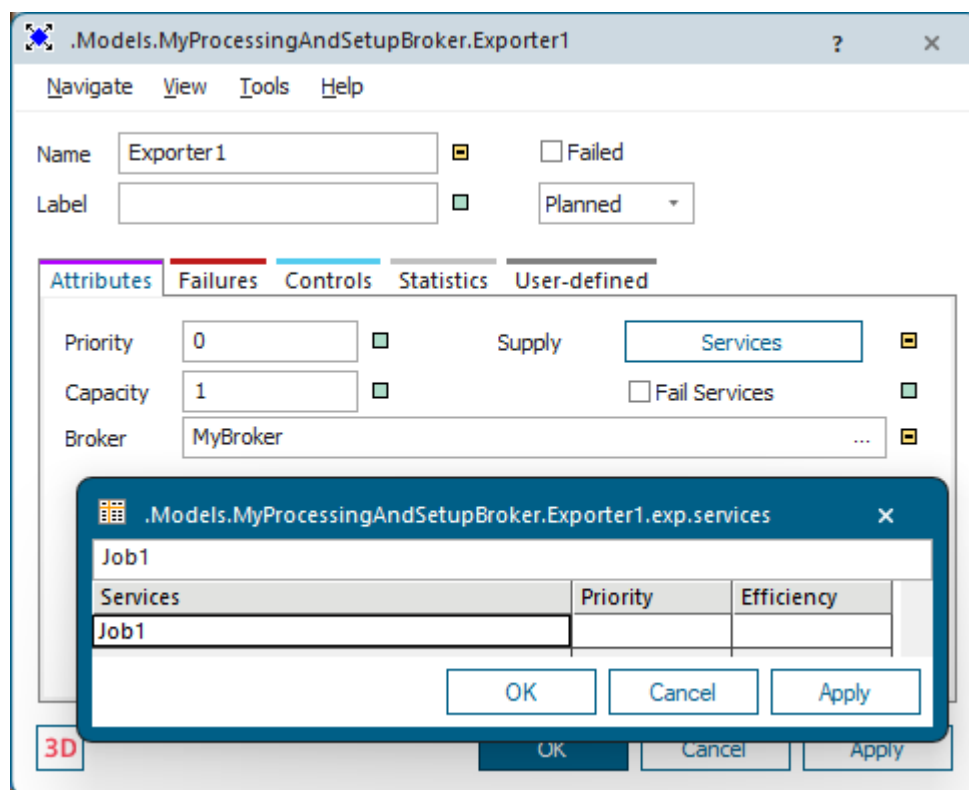
Click **Services** and enter Setup into the column **Service**.



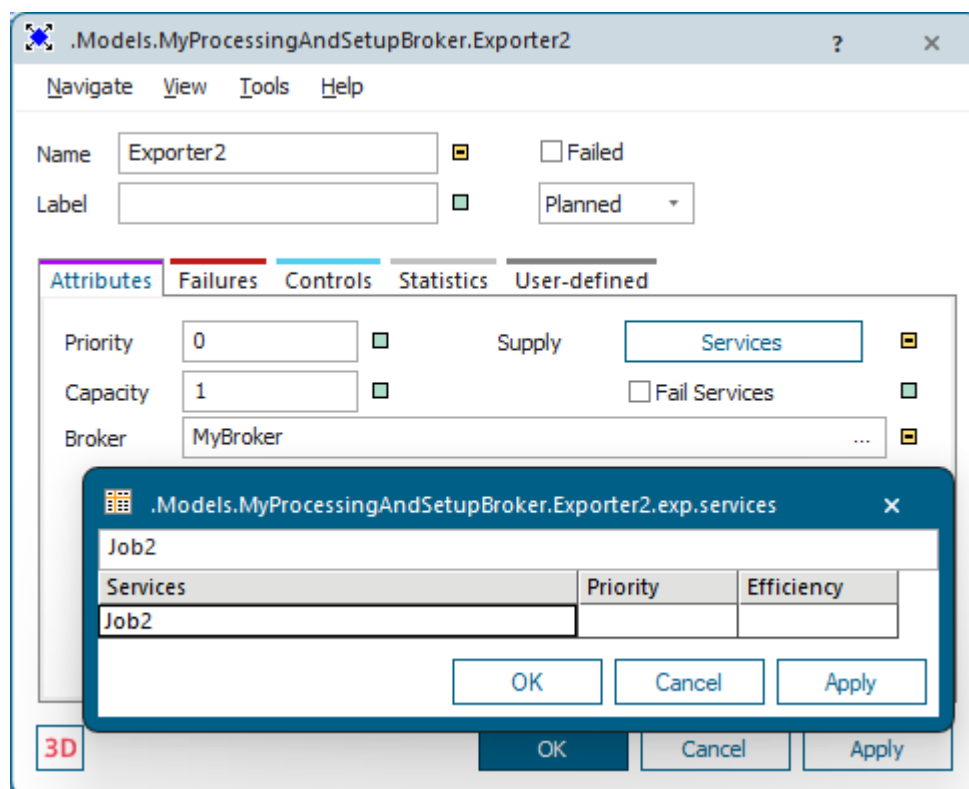
Service	Amount	Alternative
Setup	1	

Click  and select the *Broker*, which procures the services, *MyBroker* in our example.

- Next, we have to tell the *Exporters*, which services they have to provide and which *Broker* procures the services. Open *Exporter1*.
 - Turn inheritance for the **Services** off: Click the check box to the right of **Services** so that it looks like this . Click **Services** and enter Job1 and Job3 into the column **Services** of the service list.
 - Click  and select the *Broker*, which procures the services, *MyBroker* in our example.



Repeat this for *Exporter2*. Enter Job2 into the column **Services** of the service list.



Repeat this for *Exporter3*. Enter Setup into the column **Services** of the service list.